TECNOLOGIAS EMERGENTES EM ENGENHARIA DE SOFTWARE

EMERSON JOSÉ PORFÍRIO



EXPEDIENTE

Coordenador(a) de Conteúdo
Kleber Lopes Petry
Projeto Gráfico e Capa
Arthur Cantareli Silva
Editoração
Lilian Andreia Hasse
Design Educacional
Giovana Vieira Cardoso

Revisão Textual
Ariane Andrade Fabreti
Cindy Mayumi Okamoto Luca
Ilustração
Bruno Cesar Pardinho Figueiredo
Eduardo Aparecido Alves
Geison Ferreira da Silva
Wellington Vainer
Fotos
Shutterstock e Envato

FICHA CATALOGRÁFICA

N964 Núcleo de Educação a Distância. **PORFÍRIO,** Emerson José.

Tecnologias Emergentes em Engenharia de Software / Emerson José Porfírio. - Florianópolis, SC: Arqué, 2026.

184 p.

ISBN papel 978-65-5466-416-5
ISBN digital 978-65-5466-414-1

1. Tecnologias Emergentes 2. Engenharia de Software 3. EaD. I. Título.

CDD - 005.1

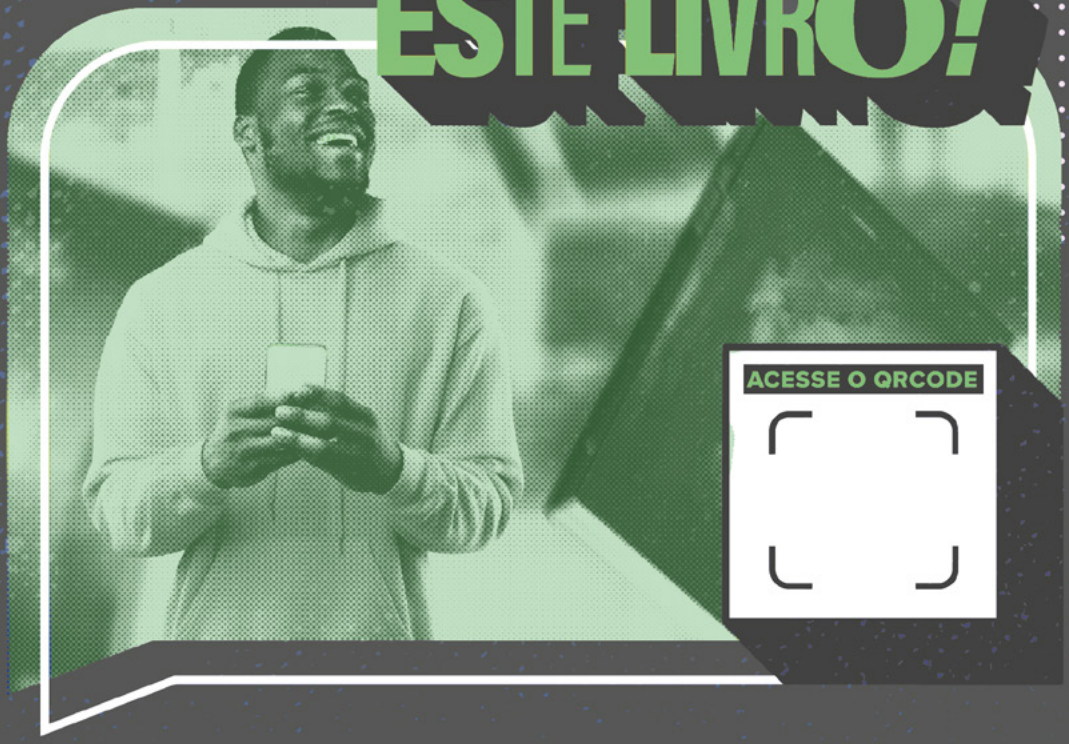
Bibliotecária: Leila Regina do Nascimento - CRB- 9/1722.

Ficha catalográfica elaborada de acordo com os dados fornecidos pelo(a) autor(a).

Ano de impressão:

Impresso por:

AVALIE ESTE LIVRO!



CRIAR MOMENTOS DE APRENDIZAGENS INESQUECÍVEIS É O NOSSO OBJETIVO E POR ISSO, **GOSTARÍAMOS DE SABER COMO FOI SUA EXPERIÊNCIA.**

Conta para nós! leva ***menos de 2 minutos*** *Vamos lá?!*

DIGITE O CÓDIGO

02512078

Aa

RESPONDA A PESQUISA

?

•••

RECURSOS DE IMERSÃO

PENSANDO JUNTOS

Este item corresponde a uma proposta de reflexão que pode ser apresentada por meio de uma frase, um trecho breve ou uma pergunta.

APROFUNDANDO

Utilizado para temas, assuntos ou conceitos avançados, levando ao aprofundamento do que está sendo trabalhado naquele momento do texto.

PRODUTOS AUDIOVISUAIS

Os elementos abaixo possuem recursos audiovisuais. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

PLAY NO CONHECIMENTO

Professores especialistas e convidados, ampliando as discussões sobre os temas por meio de fantásticos podcasts.

EU INDICO

Utilizado para agregar um conteúdo externo.

EM FOCO

Utilizado para aprofundar o conhecimento em conteúdos relevantes utilizando uma linguagem audiovisual.

ZOOM NO CONHECIMENTO

Utilizado para desmistificar pontos que possam gerar confusão sobre o tema. Após o texto trazer a explicação, essa interlocução pode trazer pontos adicionais que contribuam para que o estudante não fique com dúvidas sobre o tema.

INDICAÇÃO DE FILME

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de filmes que se conectam com o tema do conteúdo.



INDICAÇÃO DE LIVRO

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de livros que agregarão muito na sua vida profissional.



CAMINHOS DE APRENDIZAGEM

7

UNIDADE 1

O PARADIGMA DA FÁBRICA DE SOFTWARE 8

33

UNIDADE 2

MODELOS ORGANIZACIONAIS DE FÁBRICAS DE SOFTWARE 34

TIPOS DE SOFTWARE EM AMBIENTES INDUSTRIAIS 54

73

UNIDADE 3

LINHA DE PRODUTO DE SOFTWARE (SPL – SOFTWARE PRODUCT LINE). 74

AUTOMAÇÃO E VIRTUALIZAÇÃO EM FÁBRICAS DE SOFTWARE 90

109

UNIDADE 4

FÁBRICAS DE SOFTWARE IN-HOUSE E OUTSOURCING 110

GOVERNANÇA, QUALIDADE E PADRONIZAÇÃO. 126

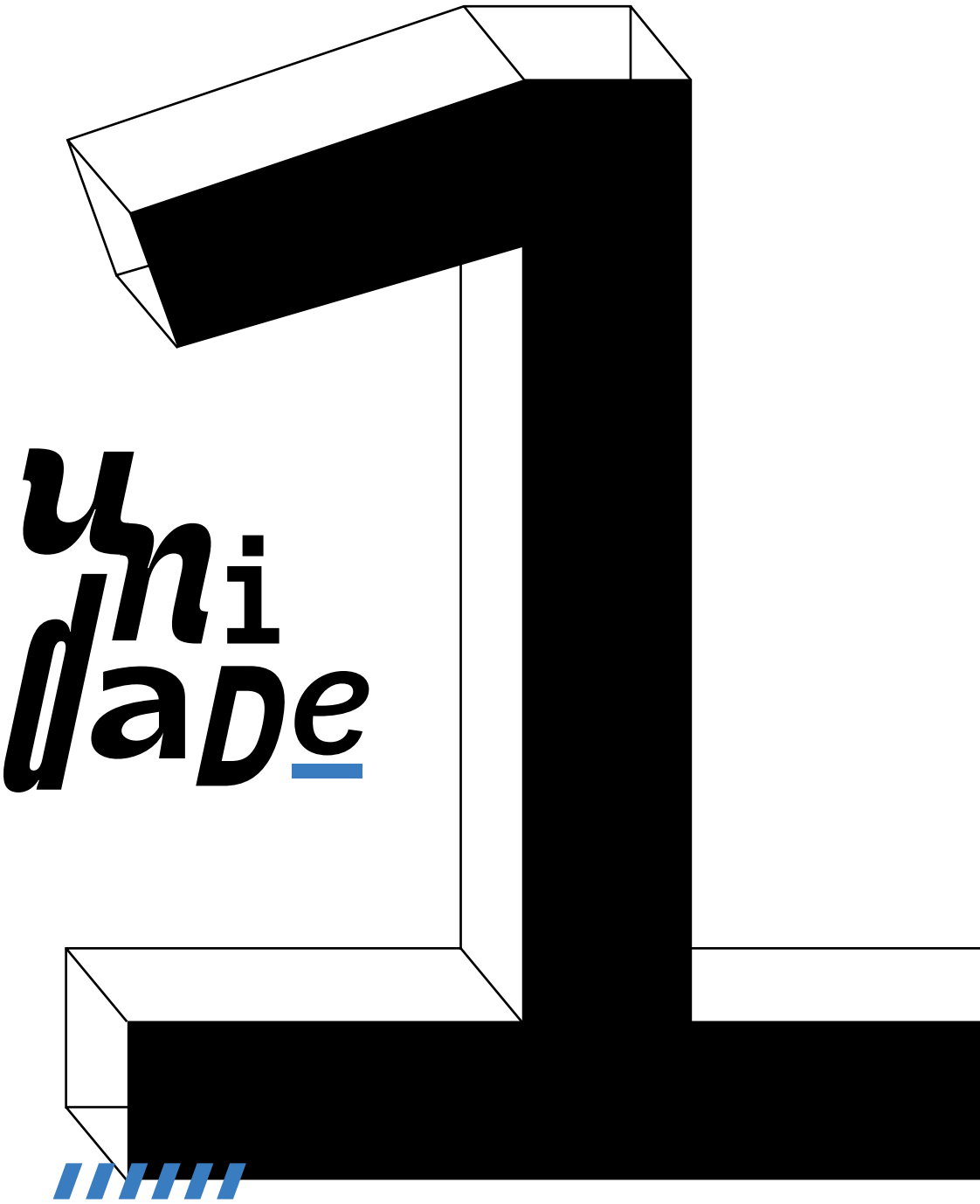
145

UNIDADE 5

TECNOLOGIAS EMERGENTES NO CONTEXTO DA FÁBRICA 146

TENDÊNCIAS E FUTURO DAS FÁBRICAS DE SOFTWARE. 164

Unicesumar





TEMA DE APRENDIZAGEM 1

O PARADIGMA DA FÁBRICA DE SOFTWARE

MINHAS METAS

- Compreender o conceito e a metáfora da fábrica de software.
- Analisar a evolução histórica do desenvolvimento de software.
- Comparar métodos tradicionais e o paradigma de fábrica.
- Identificar princípios fundamentais e métricas de qualidade.
- Avaliar os benefícios e desafios da implementação.
- Examinar estudos de caso nacionais e internacionais.
- Refletir sobre a informalidade no desenvolvimento de software.

INICIE SUA JORNADA

Você está em um estágio. A área pede "só um ajuste rápido" numa tela; outro cliente quer "uma API simples"; o gestor cobra "para ontem". Cada pedido chega por WhatsApp, sem o mesmo formato. Você começa a fazer, pausa e volta porque faltou um exemplo, o *tester* acha um erro que "ninguém combinou", a fila cresce. No fim, todo mundo trabalhou muito e, mesmo assim, o prazo estoura.

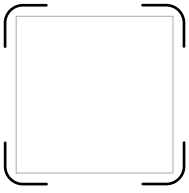
Resolver esse cenário importa porque o **tempo perdido vira custo**, o **retrabalho desgasta a equipe** e a **confiança do cliente diminui**. Quando o trabalho entra do mesmo jeito, com combinados claros de início e fim, o time ganha **previsibilidade**, **qualidade** e **tranquilidade** para evoluir. Não é burocracia: é **linguagem comum** para reduzir ruído.

Escolha **três tipos de pedido** que você mais recebe (ex.: "ajuste de tela", "nova consulta", "correção crítica"). Para cada um, descreva **o que precisa existir para começar** (exemplos, critérios de aceite) e **o que comprova o término** (evidências simples). Combine com o time **dois checkpoints** (revisão + teste), **meça por duas semanas** o tempo entre "pedido" e "entregue". Só isso já revela gargalos.

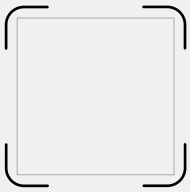
O que hoje depende do "jeito de cada um"? Onde a fila trava (espera por decisão, por *review*, por ambiente)? Que **acordo mínimo** poderia ser testado amanhã para reduzir risco ou encurtar caminho? Como você mostraria, em uma entrevista, **o antes e o depois** dessas pequenas mudanças?

PLAY NO CONHECIMENTO

Preparando o terreno para entregar com confiança.
Esse episódio trará uma conversa direta e motivadora para você refletir sobre como tornar o cotidiano de desenvolvimento mais leve e previsível, por meio pequenas atitudes que cabem na sua rotina hoje mesmo. Clique para ouvir!



VAMOS RECORDAR?



Antes de mergulhar nos modelos de fábrica, vale relembrar com o que você já teve contato: um bom **pedido** costuma explicitar problema, objetivo e exemplos de entrada/saída; **critérios de aceite** tornam verificável o que "está pronto" (pense em "dado X, quando Y, então Z"); **mudanças pequenas** facilitam revisão e evitam retrabalho; e o básico de **controle de versão** – *branches* curtas, *commits* atômicos e mensagens que contam a história da mudança – confere previsibilidade ao fluxo. Se quiser revisitar rapidamente esses fundamentos, o guia oficial do Git apresenta um percurso compacto e direto.

DESENVOLVA SEU POTENCIAL

Você já viveu um projeto em que "todo mundo trabalhou muito", mas a entrega atrasou, os bugs voltaram e ninguém sabia dizer exatamente onde o tempo foi gasto? Em estágios, *freelas*, TCC e até projetos internos, é comum cada pessoa trabalhar "do seu jeito": nomes diferentes para tarefas, revisão sem critério, testes quando sobra tempo, decisões perdidas no chat. O resultado é fila, retrabalho e desgaste com o cliente, mesmo quando a equipe é talentosa e esforçada.

A metáfora de **fábrica de software** aparece como uma lente para organizar esse caos. Pense menos em "engessar" e mais em dar **cadência**: pedidos que chegam do mesmo jeito, passos mínimos para transformar pedido em entrega, sinais claros de qualidade. Em vez de depender de heróis, o time passa a depender de **acordos simples**: como a demanda entra, o que precisa estar pronto para começar, o que significa "feito" e como olhar para dados básicos do fluxo. Essa visão serve tanto a um órgão público com contrato de serviço quanto a um app que você está construindo com amigos.

Imagine você e seu time combinando, de antemão, "qual tipo de pedido existe", "o que precisa estar claro para começar" e "o que comprova que terminamos bem". É disso que trata o paradigma de fábrica de software: uma **linguagem comum operacional** a qual não engessa, mas organiza o cotidiano, por meio

de catálogo de serviços, critérios explícitos de entrada (DoR) e de conclusão (DoD), mais uma esteira enxuta ligando desenvolvimento, verificação e entrega com evidências leves (ABNT, 2009; Sato, 2016; Softex, 2016).

No Brasil, essa forma de trabalhar conversa diretamente com governança e prestação de contas: quando o serviço está padronizado e acompanhado por medições simples, fica mais fácil cumprir os SLA e demonstrar rastreabilidade em ambientes auditáveis, como órgãos públicos e bancos (Brasil, 2025; TCU, 2014).



Quando você abre o **catálogo de serviços**, percebe que ele não é um "formulário frio", e sim um acordo vivo entre quem pede e quem faz. "Nova API", "evolução de tela" ou "correção crítica P1" deixam de ser rótulos genéricos para ganhar **entrada mínima** (exemplos/artefatos) e **saída verificável** (DoD com evidências). O efeito prático é imediato: menos idas e vindas, menos "eu achei que…", mais **previsibilidade**.

A previsibilidade, aqui, não surge de reuniões mais longas, mas do encadeamento claro: triagem → desenvolvimento com **pequenos PR** → **testes de contrato** para integrar serviços → validação e publicação automatizadas (Sato, 2016;

Softex, 2016). Ao longo desse fluxo, usamos **quality gates** que só fazem sentido quando **reduzem risco** ou **encurtam caminho**; caso contrário, viram teatro e devem ser removidos (Albino; Aguiar, 2017).

Você pode estar se perguntando: "como sei se está funcionando?". Nós respondemos medindo **poucas coisas, sempre**. O **lead time** por tipo de serviço mostra o tempo real da ideia à produção; o **% no SLA** revela previsibilidade; a **taxa de reabertura** expõe qualidade efetiva. Não comparamos pessoas, e sim o **sistema antes/depois**, para decidir a próxima melhoria (Albino; Aguiar, 2017; Softex, 2016).

Em seguida, quando o produto cresce, o **reuso** deixa de ser "copiar e colar" e passa a ser uma **linha de produtos**: componentes versionados com mudanças declaradas, **contratos executáveis** (API/eventos) e **variabilidades mapeadas**, garantindo compatibilidade entre equipes diferentes (ABNT, 2009; Softex, 2016; Valente, 2020).

Esse jeito de trabalhar também afeta a produção e a **operação**. Em vez de "soltar" e torcer, definimos os **SLI/SLO** simples (por exemplo, latência P95 e taxa de erro) e usamos **orçamento de erro** para equilibrar estabilidade, bem como velocidade: se o orçamento termina, pausamos lançamentos e priorizamos confiabilidade, ou seja, decisão técnica, registrada e rastreável (CGU, 2018; TCU, 2018).

Em cenários governamentais, ainda conectamos **interoperabilidade** (ePING) e **proteção de dados** (LGPD) a esse cotidiano: contratos de integração claros, minimização de dados e *security checks* automáticos tornam-se parte dos *gates* (Brasil, 2018a; Brasil, 2018b).

Panorama histórico e consolidação do paradigma

A engenharia de software percorreu um arco que vai dos métodos artesanais (anos 1950-1960) ao desenvolvimento estruturado (1970-1980), passando por normas e modelos de processo nos anos 1990 (ISO/IEC 12207, ISO 9001, CMMI).

A partir dos anos 2000, a automação e os processos repetíveis prepararam o terreno para abordagens ágeis e, na década de 2010, para DevOps/SRE, com CI/CD, observabilidade e confiabilidade como metas de produto.

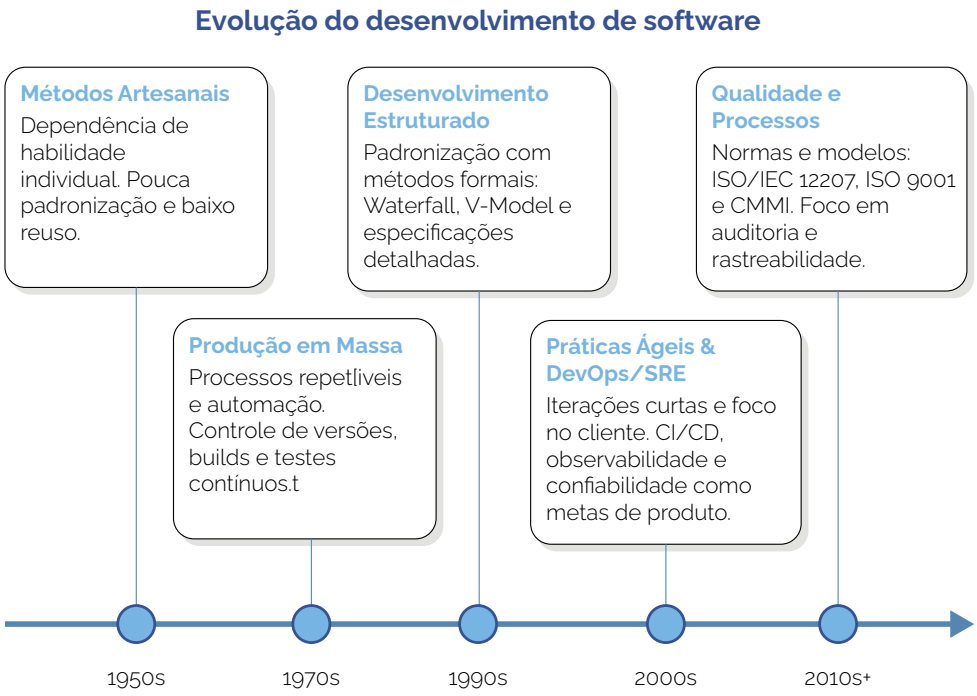


Figura 1 – Linha do tempo que sintetiza a evolução do desenvolvimento de software
Fonte: gerada por ChatGPT em 9 set. 2025.

Descrição da imagem: a figura mostra uma linha do tempo horizontal que ilustra a evolução do desenvolvimento de software. No topo, há o título centralizado "Evolução do desenvolvimento de software". A linha do tempo possui uma seta apontando da esquerda para a direita, indicando o progresso dos anos: as décadas de 1950, 1970, 1990, 2000, 2010+. Distribuídos acima e abaixo da linha, há cinco caixas retangulares com cantos arredondados e fundo branco, cada uma relacionada a um período específico da linha do tempo. A primeira caixa à esquerda, acima da linha, está rotulada como "Métodos Artesanais" e apresenta o texto: "Dependência de habilidade individual. Pouca padronização e baixo reuso". Abaixo da linha, alinhada ao período dos anos 1970, encontra-se "Produção em Massa", com o texto: "Processos repetitivos e automação. Controle de versões, *builds* e testes contínuos". Logo acima da linha, nos anos 1970, está "Desenvolvimento Estruturado" e o texto: "Padronização com métodos formais: *Waterfall*, V-Model e especificações detalhadas". Mais à direita, na altura dos anos 1990, está a caixa "Qualidade e Processos" com o texto: "Normas e modelos: ISO/IEC 12207, ISO 9001 e CMMI. Foco em auditoria e rastreabilidade". A última caixa, posicionada abaixo da linha nos anos 2010+, apresenta "Práticas Ágeis & DevOps/SRE" com o texto: "Iterações curtas e foco no cliente. CI/CD, observabilidade e confiabilidade como metas de produto". Em cada marcação da linha do tempo, há um círculo azul preenchido, reforçando o período correspondente de cada abordagem. Fim da descrição.

Do artesanal à fábrica: por que padronizar?

No modelo artesanal, as decisões tornam-se tácitas, a qualidade depende de indivíduos e a escalabilidade é limitada. A fábrica de software propõe uma plataforma operacional: catálogo de serviços, critérios de entrada/saída, esteiras automatizadas e métricas de fluxo. Tal plataforma reduz a variação, aumenta a previsibilidade, facilita a melhoria contínua. Em ambientes brasileiros, governamentais e bancários, a adoção trouxe transparência (evidências em *gates*) e rastreabilidade para auditorias; em empresas digitais, viabilizou escala com segurança.

Modelos de produção de softwares: Artesanal x Fábrica

Modelo Artesanal	Modelo de Fábrica
⊗ Processos pouco definidos e documentação irregular	✓ Processos padronizados e bem documentados
⊗ Trabalho individualizado, dependente de habilidades	✓ Trabalho em equipe, doco na repetibilidade
⊗ Dificuldade de escalar e manter a qualidade	✓ Escalabilidade com controle rigoroso de qualidade
⊗ Controle de produtividade informal	✓ Monitoramento contínuo da produtividade

Figura 2 – Tabela comparativa entre o modelo artesanal e o modelo de fábrica
Fonte: gerada por ChatGPT em 9 set. 2025.

Descrição da imagem: a figura apresenta uma tabela comparativa sobre modelos de produção de software, com o título em destaque no topo: "Modelos de produção de software: Artesanal × Fábrica". A tabela é dividida em duas colunas principais, cada uma com o cabeçalho em fundo azul-claro: "Modelo Artesanal", à esquerda, "Modelo de Fábrica", à direita. Há quatro linhas de comparação abaixo dos cabeçalhos. Na coluna "Modelo Artesanal", cada linha contém um texto à esquerda, acompanhado por um ícone circular vermelho com um "X", indicando aspectos negativos. Os textos são: "Processos pouco definidos e documentação irregular", "Trabalho individualizado, dependente de habilidades", "Dificuldade de escalar e manter a qualidade" e "Controle de produtividade informal". Na coluna "Modelo de Fábrica", cada linha à direita apresenta textos acompanhados por um símbolo verde circular com um símbolo de "*check*", indicando pontos positivos. Os textos dizem: "Processos padronizados e bem documentados", "Trabalho em equipe, foco na repetibilidade", "Escalabilidade com controle rigoroso de qualidade" e "Monitoramento contínuo da produtividade". Fim da descrição.

Processos repetíveis: como a fábrica opera

Uma fábrica organiza o trabalho em esteiras com *quality gates* que realmente reduzem risco. Exemplo de fluxo: Requisitos (entrada mínima e DoR) → Análise e Projeto (*impact analysis*, contratos de API/eventos) → Implementação (pequenos PR, *lint*, *build* e testes unitários) → Testes (contratos/integração; poucos E2E para jornadas críticas) → Implantação (CI/CD com *feature flags*, *rollbacks* e verificação pós-deploy) → Suporte/Operação (SLI/SLO, *post-mortem* sem culpa).

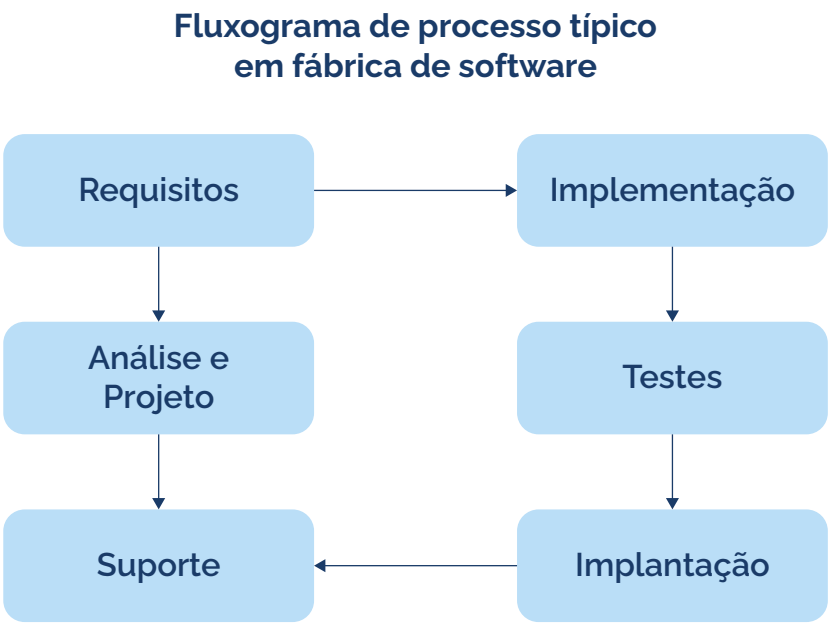


Figura 3 – Fluxo macro de uma fábrica de software, com etapas padronizadas e setas de passagem e retorno
Fonte: gerada por ChatGPT em 12 set. 2025.

Descrição da imagem: a figura apresenta um fluxograma com o título centralizado na parte superior: "Fluxograma de processo típico em fábrica de software". Abaixo do título, há seis retângulos de fundo azul-claro, dispostos em duas colunas, conectados por setas azuis que indicam o fluxo do processo. Na coluna da esquerda, de cima para baixo, estão os seguintes retângulos: "Requisitos", "Análise e projeto" e "Suporte". Na coluna da direita, a partir do topo, encontram-se: "Implementação", "Testes" e "Implantação". O fluxo inicia-se em "Requisitos". Uma seta horizontal aponta da caixa "Requisitos" para a direita, ligando-a à "Implementação". A partir daí, uma seta vertical desce até "Testes", que, por sua vez, está conectada por uma seta vertical à "Implantação". A partir daí, uma seta horizontal volta para a esquerda, conectando-se com "Suporte". Além disso, do topo de "Requisitos" parte uma seta vertical que se liga à "Análise e projeto"; a partir dessa, outra seta desce até "Suporte". Fim da descrição.

Trataremos a "fábrica" pela lente do dia a dia, começando pelo que muda a conversa: o **catálogo de serviços**. Em vez de pedidos difusos, o time passa a falar uma língua comum, ou seja, "nova api", "evolução de tela", "correção crítica p1", "integração externa", "*onboarding* de componente", "migração de versão". Cada tipo tem **entrada mínima** (o que precisa existir para começar) e **saída verificável/ DoD** (o que prova que terminou), com **pontos de inspeção** e **SLA de ciclo**. Esse "menu" reduz ambiguidade, encurta negociações e permite **medir lead time por serviço**, mostrando onde estão os gargalos e onde vale ajustar limites de **WIP**.

A qualidade, aqui, não é terceirizada: é **embutida na esteira**. Reabertura frequente e bug recorrente são sinais de **contratos fracos** e **DoD difuso**. O antídoto é fortalecer **testes de contrato** (OpenAPI/AsyncAPI, *consumer-driven*), análise estática e *security scanning*, executar mudanças em **ambientes efêmeros por PR,** bem como registrar **evidence as code** (relatórios automáticos nos *gates*). Quando você acompanha **taxa de reabertura**, *Change Failure Rate* (**CFR**) e **MTTR**, a melhoria deixa de ser opinião, vira rotina: a quebra que volta some, a revisão fica objetiva e o ciclo estabiliza.

Para ganhar escala sem "reinventar a roda" a cada entrega, entra a **Linha de Produtos de Software (LPS)**: uma **arquitetura de referência** orienta **componentes versionados** (semântico), **variabilidades mapeadas** e **políticas de depreciação** claras. Publicar contratos e *changelogs*, além de validar compatibilidade com **testes de contrato**, evita retrocessos silenciosos.

Como bússola prática, busque, a cada *sprint*, o seguinte: o uso de **ao menos 30% das entregas** por **um componente comum**: sinal de que o reuso está vivo e pagando a conta.

As métricas entram como poucas e sempre: **lead time por serviço** (P50/P90) para ver mediana e dispersão; **percentual no SLA** para previsibilidade; **taxa de reabertura** para qualidade efetiva; **CFR** e **MTTR** para estabilidade operacional. Compare **sistema antes/depois**, não pessoas entre si, e traga os números à **retrospectiva operacional**: se um *gate* não reduz risco nem encurta caminho, é **teatro**: elimine ou simplifique.

No **Brasil**, a fábrica consolidou-se em **governo** e **bancos** porque responde a requisitos duros de **evidência**, **SLA contratual**, **auditoria** e **rastreabilidade**. O ponto é fazer isso **com automação e dados**, não com papelada: catálogo claro, esteiras confiáveis, contratos testáveis e indicadores mínimos em produção dão visibilidade sem travar a inovação.

Por fim, amarre o fluxo de ponta a ponta. **Mapear da ideia ao deploy** mostra onde a fila de revisão acumula, o ambiente instável cobra pedágio e as aprovações difusas atrasam.

Diferenciar **DoR** (o acordo para começar) de **DoD** (o que comprova terminar) dá previsibilidade; **um teste automatizado mínimo** evita boa parte do retrabalho; um **pipeline simples e visível** (*lint* → *build* → testes → publicação) deixa rastros verificáveis.

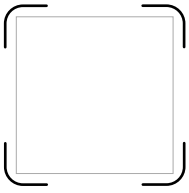
A lista de prontidão, curta e localizada (link para checklist, *script*, contrato), tem papel **pedagógico**: lembra onde vivem os critérios e reforça que o time mede o básico (*lead time*, reaberturas).

Com essa base, a metáfora de **fábrica de software** deixa de ser jargão: torna-se linguagem comum, pequenas padronizações e métricas que guiam a melhoria sem burocracia.

EU INDICO

Governo digital: catálogo e padrões

Conheça o CATSer-TIC, iniciativa federal que padroniza catálogos de serviços de TI em contratações públicas. Ajuda a visualizar, na prática, como tipos de serviço, SLAs e evidências ganham previsibilidade e auditoria, exatamente o que discutimos no modelo "orientado a processos".



FÁBRICA ≠ METODOLOGIA

Para evitar confusões conceituais, vale separar "**metodologia**" de "**fábrica**". Metodologias como Scrum, Kanban ou cascata descrevem **rotinas de trabalho**: papéis, cadências, artefatos e ritos de coordenação diária. A **fábrica de software**, por sua vez, opera em outra camada: é o "sistema operacional" da operação, pois define **o que pode ser solicitado** (catálogo de serviços), **como uma demanda entra** (critérios de prontidão), **como flui** (esteiras e automatizações), **o que significa concluir** (*Definition of Done*) e **como é inspecionado** (*quality gates*, SLA e métricas de fluxo).



Assim, é perfeitamente possível **rodar Scrum em uma** fábrica: a metodologia organiza as interações do time; a fábrica garante previsibilidade, rastreabilidade e evidência objetiva de qualidade (ABNT, 2009; Softex, 2016; Valente, 2020).

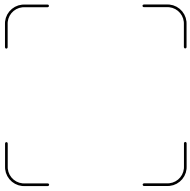
O **catálogo de serviços** é o ponto de virada porque transforma pedidos difusos em **tipos de serviço solicitáveis** com **entradas mínimas** e **saídas verificáveis**. Quando "evolução de tela", "nova api" ou "correção crítica" passam a ter linguagem comum, exemplos, critérios de aceite e SLA, a conversa deixa de ser sobre opiniões e torna-se **acordo observável**: o que precisa existir para começar, o que comprova terminar, em quanto tempo costuma fluir e onde o processo trava. Em organizações brasileiras, esse enquadramento reduz **retrabalho**, torna as **revisões mais objetivas**, cria base para medir **lead time por tipo de serviço** e **percentual no SLA**, parâmetros valiosos tanto para governança quanto para auditoria (Softex, 2016; TCU, 2018).

A **esteira com qualidade embutida** fecha o ciclo: integração e entrega contínuas, análise estática e **testes proporcionais ao risco** (unitários e de contrato como padrão, poucos E2E bem escolhidos), além de checagens automáticas de **segurança** e **compatibilidade**.

Os **quality gates** existem para **reduzir risco ou encurtar caminho**; se não cumprem uma dessas funções, tornam-se burocracia, portanto, devem ser simplificados ou removidos. Essa infraestrutura de evidência – pipeline *as code*, contratos testáveis, SLI/SLO e registros pós-entrega – permite que a metodologia escolhida (Scrum, Kanban ou outra) **se adapte ao domínio** sem perder previsibilidade. Assim, as cadências continuam leves, mas decisões passam a se apoiar em dados do fluxo e em sinais de qualidade que emergem da própria operação (ABNT, 2009; CGU, 2018; Softex, 2016).

EU INDICO

MPS.BR: estudo de caso aplicado
Relato de implantação do MPS.BR em empresa brasileira, útil para enxergar a evolução de processos, métricas e resultados antes/depois. Conecta com nosso debate sobre padronização, gates e melhoria contínua em fábricas de software.



Alinhando o vocabulário: ALM e rastreabilidade ponta a ponta

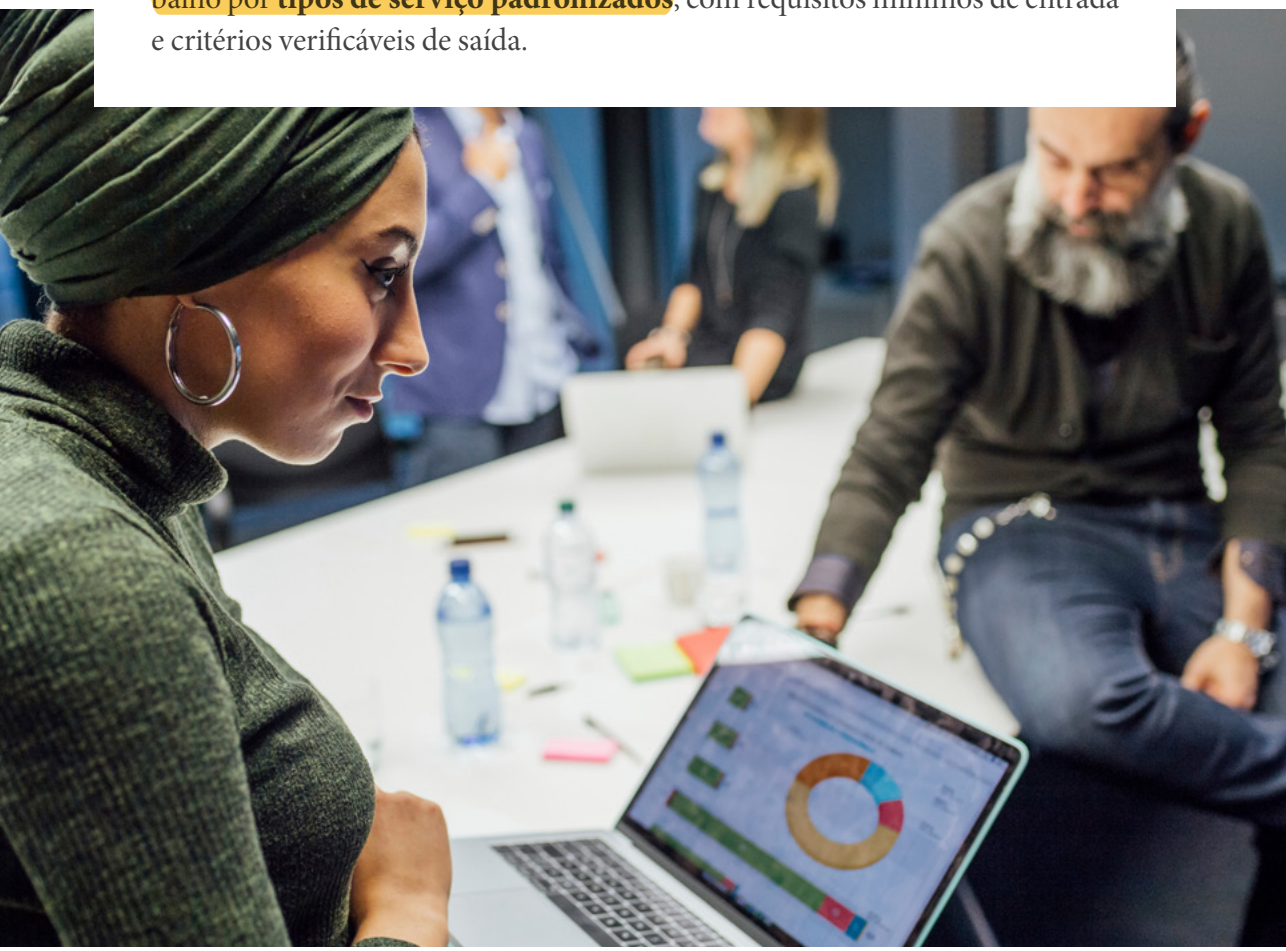
Vamos alinhar o vocabulário antes de avançar? Quando falamos em **Application Lifecycle Management (ALM)**, pense em um "fio condutor" que atravessa todo o ciclo do software, do **requisito** à **operação**. Não é uma ferramenta específica, mas um **conjunto integrado de práticas, processos e ferramentas** que dão coerência às etapas (planejamento, desenvolvimento, testes, entrega, operação) e, principalmente, **evidência** do que foi decidido e feito. O coração do ALM é a **rastreabilidade**: a capacidade de ligar cada decisão à mudança correspondente e ao resultado observado.

Na prática, **rastreabilidade ponta a ponta** significa que você consegue **seguir um item** sem rupturas. O pedido nasce como requisito, torna-se **issue/tarefa**, gera um **branch** e um **pull request**; ao abrir o PR, a **esteira (pipeline)** executa testes e verificações; aprovado, isso compõe uma **release** identificável; já em produção, os **indicadores do serviço** (SLI/SLO) mostram o efeito daquela mudança.

Essa linha contínua permite responder, com dados, às perguntas que importam: **por que mudamos**? **O que exatamente mudou**? **Qual foi o impacto no usuário**? É isso que confere previsibilidade, transparência e aprendizado real à fábrica de software.

MODELOS DE FÁBRICA

Para tratar os modelos organizacionais de fábricas de software com rigor acadêmico, é útil começar pelo **porquê** antes do **como**. Em contextos em que o volume de solicitações é alto e relativamente homogêneo, emerge o modelo que a literatura brasileira descreve como "fábrica de serviços": a organização do trabalho por **tipos de serviço padronizados**, com requisitos mínimos de entrada e critérios verificáveis de saída.



O objetivo não é burocratizar, mas **reduzir variabilidade** e permitir **medição comparável** entre áreas, por isso, a presença de SLA e indicadores de fluxo (por exemplo, *lead time* por tipo de serviço e taxa de reabertura) é estrutural, não acessória (ABNT, 2009; Softex, 2016). Em bancos e órgãos públicos, essa padronização viabiliza tanto a auditoria quanto a previsibilidade; a crítica recorrente de "balcão de tickets" não se resolve com mais formulários, mas com catálogos vivos e rotas de exceção para itens que exigem descoberta e design, preservando não só contexto, mas também valor.

Quando o problema dominante é de **evolução contínua** com incerteza de produto – hipóteses a validar, múltiplas integrações, decisões frequentes de usabilidade –, a configuração muda: times estáveis e multidisciplinares ancoram um **backlog de produto**, combinando *discovery* e *delivery* em cadências curtas. A racionalidade desse arranjo é alinhar decisões técnicas às hipóteses de valor e **minimizar custo de mudança**, o que explica a ênfase em **componentização** e **versionamento**, contratos de API/eventos e observabilidade (SLI/SLO) como base de confiabilidade (Softex, 2016; Valente, 2020).

As métricas, aqui, conectam **resultado** (adoção, conversão) à **estabilidade de mudança** (CFR, MTTR, *lead time*). Para evitar "*backlogs* infinitos", organizações brasileiras vêm adotando objetivos trimestrais e integração explícita com o catálogo de serviços para absorver a manutenção de rotina, mantendo foco e transparência.

Há cenários, contudo, em que a **data manda**: obrigações regulatórias, migrações de infraestrutura, *rollouts* nacionais. Nesses casos, o modelo de **projeto** – com escopo, orçamento e cronograma base (*baseline*) – continua pertinente desde que evitados os *big bangs*, ou seja, marcos incrementais, critérios de aceite observáveis e uma esteira de homologação que gere **evidência automática** antes do *go-live* reduzem risco operacional e de conformidade (ABNT, 2009; TCU, 2018).

As medidas que importam passam a ser cumprimento de marcos, desvio controlado de escopo e defeitos por etapa, além de SLO específicos quando há impacto no usuário final. Em plataformas digitais, é comum ver migrações conduzidas por ondas, com *feature toggles* e janelas de *rollback* previamente ensaiadas, o que concilia o imperativo da data com a segurança da transição.

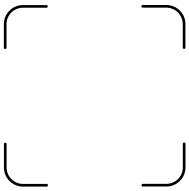
Por fim, à medida que as iniciativas se multiplicam e se interdependem, a unidade de análise sobe para **programas/portfólio**: aqui, o problema central é **coordenação**, em suma, tornar visíveis dependências, sincronizar janelas de liberação, alocar capacidade por valor e risco. A literatura de governança pública e de governo

digital brasileira recomenda cadências explícitas de *release train* e quadros de dependências que permitam remover bloqueios de forma antecipada (TCU, 2014).

O ponto comum aos quatro modelos é a **infraestrutura de evidência**: esteiras com qualidade embutida, contratos testáveis, métricas estáveis e registros pós-entrega. Em vez de rituais prescritivos, trata-se de **mecanismos verificáveis** que conectam desenho organizacional, fluxo de trabalho e resultado observado, razão pela qual cada modelo é escolhido não por moda, mas pela **adequação ao tipo de demanda** predominante e pelos **indicadores** que demonstram sua eficácia no contexto brasileiro.

EU INDICO

Transformação digital municipal (saúde)
Casos apresentados na Semana de Inovação da Enap mostram como serviços públicos digitais organizam portfólios, sincronizam releases e medem impacto em jornadas do cidadão. Referências úteis para nosso eixo de programas/portfólio.



Como combinar (governança leve)

Quando falamos em "combinar modelos com governança leve", a ideia é ter **uma porta de entrada única** para todas as demandas e, a partir dela, **encaminhar cada pedido ao fluxo mais adequado**, sem criar comitês, planilhas paralelas ou camadas de aprovação que só atrasam. Na prática, a triagem olha três perguntas simples: é uma demanda **rotineira e repetível**? Vai para o **Catálogo** (com entrada mínima e DoD claros).

Envolve **descoberta, hipóteses ou evolução de um produto**? Segue para o **time de Produto** (com *roadmap* e épicas). Tem **data inegociável** ou depende de um **marco regulatório**? Trate como **Projeto**, com marcos incrementais plugados na mesma esteira de entrega.

Você deve perceber que não são três organizações paralelas, e sim **três "pistas" do mesmo aeroporto**: a triagem decide em qual pista pousar, mas **o chão operacional é comum**.

Essa base comum evita atritos: **PR pequenos**, **testes de contrato** para integrações, **evidence as code** nos *gates*, **post-mortem sem culpa** e um **painel único** que mostra o que importa (*lead time*, % no SLA, reaberturas, CFR e MTTR). Com esses acordos, a troca de pista deixa de ser traumática: uma demanda pode começar no Catálogo e, ao revelar incerteza, migrar para Produto; um item de Produto pode gerar um **projeto pontual** quando surgir uma exigência legal, e nada disso exige "reinventar o processo", porque **a esteira é a mesma**.

Para dimensionar a operação, use uma regra inicial **apenas como ponto de partida** (cerca de 60-70% no Catálogo, 20-30% em Produto, ~10% em Projetos especiais) e **ajuste com dados**: se o *lead time* do Catálogo dispara, talvez haja trabalho de Produto disfarçado; se surgem muitos "projetos", pode ser sinal de que faltam componentes reutilizáveis; se o Programa (portfólio) vive bloqueado, as dependências não estão visíveis.

O papel da governança leve é **dar visibilidade e remover bloqueios**, não decidir por todos. Em ciclos curtos (semanal/quinzenal), a triagem revisa **o que entrou, para onde foi e por que**, confirma se as **políticas comuns** estão vivas e pergunta **o que mudou nos números**. Se um indicador piora (por exemplo, reaberturas acima do combinado), a resposta não é criar mais formulários, e sim **ajustar um acordo**: fortalecer o contrato de API, revisar o DoD, melhorar a observabilidade.

Com o tempo, a combinação deixa de ser um "manual", torna-se **um reflexo de operação**: cada demanda segue para o fluxo certo, todos compartilham a mesma esteira, e os números contam, de forma serena, se estamos no caminho.

DESAFIOS (E COMO MITIGAR)

Quando a fábrica começa a ganhar forma, o primeiro obstáculo costuma ser a **resistência a padrões**. Funciona melhor quando alinhamos o **porquê** e o **como mediremos**, quando iniciamos com **times-piloto** e mostramos **ganhos antes/depois**; a adesão aos acordos (**DoR/DoD**) e a **queda de reaberturas** contam essa história sem disputa de narrativas.

Logo aparece o tema do **custo inicial de automação**: o caminho é priorizar o que remove gargalos visíveis – *build*, **testes de contrato**, *lint* – e evoluir **em camadas**; a melhora reflete-se no **lead time P50/P90** e no **tempo de revisão**.

Em ambientes frágeis, a **dívida de automação** pede disciplina: reservar uma **capacidade fixa** (10-15%) para estabilizar, padronizar **ambientes efêmeros** e **templates**; o resultado aparece no **MTTR** e na redução de **flakiness** de testes.

Outra dor frequente é o **catálogo mal desenhado**, ou seja, muitos tipos, pouca clareza. Aqui vale consolidar por **valor e frequência**, definir **entradas mínimas** e **DoD curtos**, bem como fazer **revisões trimestrais**: a qualidade do desenho transparece no **% no SLA por tipo** e no retrabalho por **classificação incorreta**.

Já os **SLA/SLI/SLO mal calibrados** pedem método: calibrar metas com **dados históricos**, separar **SLO** (interno) de **SLA** (externo) e usar **orçamento de erro** para decidir quando estabilizar; o sucesso aparece no **cumprimento de SLO** e na redução de violações sem impacto ao usuário.

Há ainda o **choque entre modelos** (Processos × Produto × Projeto × Programa). A solução prática é **triagem única** com regras claras de encaminhamento e uma **pista de exceção** quando há descoberta/UX. Observe o **retrabalho por troca de fluxo** e o **throughput por modelo** para ajustar a rota. Em paralelo, precisamos vigiar o "**teatro de processo**": *gates* que não reduzem risco se tornam burocracia; revisá-los à luz de **incidentes reais** e remover o que não corta risco nem tempo tende a melhorar a *Change Failure Rate* (**CFR**), além de reduzir o **tempo gasto em cada gate**.

Por fim, a **observabilidade insuficiente** impede decisões: um **painel mínimo por serviço** (latência P95, taxa de erro), **logging padronizado** e **contratos de métricas** elevam o sinal sobre o ruído; a **cobertura de SLI** e a qualidade do **alerta** mostram quando estamos, de fato, enxergando o sistema.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Chegou a hora de ligar, de forma direta, o que você estudou ao que as empresas esperam ver funcionando no dia a dia. Pense no catálogo de serviços, nos combinados de entrada e saída (DoR/DoD), nas esteiras de integração e entrega, na qualidade embutida, nas métricas de fluxo, no reuso e nos acordos de confiabilidade: tudo isso ganha sentido quando aparece como evidência prática. Dessa forma, a teoria transforma-se em algo que você consegue mostrar, medir e defender em reuniões, avaliações e entrevistas.

> Em uma equipe de produto, desenvolvedores que trabalham com *pull requests* pequenos, testes automatizados e contratos de API **agilizam a revisão e estabilizam o lead time**. Profissionais de qualidade **criam quality gates para evitar retrabalho**, sem impor processos rígidos. Quem atua em plataforma mantém *pipelines* e ambientes efêmeros para entregas rápidas e seguras. Perfis de SRE definem "estar bem" por SLI e SLO e usam orçamento de erro para decisões equilibradas. Produto conecta catálogo ao valor de negócio, priorizando dados sobre opinião, enquanto lideranças técnicas sustentam arquitetura, reuso e versionamento para garantir entregas futuras estáveis.

Essa cena não é abstrata: aparece com força em governo digital, bancos e *fintechs*, saúde, educação, varejo e em *software houses* que precisam crescer sem perder previsibilidade.

Quando você se prepara para o mercado, o que conta é a clareza do que foi construído e o rastro deixado por ela. Um catálogo de serviços simples, com tipos de demanda, entrada mínima, DoD e *checkpoints*, muda a conversa porque possibilita comparar antes e depois do *lead time* por serviço e do percentual no SLA.

Uma esteira que mostre *lint*, *build*, testes unitários e de contrato, publicação e *deploy* comprova qualidade embutida, bem como tende a reduzir reaberturas e filas de revisão, assim, ninguém precisa acreditar "na palavra", porque o próprio pipeline se torna a evidência. Reuso bem tratado aparece na forma de um componente versionado com *changelog* e guia de adoção, capaz de poupar defeitos por incompatibilidade, além de encurtar a evolução de vários serviços.

Em produção, um painel discreto com latência P95 e taxa de erro conta a história com poucas linhas: quando os SLO estão sob controle, seguimos; quando não, pausamos lançamentos, corrigimos e retomamos, registrando a decisão. Segurança e conformidade também entram nesse cotidiano: verificações automáticas de segredos e dependências vulneráveis, além de tratamentos mínimos de dados em linha com a LGPD, evitam surpresas sem transformar a rotina em burocracia.

Na hora de narrar suas experiências, prefira histórias curtas, situadas e com números. Em vez de declarar "melhoramos bastante", diga que o *lead time* mediano caiu de 14 para seis dias depois que o time adotou catálogo, pequenos PR e contratos de API, e que a taxa de reabertura desceu de 17 para 4%.

Em vez de afirmar "investimos em reuso", explique que dois componentes foram publicados com versionamento semântico e cinco serviços migraram em três *sprints* sem quebras. Essa forma de contar mostra não apenas domínio técnico, mas também responsabilidade pelo resultado.

Para que tudo fique à mão, organize um conjunto enxuto de evidências: um Readme de uma página com objetivos, arquitetura, catálogo, políticas de PR e o pipeline *as code*; três imagens que mostrem o pipeline executando, o gráfico simples de *lead time* (P50/P90) e o painel com SLI/SLO; um relato breve de aprendizado após um incidente; a referência a um componente reutilizável, com exemplo de uso. Não esqueça de anonimizar dados e credenciais: profissionais cuidadosos protegem o que não deve ser exposto.

Conforme você avançar, perceberá que o cenário muda: muitas organizações estão fortalecendo *platform engineering* para oferecer templates, *pipelines*, ambientes e observabilidade "como produto" interno; a inteligência artificial já participa do ciclo de engenharia em revisão de código e geração de testes, exigindo tanto curadoria quanto responsabilidade; práticas de FinOps pedem decisões sustentáveis de custo e desempenho; contratos executáveis – especificações e políticas "*as code*" – validam integridade a cada entrega. Em comum, há a necessidade de transformar conceitos em rotina visível, isto é, menos promessa, mais rastro.

No fim, a mensagem é simples: teoria sem prática torna-se ritual, prática sem teoria termina em improviso. Ao reunir evidências de fluxo, qualidade, reuso e confiabilidade, você mostra que sabe construir sistemas funcionais e evolutivo, exatamente o que o mercado procura.

1. Em operações de desenvolvimento, diferentes indicadores podem ser usados para acompanhar o andamento das entregas e ocorrências ao longo do processo. A escolha e a interpretação desses indicadores variam conforme o contexto e as necessidades de gestão (Softex, 2016).

   Com base nesse texto, analise as afirmativas:

   I - *Lead time* por serviço (P50/P90) ajuda a entender a variação do fluxo.
   II - % no SLA mede previsibilidade de entrega.
   III - Taxa de reabertura indica retrabalho/qualidade efetiva.
   IV - Métricas devem comparar indivíduos para premiar os mais produtivos.

   É correto o que se afirma em:

   a) I, apenas.
   b) II e IV, apenas.
   c) III e IV, apenas.
   d) I, II e III, apenas.
   e) I, II, III e IV.

2. Práticas de definição prévia de entradas e saídas são comuns em operações de desenvolvimento, assim como acordos de nível de serviço. Equipes costumam registrar critérios de aceitação de trabalho (Fernandes; Teixeira, 2011).

   Sobre práticas de catalogação e critérios, analise as afirmativas:

   I - O catálogo de serviços define entradas mínimas e saídas verificáveis.
   II - SLA explicita prazos/nível de serviço acordados para o tipo de demanda.
   III - *Definition of Ready* (DoR) e *Definition of Done* (DoD) são equivalentes e usados no mesmo ponto do fluxo.

   É correto o que se afirma em:

   a) I, apenas.
   b) III, apenas.
   c) I e II, apenas.
   d) II e III, apenas.
   e) I, II e III.

3. As taxas altas de reabertura indicam falhas de qualidade. Testes de contrato e DoD claro reduzem incompatibilidades (ISO, 2017).

   Uma equipe tem 18% de reabertura por incompatibilidade entre serviços. Qual ação provavelmente reduz esse indicador?

   a) Aumentar reuniões de status.
   b) Adicionar mais testes E2E sem mexer nos demais testes.
   c) Criar um *role* de aprovação manual.
   d) Implantar testes de contrato entre serviços e reforçar DoD com casos de integração.
   e) Proibir *pull requests* fora do horário comercial.

REFERÊNCIAS

ABNT – ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 12207**: engenharia de software – processos do ciclo de vida de software. Rio de Janeiro: ABNT, 2009.

ALBINO, R.; AGUIAR, F. **Métricas ágeis**: obtenha melhores resultados em sua equipe. São Paulo: Casa do Código, 2017.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados (LGPD). Brasília, DF: Presidência da República, 2018a. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 6 nov. 2025.

BRASIL. Ministério da Gestão e da Inovação em Serviços Públicos. Instrução Normativa SGD/ME nº 1, de 4 de abril de 2019. Dispõe sobre o processo de contratação de soluções de Tecnologia da Informação e Comunicação – TIC pelos órgãos e entidades integrantes do Sistema de Administração dos Recursos de Tecnologia da Informação – SISP do Poder Executivo Federal. **Gov.br**, Brasília, DF, 7 ago. 2025. Governo Digital. Disponível em: https://www.gov.br/governodigital/pt-br/contratacoes-de-tic/legislacao/processo-de-contratacao-de-solucoes-de-tic-regido-pela-lei-ndeg-8-666-de-1993/instrucao-normativa-sgd-me-no-1-de-4-de-abril-de-2019. Acesso em: 6 nov. 2025.

BRASIL. Ministério do Planejamento, Desenvolvimento e Gestão. Secretaria de Tecnologia da Informação e Comunicação. **ePING** – Padrões de interoperabilidade de governo eletrônico: documento eletrônico. Brasília, DF: Ministério do Planejamento, Desenvolvimento e Gestão, 2018b. Disponível em: https://eping.governoeletronico.gov.br/#apresentacao. Acesso em: 2 dez. 2025.

CGU – CONTROLADORIA GERAL DA UNIÃO. **Guia de gestão de riscos**: integridade, gestão de riscos e controles internos na APF. Brasília, DF: CGU, 2018.

FERNANDES, A. A.; TEIXEIRA, D. de S. **Fábrica de software**: implantação e gestão de operações. São Paulo: Atlas, 2011.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 12207**: systems and software engineering – software life cycle processes. Geneva: ISO, 2017.

SATO, D. **DevOps na prática**: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2016.

SOFTEX – ASSOCIAÇÃO PARA PROMOÇÃO DA EXCELÊNCIA DO SOFTWARE BRASILEIRO. **MPS.BR**: guia geral. Brasília, DF: Softex, 2016.

TCU – TRIBUNAL DE CONTAS DA UNIÃO. **Manual de auditoria de tecnologia da informação**. Brasília, DF: TCU, 2018.

TCU – TRIBUNAL DE CONTAS DA UNIÃO. **Referencial básico de governança de TI (iGovTI)**. Brasília, DF: TCU, 2014.

VALENTE, M. T. **Engenharia de software moderna**. Belo Horizonte: Independente, 2020.

CONFIRA SUAS RESPOSTAS

1. Alternativa D.

   As afirmativas I, II e III são práticas recomendadas.
   A afirmativa IV está incorreta, pois gera distorções e não melhora o sistema.

2. Alternativa C.

   As afirmativas I e II são conceitos centrais.
   A afirmativa III está incorreta, pois DoR qualifica a entrada, DoD qualifica a saída.

3. Alternativa D.

   Contratos + DoD atacam a causa (quebra de interface).
   As demais alternativas não tratam a fonte do retrabalho.

MEU ESPAÇO

Unicesumar





TEMA DE APRENDIZAGEM 2

MODELOS ORGANIZACIONAIS DE FÁBRICAS DE SOFTWARE

MINHAS METAS

- Reconhecer diferenças entre fábrica por processos, produtos, projetos e programas.
- Identificar quando adotar cada modelo e seus *trade-offs*.
- Relacionar modelo organizacional aos papéis e à estrutura de equipes.
- Escolher métricas e KPIs adequados para cada modelo.
- Entender os impactos no ciclo de desenvolvimento (cascata, ágil, híbrido).
- Ler exemplos brasileiros e extrair lições aplicáveis.
- Montar um plano de transição/combinação de modelos no próprio contexto.

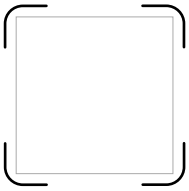
INICIE SUA JORNADA

Você entra num time que "vira *sprint*" sem parar. O diretor cobra previsibilidade; as áreas pedem funcionalidades conflitantes; um cliente exige data fixa para auditoria. O que muda se a fábrica for organizada por processos, produtos, projetos ou programas?

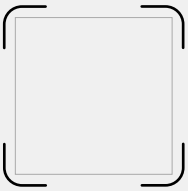
O modelo define quem decide, como o trabalho flui e o que medir, pode reduzir retrabalho e disputa por prioridade. Pegue dez demandas reais da sua empresa/estudo: classifique-as por tipo (manutenção de produto, projeto regulatório, fluxo padrão, iniciativa transversal) e esboce qual modelo atenderia melhor a essas demandas. Onde seu contexto precisa de padronização, onde pede foco em produto e onde exige entrega por projeto? O que pode ser coordenado por programa?

PLAY NO CONHECIMENTO

Nesse curto episódio, você ouvirá histórias reais de times brasileiros lidando com filas, prioridades conflitantes e "datas que não podem falhar". Quais sinais aparecem no seu contexto e como reconhecê-los no dia a dia? Dê o play e traga anotações, elas serão úteis nas atividades do tema.



VAMOS RECORDAR?



Antes de avançar, vale relembrar bases rápidas que ajudam a ler qualquer organização sem entrar (ainda) nos modelos deste tema. Quando dizemos processo, estamos falando de como o trabalho acontece do pedido à entrega; quando falamos em organização do trabalho, tratamos de quem decide, como prioriza e como as áreas conectam-se. Um *backlog* é a fila visível de demandas; um *roadmap*, o plano de evolução no tempo; portfólio, o conjunto de iniciativas que competem por capacidade. Em fluxo, o *lead time* é o tempo total da ideia à entrega; WIP, o trabalho em progresso; reabertura sinaliza qualidade não confirmada e, para conversar com dados, uma visualização simples do fluxo (quadro/Kanban) já ajuda a enxergar onde a fila trava e onde o time pode atuar primeiro.

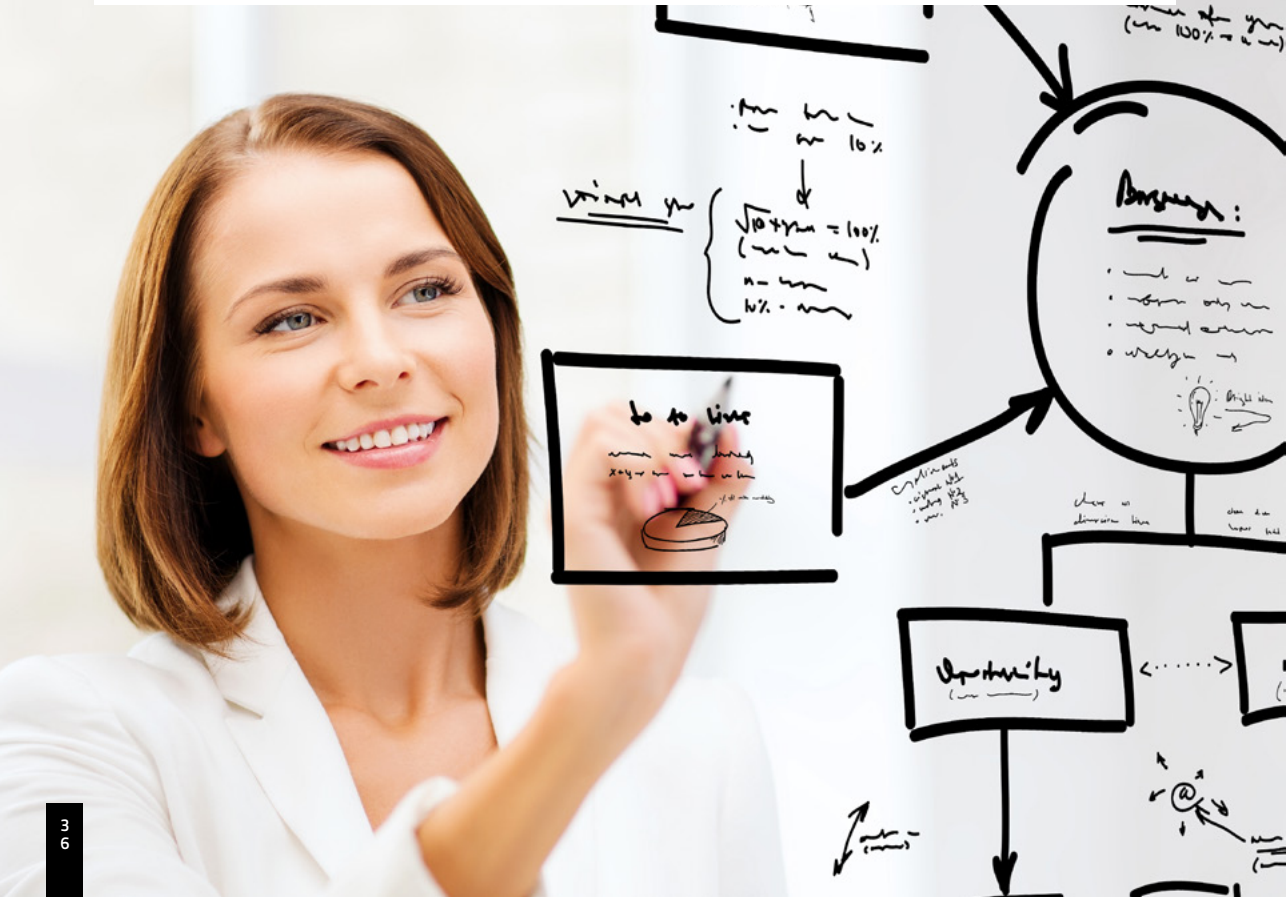
DESENVOLVA SEU POTENCIAL

PANORAMA: POR QUE O MODELO ORGANIZACIONAL DECIDE O JOGO?

Quando uma fábrica de software "**anda no tranco**", muitas vezes, o problema não é de ferramenta nem de método de time: é modelo organizacional mal escolhido.

Em termos simples, há quatro arranjos que aparecem (e podem coexistir): orientado a processos (padronização e previsibilidade), a produtos (reuso e evolução contínua), a projetos (entrega com data ou marco) e a programas (coordenação de portfólios) (ABNT, 2009; Softex, 2016a).

Cada modelo muda a conversa: quem decide, como prioriza, como o trabalho flui e o que medir. Setores brasileiros regulados – governo e bancos – tendem a valorizar processos e programas (Brasil, 2019; TCU, 2014), enquanto software *houses* e *fintechs* frequentemente combinam produtos e projetos.



FÁBRICA ORIENTADA A PROCESSOS: PREVISIBILIDADE PELA PADRONIZAÇÃO

O que é, quando faz sentido e como não virar burocracia

O que é, quando faz sentido e como não virar burocracia. Na organização por processos, catalogamos os tipos de serviço (entrada mínima, saída verificável) e rodamos o fluxo em esteiras padronizadas com *quality gates* leves, todos justificáveis por redução de risco ou encurtamento de caminho. É o arranjo ideal para alto volume repetitivo, exigência de SLA e rastreabilidade (ABNT, 2019; Softex, 2016b). Em secretarias estaduais e tribunais, por exemplo, o catálogo evita pedidos difusos e facilita auditorias (TCU, 2014).

APROFUNDANDO

Pense no catálogo como um acordo de trabalho que elimina ambiguidades antes que elas virem retrabalho. Para começar, rascunhe três tipos de serviço muito presentes no dia a dia. Quando alguém pede uma nova API, a conversa não começa pelo prazo, mas pelo que precisa existir para iniciar: um contrato rascunhado e alguns exemplos de uso; termina quando o **teste de contrato** está verde e a **documentação** foi publicada, esse é o nosso **Done**. Se a demanda for evolução de tela, a entrada mínima é um protótipo simples, com critérios de aceite objetivos; a conclusão acontece com casos de teste executados e release *notes* visíveis. Já para correção P1 em produção, a porta de entrada é a evidência do defeito; a saída aceitável é o *hotfix* validado e o *roll-back* testado, para ninguém ficar refém da sorte.

A graça do catálogo é poder medir o que importa sem esforço extra: observar o **lead time por serviço** (P50/P90), o **percentual cumprido no SLA** e a **taxa de reabertura** já conta a história do fluxo.

Há um cuidado que não pode faltar: evitar o "teatro de processo". Se algum *gate* não reduz risco nem encurta caminho, ele atrapalha, por isso, uma revisão trimestral para eliminar o que não agrega mantém o processo vivo e honesto (Albino; Aguiar, 2017).

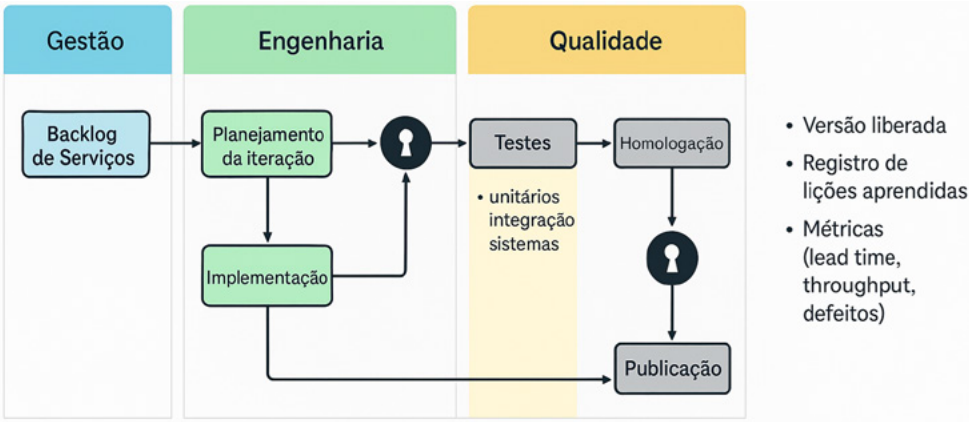


Figura 1 – Fluxo padrão orientado a processos: do backlog de serviços aos *gates* (revisão + testes + publicação)
Fonte: gerado por ChatGPT em 26 set. 2025.

Descrição da imagem: a figura apresenta um fluxo de trabalho do backlog de serviços até os *gates* (revisão, testes e publicação), organizado em raias funcionais coloridas: gestão (azul-claro) e engenharia (verde) estão separadas por uma linha divisória, seguidas por qualidade (amarelo) e operações (cinza). As setas mostram a passagem por triagem/priorização, planejamento da iteração, implementação, revisão por pares, testes (unitários, integração e sistemas), homologação e publicação. As saídas destacadas são: versão liberada, registro de lições aprendidas e métricas (*lead time*, *throughput*, defeitos). Fim da descrição.

FÁBRICA ORIENTADA A PRODUTOS: REÚSO QUE ESCALA E REDUZ RETRABALHO

Vamos discutir arquitetura de referência, componentes versionados e variabilidades? No arranjo **por produtos**, o centro é a **evolução contínua** de plataformas e serviços. Para evitar "copiar e colar", bem como o caos de dependências, adotaremos **arquitetura de referência**, **componentes reutilizáveis** versionados e **variabilidades mapeadas** (linha de produtos).

A operação fica mais fluida quando contratos de API/eventos são **executáveis** (testes de contrato) e *changelogs* orientam migrações (ABNT, 2009; Softex, 2016a). Empresas SaaS brasileiras obtêm ganhos diretos de **tempo** e **qualidade** ao tratar reuso como **produto interno**.

KIT DE PRODUTO

Visão, *guidelines* de arquitetura, catálogo de componentes e templates de serviço.

GOVERNANÇA LEVE

Janelas de depreciação e migração guiada por *playbooks*.

KPIS DE PRODUTO

Adoção de componentes, efeitos por versão, *cycle time* por épica, *lead time* de evolução.

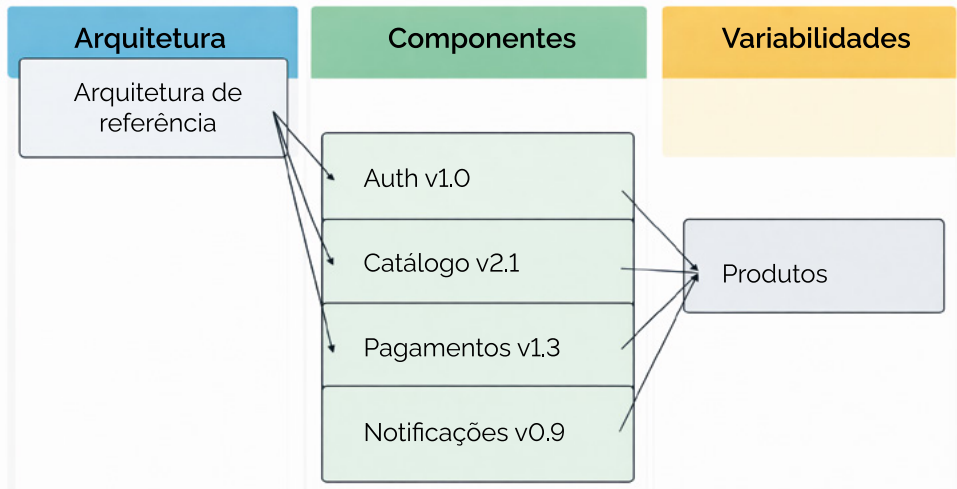


Figura 2 – Mapa de produto e linha de produtos (LPS): arquitetura de referência, componentes reutilizáveis, variabilidades e políticas comuns / Fonte: gerado por ChatGPT em 26 set. 2025.

Descrição da imagem: quadro dividido em três colunas principais: "Arquitetura", "Componentes" e "Variabilidades". Na coluna da esquerda, em azul, há um retângulo com o texto "Arquitetura de referência", do qual saem várias setas apontando para a coluna do meio. A coluna central, em verde, contém um grande retângulo claro dividido em quatro faixas horizontais empilhadas, cada uma representando um componente com nome e versão: "Auth v1.0" na parte superior, "Catálogo v2.1" logo abaixo, "Pagamentos v1.3" em seguida e "Notificações v0.9" na parte inferior. Da borda direita desse bloco central saem várias setas em direção a um retângulo cinza claro, posicionado na coluna da direita, com o texto "Produtos". Acima desse retângulo, ainda na coluna da direita, há outro retângulo amarelo com o título "Variabilidades" e, no interior, o mesmo texto "Variabilidades", sugerindo que a arquitetura de referência alimenta os componentes, que por sua vez se conectam aos produtos e às suas variabilidades. Fim da descrição.



FÁBRICA ORIENTADA A PROJETOS: QUANDO A DATA MANDA

Sobre as entregas pontuais, marcos e integração com a esteira comum, projetos têm escopo/cronograma definidos (migração de *gateway*, adequação fiscal, *rollout* regional). O risco é cair no big bang.

O caminho brasileiro mais sólido tem sido combinar marcos com entregas incrementais conectadas à esteira padronizada: homologação, *feature flags*, *cutover* com *roll-back* (TCU, 2018; ABNT, 2009).

EAP LEVE + MARCOS
Com critérios de aceite e verificação objetiva.
INTEGRAÇÃO À ESTEIRA CI/CD COMUM
Garantindo padronização dos processos de *build*, testes e *deploy dress rehearsal* de *cutover*.
KPIS DE PROJETO
Cumprimento de marcos, desvio de prazo/custo, defeitos por etapa.

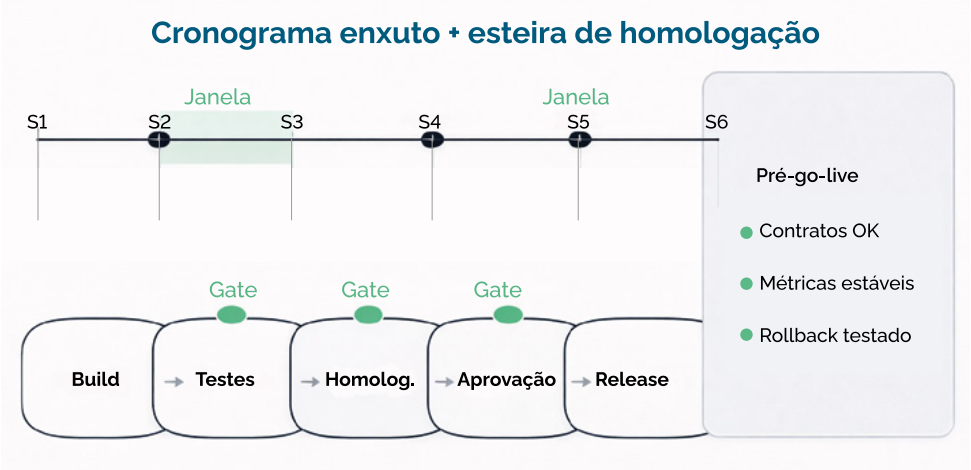


Figura 3 – Cronograma enxuto + esteira de homologação: *milestones*, janelas de liberação e checagens pré-*go-live* / Fonte: gerado por ChatGPT em 26 set. 2025.

Descrição da imagem: na faixa superior da imagem, sob o título "Cronograma enxuto + esteira de homologação", há uma linha do tempo de seis *sprints* (S1–S6) com três marcos: M1 em S2, M2 em S4 e M3 em S5, e janelas de liberação destacadas em verde (entre S2 e S3 e em S5). Abaixo, a esteira de homologação apresenta cinco etapas em sequência: *Build* → Testes → Homologação → Aprovação → Release; pequenos indicadores circulares verdes marcam os *gates* de testes, homologação e aprovação. À direita, um cartão pré-*go-live* sintetiza as checagens: contratos OK, métricas estáveis e *rollback* testado. Fim da descrição.



FÁBRICA ORIENTADA A PROGRAMAS: ORQUESTRAR O PORTFÓLIO

Vamos tratar das dependências, priorização por valor/risco e métricas de portfólio? Programas coordenam frentes interdependentes (experiência do cidadão, *open finance*, interoperabilidade entre órgãos). A governança eficaz decide alocação por valor e risco, remove bloqueios e torna visíveis as dependências (TCU, 2014; Brasil, 2018). O objetivo não é criar camadas de comitês, mas sincronizar cadências.

RITUAIS DE SINCRONIZAÇÃO (QUINZENAL/MENSAL)

Focados em **bloqueios** e **apostas** (não em status).

MAPA DE DEPENDÊNCIAS E ORÇAMENTO DE RISCO

Por exemplo: improváveis/impacto alto → planos de contingência.

KPIS DE PROGRAMA

Throughput do portfólio, dependências resolvidas por ciclo, alinhamento a objetivos estratégicos.

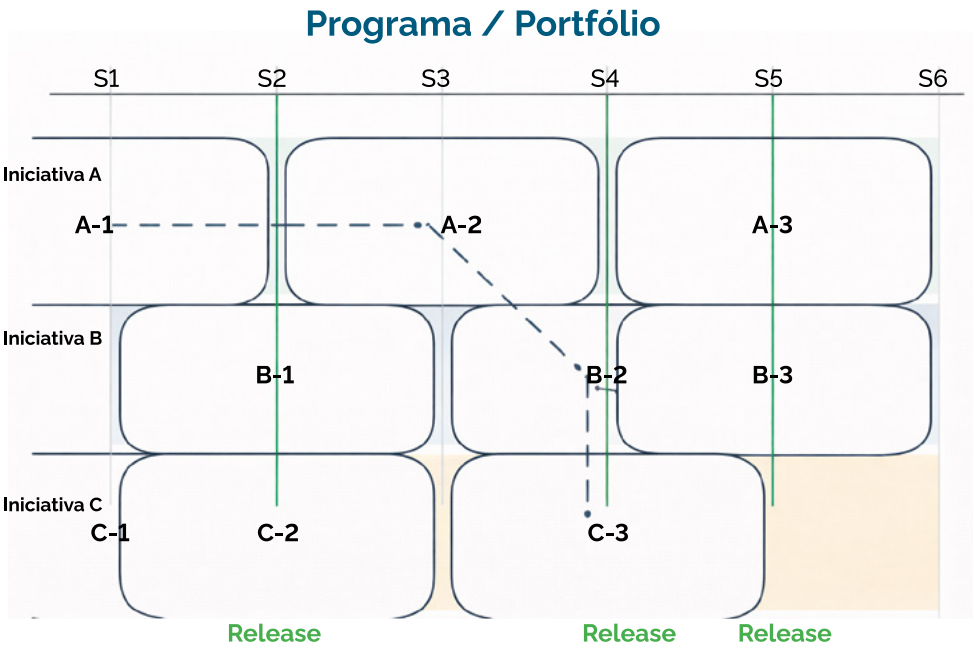


Figura 4 – Quadro de programa/portfólio: iniciativas, dependências e cadências
Fonte: gerado por ChatGPT em 26 set. 2025.

Descrição da imagem: cronograma visual de iniciativas de um programa, dividido por colunas de tempo (S1 a S6) e linhas representando "Iniciativa A", "Iniciativa B" e "Iniciativa C". Cada iniciativa tem blocos desenhados que mostram suas etapas, marcadas por códigos como "A-1", "A-2", "A-3", "B-1" etc. Linhas tracejadas indicam dependências entre etapas de diferentes iniciativas. Ao longo das colunas, há linhas verticais, e a palavra "Release" marca entregas principais em pontos específicos do tempo, facilitando o acompanhamento dos marcos do projeto. Fim da descrição.

COMO ESCOLHER E COMBINAR MODELOS (SEM MODISMOS)

Antes de decidir "virar produto", "implantar processos ágeis" ou "gerenciar por projetos", pare e olhe para o tipo de trabalho que você realmente tem em mãos. Modelos organizacionais não são times de futebol: você não "torce" para um único. Eles são ferramentas para problemas diferentes. A boa prática é **combinar** modelos e **trocar de marcha** quando os sinais mostrarem que a configuração atual não está entregando. Em linhas gerais:

- **Processos**: padronizam rotinas repetíveis e auditáveis, para ganhar previsibilidade e escala na operação.
- **Produtos**: organizam equipes e *backlog* em torno de **plataformas**/soluções com evolução contínua e reutilização.
- **Projetos**: dão foco a **escopos** com data, custo e escopo definidos (ex.: regulatório, *go-live*).
- **Programas**: coordenam **múltiplas frentes interdependentes** rumo a um objetivo estratégico comum.

A escolha não é ideológica; é **contextual**. O mesmo portfólio pode ter atendimento de serviços (Processos), um *core platform* (Produtos), iniciativas com marcos inegociáveis (Projetos) e transformação transversal (Programas).

Uma matriz de decisão simples para o seu contexto

Use a regra-prática, a seguir, como ponto de partida, e ajuste-a às suas realidades e métricas internas:

- **Alto volume repetível/auditável → Processos**
  Quando há fila contínua (ex.: correções conhecidas, solicitações padrão, esteiras CI/CD), você quer *lead time* estável, % no SLA alto e baixa variância.
- **Plataforma com evolução contínua → Produtos**
  Caso exista uma base comum (ex.: app, *core banking*, plataforma de dados) com *roadmap* de médio/longo prazo e oportunidade clara de **reuso/componente**.

- **Data fixa / regulatório → Projetos**
  Quando há marcos inegociáveis e critérios de aceitação claros, você gerencia risco, caminho crítico e escopo fechado/gerenciado.
- **Múltiplas frentes interdependentes → Programas**
  Se várias *squads*/projetos precisam convergir (ex.: omnicanalidade, migração *cloud enterprise*), exigindo governança integrada e benefícios compartilhados.
- **Sinais de troca (quando mudar a marcha)**
- **Lead time errático** nos serviços? Falta **Processo** (padrões, WIP, filas, SLAs).
- **Retrabalho entre times** e duplicação de esforço? Falta **Produto** (plataforma, catálogo de componentes).
- **Marcos inegociáveis** em risco? Trate como **Projeto** (*baseline*, gestão de mudanças).
- **Conflitos entre squads** e dependências travando? Eleve a **Programa** (*roadmap* integrado, gestão de interdependências) (ABNT, 2019; Softex, 2016b).

IMPACTOS NA GESTÃO DE EQUIPES E PAPÉIS (BRASIL REAL)

Sobre cargas cognitivas, *handoffs* e autonomia com responsabilidade: quando a organização se orienta por processos, alguém precisa zelar pelo catálogo, pelos *gates* e pelas métricas de fluxo para o trabalho entrar do mesmo jeito e sair de forma verificável. Quando o foco são produtos, a conversa muda: entra em cena o *product ownership*, uma arquitetura de referência que dá rumo às decisões, e a disciplina de versionamento, a qual evita quebras entre times.

Em iniciativas com data e escopo definidos, a lógica de **projetos** pede gestão de riscos e de **marcos** bem clara; já em frentes interdependentes, o olhar de **programas** coordena dependências e direciona esforço pelo **valor estratégico** (TCU, 2014; Valente, 2020). No setor público, essa prática conversa com a **Instrução Normativa nº 1/2019 (SGD/ME)**, que orienta contratações e acompanhamento de TI sem excesso de formalismo (Brasil, 2019).

O cuidado permanente é manter **governança leve**: menos papel e mais **evidência**: relatórios automáticos que nascem da própria esteira e *dashboards* simples que mostram o essencial.

> **Risco de silo x risco de volatilidade**: processos demais isolam; produto sem governança volatiliza; projeto sem integração quebra; programa sem foco empaca. O equilíbrio é dinâmico.

CICLOS DE DESENVOLVIMENTO E FERRAMENTAS QUE ENCAIXAM

Cascata, ágil e híbridos sob medida compartilhando a mesma esteira. Quando a fábrica se organiza bem, o ritmo muda conforme a necessidade: nos processos, o trabalho flui em ciclos curtos e quase contínuos; em produtos, a conversa gira em torno do *roadmap* e das épicas que puxam a evolução; em projetos, marcos são combinados com incrementos para evitar o big bang; e quando há muita interdependência, programas abrem janelas de sincronização para manter todos alinhados.

Por baixo de qualquer arranjo, a base é a mesma: esteiras CI/CD com qualidade embutida, em que *lint*, *build*, testes unitários e de contrato, publicação e *deploy* com *feature flags* acontecem como caminho padrão, seguidos de uma verificação pós-*deploy* que fecha o ciclo com segurança (Sato, 2016; Softex, 2016c).



Em produção, poucos indicadores bem escolhidos – SLIs/SLOs – dão transparência sobre o que significa "estar bem" e sustentam decisões serenas usando orçamento de erro: quando a meta estoura, pausa-se o ritmo, corrige-se o rumo e só então se acelera de novo (CGU, 2018; TCU, 2018).

PRINT 1
Pipeline executando (*build* → testes → publicação).
PRINT 2
Gráfico de *lead time* P50/P90 por classe de demanda.
PRINT 3
Painel SLI/SLO (latência P95, taxa de erro).

MINICASES BRASILEIROS (APLICADOS)

Trataremos de três retratos curtos, para inspiração. Pense em três cenas, todas reais no nosso contexto: no setor público, uma secretaria estadual de saúde decide padronizar os serviços digitais e centralizar a entrada das demandas; com a mesma esteira e regras para todos, o percentual dentro do SLA sobe de 62% para 89%, e as auditorias deixam de ser "caça-papel": passam a consumir as evidências que já nascem dos *gates* (Brasil, 2019; TCU, 2014).

Em um banco ou *fintech*, a conversa muda para reuso e previsibilidade: a plataforma publica componentes estáveis, versão 1.0.0 com *changelog* claro, e quando surge uma obrigação regulatória, o time trata como projeto com marcos incrementais, nada de big bang no fim (ABNT, 2009; TCU, 2018).

Já em um SaaS nacional, a adoção de um componente de autenticação versionado derruba defeitos por release e encurta o *lead time* mediano; as demandas de rotina entram por catálogo com SLA, mantendo o fluxo previsível sem engessar a evolução do produto (Albino; Aguiar, 2017; Softex, 2016d).

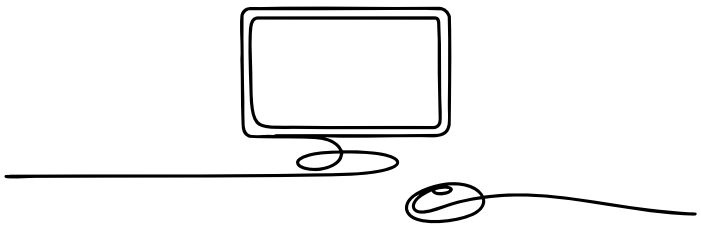
PLANO DE APLICAÇÃO (30 DIAS): SEU EXPERIMENTO CONTROLADO

Demonstraremos, a seguir, evidências que contam a história. Transformaremos isso em um caminho contínuo, fácil de acompanhar. Na primeira semana, apenas observe e classifique o que chega: pegue as próximas 20 demandas e identifique se cada uma encaixa-se melhor em Processos, Produtos, Projetos ou Programas.

Na segunda semana, escolha um KPI por modelo; algo simples e objetivo, como % no SLA para Processos, adoção de componente para Produtos, cumprimento de marcos para Projetos e *throughput* para Programas.

Na terceira semana, ajuste a triagem com o que aprendeu e publique dois artefatos mínimos que fornecem transparência: um catálogo de serviços enxuto e um *roadmap* de produto claro o suficiente para orientar as próximas decisões.

Na quarta semana, conte a história com dados: monte um gráfico simples de antes e depois – nada sofisticado, só o necessário para enxergar a mudança – e escreva oito linhas registrando as lições aprendidas: o que funcionou, o que ainda atrapalha e qual será o próximo passo. O objetivo é deixar as evidências falarem por você, sem precisar de promessas ou jargões.



EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Quando você olha para frente, o segredo não é decorar rótulos – Processos, Produtos, Projetos e Programas –, e sim reconhecer como cada escolha organiza o trabalho e onde essa organização aparece de forma palpável.

Se a empresa precisa de previsibilidade e rastreabilidade, padronizar entradas e saídas dá segurança para crescer. Em plataformas digitais, reuso e evolução contínua de componentes evitam retrabalho. Quando a data manda, marcos e integrações planejadas reduzem o risco. Com muitas frentes em paralelo, o olhar de portfólio expõe dependências que travam a entrega. Quem enxerga essas diferenças propõe "o caminho certo para o problema certo", e isso pesa em entrevistas, avaliações e no dia a dia.

Esse discernimento é visto em sinais que recrutadores reconhecem: em Processos, um catálogo de serviços com combinados claros e evidências de fluxo; em Produto, *roadmaps*, componentes versionados e contratos de API/eventos; em projetos, critérios de aceite e marcos sem big bang; em programas, a capacidade de remover bloqueios e priorizar por valor e risco.

No Brasil, governo e bancos pedem previsibilidade e trilha de auditoria; software *houses*, *fintechs* e startups valorizam velocidade com estabilidade; saúde, educação e varejo exigem coordenação e interoperabilidade. Em comum, importa falar em evidências: tempo de passagem, reaberturas em queda após ajuste de contrato, adoção real de componente que reduziu defeitos, painéis simples indicando quando estabilizar antes de acelerar.

No portfólio, mostre um rastro claro: catálogo enxuto com critérios, pipeline *as code* com testes, componente versionado com *changelog*/exemplo e um breve *post-mortem*. Tudo isso evidencia como você decide, entrega e evolui.

Troque "qual modelo é melhor?" por "qual modelo responde melhor à demanda agora e como os dados provam isso?". Com critérios, acordos e indicadores, teoria torna-se prática, bem como valor entregue.

1. Quando múltiplas frentes caminham em paralelo, a visibilidade sobre relações entre trabalhos ajuda a avaliar prioridades e alocar esforços. Nem toda iniciativa exige o mesmo nível de coordenação, mas alguma leitura integrada é necessária. O desenho adotado deve ser proporcional ao objetivo e às restrições (Brasil, 2018; TCU, 2014).

   Com base em seus conhecimentos, analise as afirmativas, a seguir:

   I - Os rituais de sincronização focam em bloqueios e apostas (não apenas em status).
   II - Um mapa de dependências apoia decisões de sequenciamento.
   III - Os KPIs de programa incluem *throughput* do portfólio e dependências resolvidas.
   IV - Os programas tornam dispensável a coordenação entre áreas.

   É correto o que se afirma em:

   a) I e IV, apenas.
   b) II e III, apenas.
   c) III e IV, apenas.
   d) I, II e III, apenas.
   e) I, III e IV, apenas.

2. Alguns arranjos de trabalho dão ênfase à consistência e à compatibilidade entre componentes ao longo do tempo, exigindo disciplina na evolução dos artefatos (Pohl; Böckle; Linden, 2005; Valente, 2020).

   Qual prática é mais coerente com uma fábrica orientada a produtos?

   a) Aprovação manual adicional antes de todo *deploy*.
   b) Componentes versionados e contratos executáveis para APIs/eventos.
   c) Concentração de aprovações em uma única janela semanal.
   d) Duplicar componentes para acelerar entregas.
   e) Eliminar *changelogs* para evitar burocracia.

3. Iniciativas com objetivos e prazos definidos podem adotar práticas de redução de risco, conectando entregas a mecanismos padronizados de verificação (ISO, 2017; Sato, 2016; TCU, 2018).

   Qual combinação tende a reduzir risco em projeto com data inegociável?

   a) Entrega única no final e testes pós-*go-live*.
   b) Marcos + entregas incrementais + integração à esteira CI/CD comum.
   c) Transferência de testes para o time de produção.
   d) Suspensão de qualquer release até a data final.
   e) Substituição de critérios de aceite por aceite verbal.

REFERÊNCIAS

ABNT – ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 12207**: engenharia de software – processos do ciclo de vida de software. Rio de Janeiro: ABNT, 2009.

ABNT – ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 20000-1**: tecnologia da informação – gestão de serviços – Parte 1: requisitos do sistema de gestão de serviços. Rio de Janeiro: ABNT, 2019.

ALBINO, R.; AGUIAR, F. **Métricas ágeis**: obtenha melhores resultados em sua equipe. São Paulo: Casa do Código, 2017.

BRASIL. Ministério da Gestão e da Inovação em Serviços Públicos. Instrução Normativa SGD/ME nº 1, de 4 de abril de 2019. Dispõe sobre o processo de contratação de soluções de Tecnologia da Informação e Comunicação – TIC pelos órgãos e entidades integrantes do Sistema de Administração dos Recursos de Tecnologia da Informação – SISP do Poder Executivo Federal. **Gov.br**, Brasília, DF, 7 ago. 2025. Governo Digital. Disponível em: https://www.gov.br/governodigital/pt-br/contratacoes-de-tic/legislacao/processo-de-contratacao-de-solucoes-de-tic-regido-pela-lei-ndeg-8-666-de-1993/instrucao-normativa-sgd-me-no-1-de-4-de-abril-de-2019. Acesso em: 3 dez. 2025.

BRASIL. Ministério do Planejamento, Desenvolvimento e Gestão. Secretaria de Tecnologia da Informação e Comunicação. **ePING** – Padrões de interoperabilidade de governo eletrônico: documento eletrônico. Brasília, DF: Ministério do Planejamento, Desenvolvimento e Gestão, 2018b. Disponível em: https://eping.governoeletronico.gov.br/#apresentacao. Acesso em: 2 dez. 2025.

CGU – CONTROLADORIA GERAL DA UNIÃO. **Guia de gestão de riscos**: integridade, gestão de riscos e controles internos na APF. Brasília, DF: CGU, 2018.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 12207**: systems and software engineering – software life cycle processes. Geneva: ISO, 2017.

POHL, K.; BÖCKLE, G.; LINDEN, F. J. van der. **Software product line engineering**. Berlin: Springer, 2005.

SATO, D. **DevOps na prática**: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2016.

SOFTEX – ASSOCIAÇÃO PARA PROMOÇÃO DA EXCELÊNCIA DO SOFTWARE BRASILEIRO. **Guia de implementação** fundamentação para implementação do MR-MPS-SW. Brasília, DF: Softex, 2016a. 4 partes.

REFERÊNCIAS

SOFTEX – ASSOCIAÇÃO PARA PROMOÇÃO DA EXCELÊNCIA DO SOFTWARE BRASILEIRO. **Guia geral MPS de gestão de pessoas**. Brasília, DF: Softex, 2016b.

SOFTEX – ASSOCIAÇÃO PARA PROMOÇÃO DA EXCELÊNCIA DO SOFTWARE BRASILEIRO. **MPS.BR**: guia de aquisição. Brasília, DF: Softex, 2016c.

SOFTEX SOFTEX – ASSOCIAÇÃO PARA PROMOÇÃO DA EXCELÊNCIA DO SOFTWARE BRASILEIRO. **MPS.BR**: guia de medição. Brasília, DF: Softex, 2016d.

TCU – TRIBUNAL DE CONTAS DA UNIÃO. **Manual de auditoria de tecnologia da informação**. Brasília, DF: TCU, 2018.

TCU – TRIBUNAL DE CONTAS DA UNIÃO. **Referencial básico de governança de TI (iGovTI)**. Brasília, DF: TCU, 2014.

VALENTE, M. T. **Engenharia de software moderna**. Belo Horizonte: Independente, 2020.

CONFIRA SUAS RESPOSTAS

1. Alternativa D.

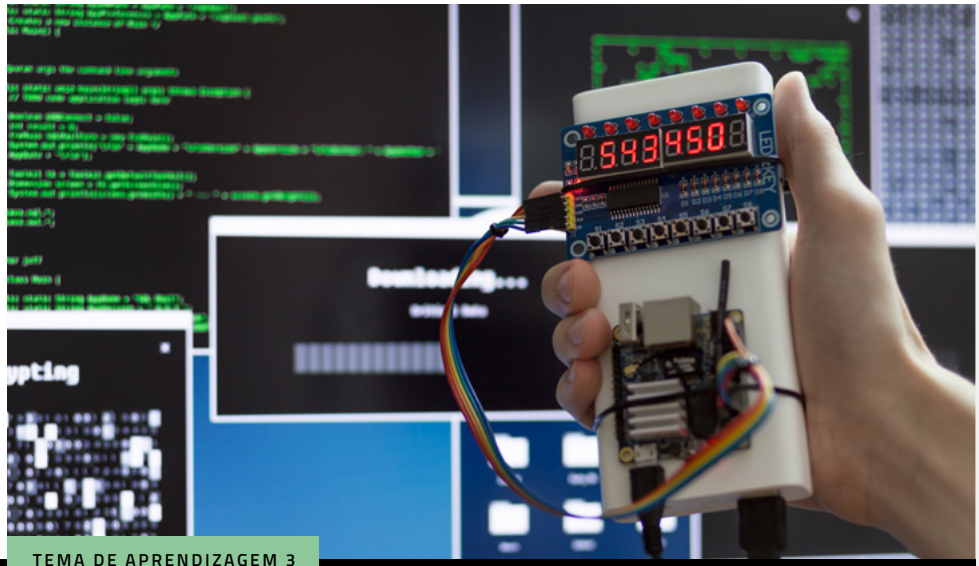
   A afirmativa I está correta, pois sincronizações efetivas tratam bloqueios e decisões.
   A afirmativa II está correta, pois o mapa de dependências sustenta sequenciamento.
   A afirmativa III está correta, pois métricas de programa incluem *throughput* do portfólio e dependências resolvidas.
   A afirmativa IV está incorreta, pois programas exigem coordenação, não a dispensam.

2. Alternativa B.

   A alternativa A está incorreta, pois aprovações manuais extras não endereçam compatibilidade/reuso.
   A alternativa B está correta, pois componentes versionados e contratos executáveis para APIs/eventos preservam compatibilidade e viabilizam reuso.
   A alternativa C está incorreta, pois janelas rígidas únicas tendem a concentrar risco.
   A alternativa D está incorreta, pois duplicação aumenta dívida técnica e inconsistência.
   A alternativa E está incorreta, pois eliminar *changelogs* reduz transparência e previsibilidade.

3. Alternativa B.

   Incrementos com marcos e integração à esteira comum antecipam validações e reduzem risco de big bang; e as demais mantêm ou ampliam o risco.



TEMA DE APRENDIZAGEM 3

TIPOS DE SOFTWARE EM AMBIENTES INDUSTRIAIS

MINHAS METAS

- Identificar e classificar softwares industriais por categoria e propósito.
- Explicar requisitos não funcionais centrais de cada categoria.
- Analisar usos em saúde, indústria, agro e logística, ligando requisitos a cenários.
- Reconhecer as principais normas da indústria.
- Discutir riscos de segurança e privacidade e controles conceituais.
- Descrever conceitualmente integrações entre gestão, manufatura, operação e dispositivos.
- Argumentar decisões arquiteturais com rastreabilidade e trade-offs de custo e risco.

INICIE SUA JORNADA

Você chega ao estágio e, antes do café, três chamados: a checklist do armazém não sincroniza; a ordem criada na gestão não aparece na execução; o painel some com um alerta vindo de sensores. Onde está o gargalo? É falha do app de campo, do sistema corporativo, da operação ou do dispositivo? E, mais importante: o que priorizar agora, com pouco tempo e alto impacto?

> Perceba o que está em jogo: segurança, custos, prazos e reputação. Em saúde, decisões podem afetar pessoas; no agro, uma janela perdida compromete a safra; na indústria, minutos valem dinheiro. Entender para qual tipo de software você está olhando, quais requisitos não funcionais são críticos (desempenho, disponibilidade, segurança, privacidade) e em qual domínio ocorre o problema é o que transforma "achismo" em critério.

Sem teoria pesada ainda, só ação mínima e consciente:

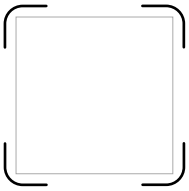
- **Desenhe o caminho feito pelo dado**: de quem gera (dispositivo/operador) até onde aparece (painel/relatório). Quem conversa com quem?
- **Observe três sinais por um turno**: tempo de resposta percebido, falhas de sincronização e minutos de indisponibilidade.
- **Compare domínios**: hospital, campo e armazém pedem o mesmo nível de risco? O que muda na prioridade?
- **Rascunhe integrações em linguagem simples**: pontos únicos de falha e um plano B viável (reenvio, alerta manual, verificação dupla).
- Esses micropassos ajudam a **classificar o software**, **mapear requisitos sensíveis** e **coletar evidências** para a primeira decisão (ISO, 2017).

Agora, pergunte-se:

- Qual chamado ataco primeiro e por que (impacto em pessoas, perdas, tempo)?
- Quais requisitos não funcionais estão mais pressionados em cada ponto do fluxo?
- Que regras e boas práticas devo consultar, mesmo no diagnóstico inicial, para não violar segurança e privacidade?
- Como registro os *trade-offs* (tempo x custo x risco) de forma rastreável para justificar a escolha?

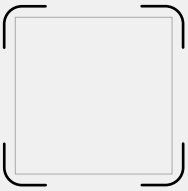
PLAY NO CONHECIMENTO

Em menos de quatro minutos, um panorama direto de softwares em ambientes industriais, do PLC ao app na nuvem. Veja como disponibilidade, latência, operação off-line e rastreabilidade moldam a engenharia e onde moram conformidade, segurança e integração. Dê o play!



VAMOS RECORDAR?

Antes de avançarmos, retome três ideias-base que facilitarão a leitura:



- **Requisitos**: diferencie "o que o software faz" (funcionais) de "como faz" (não funcionais: desempenho, disponibilidade, segurança, privacidade).
- **Camadas**: relembre a separação entre gestão (processos), execução/manufatura (operações), operação/dispositivos (sensores/atuadores) e painéis/relatórios (decisão).
- **Domínios e risco**: saúde, indústria, agro e logística mudam prioridades e critérios de aceite.

Assista a um vídeo rápido para reforçar **requisitos funcionais x não funcionais**.

DESENVOLVA SEU POTENCIAL

MAPA DO TERRENO

Antes de falar de tecnologias, vamos localizar o terreno? Ambientes industriais costumam combinar camadas com funções distintas – do dispositivo à nuvem, da execução à gestão. É essa geografia que explica por que o mesmo "incidente" pede métricas e ações diferentes (Sommerville, 2018). Esse mapa serve como legenda para tudo que vem depois.

Onde cada software vive

Aqui, descrevemos, em linguagem direta, quem fala com quem e o que cada camada valoriza. O objetivo é você identificar rapidamente onde está o problema (dispositivo, execução, gestão, nuvem, apoio à decisão) e, a partir disso, escolher métricas mínimas e evidências adequadas para decidir.



Com o terreno nomeado, a Figura 1 sintetiza as principais categorias e suas relações. Use-a como referência rápida quando precisar decidir **onde medir primeiro**. Leia a figura como um radar: o núcleo representa dados/serviços comuns; cada raio, uma categoria típica. Ao identificar o raio certo, você reduz o ruído, assim, direciona diagnóstico, teste e priorização.

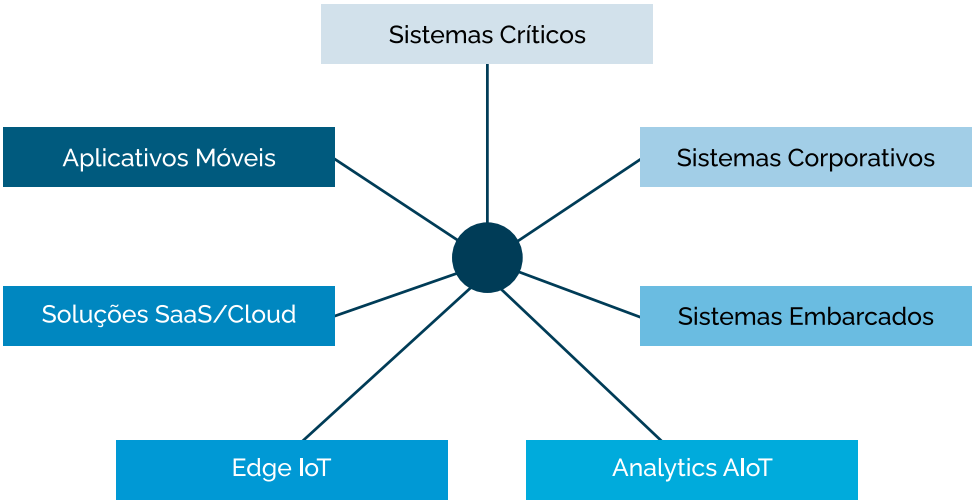


Figura 1 – Classificação de softwares em ambientes industriais (visão geral)
Fonte: gerado por ChatGPT em 21 out. 2025.

Descrição da imagem: diagrama radial com um núcleo central conectado por linhas a sete caixas de texto, cada uma rotulada com um título: "Sistemas embarcados", "Sistemas corporativos", "Sistemas críticos", "Aplicativos móveis", "Soluções SaaS/Cloud", "Edge/IoT" e "Analytics/AIoT". As caixas representam categorias típicas de software em ambientes industriais e evidenciam que todas se integram por meio de dados e serviços. Fim da descrição.

Quando alguém diz "o sistema falhou", a primeira pergunta é: qual sistema? Em empresas industriais e de serviços há, na prática, cinco camadas que convivem e, às vezes, confundem-se. Há quem viva no dispositivo/na operação, lidando com sensores, atuadores, coletores e apps de campo; quem orquestra a execução/manufatura, registrando etapas, aponta-

Em empresas industriais e de serviços há, na prática, cinco camadas

mentos e confirmações; quem sustente a gestão/negócio, com estoque, finanças, manutenção, compras; quem "rode" na nuvem, publicando painéis, APIs e eventos; e quem apoie a decisão, com análises, regras e modelos.

Nomear onde o problema mora muda o jogo, porque cada camada tem prioridades, riscos e métricas diferentes.

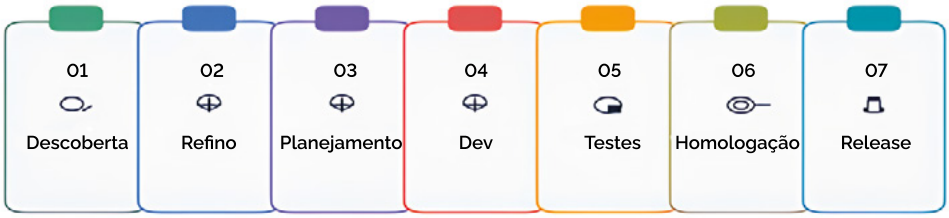


Figura 2 – Linha do tempo em sete etapas: Descoberta → Refino → Planejamento → Dev → Testes → Homologação → Release / Fonte: gerado por ChatGPT em 9 nov. 2025.

Descrição da imagem: a figura mostra uma faixa horizontal branca com sete blocos retangulares, altos e estreitos, alinhados lado a lado, cada um com cantos arredondados e uma pequena aba colorida centralizada no topo. Dentro de cada bloco há, de cima para baixo, um número grande de dois dígitos em negrito, um pequeno ícone simples em traço fino e, abaixo, uma palavra que nomeia a etapa do processo. Em ordem da esquerda para a direita, os blocos exibem: "01 Descoberta", "02 Refino", "03 Planejamento", "04 Dev", "05 Testes", "06 Homologação" e "07 Release". Fim da descrição.

DISPOSITIVO/OPERACIONAL

No dispositivo/operacional, a pergunta-chave é: "o comando chega e a resposta volta a tempo?". Aqui, latência P95, perda de pacotes, estabilidade de rede e energia fazem diferença. Quedas de leitura intermitentes pedem *buffer* local, *retry* exponencial e telemetria no próprio equipamento (Pressman; Maxim, 2016).

EXECUÇÃO/MANUFATURA

Na execução/manufatura, o que vale é disponibilidade por turno e confirmação confiável: ordens não podem "sumir". Medimos *uptime* por célula, taxa de confirmação, filas crescentes e tempos "emissão → confirmação". Um plano B simples, **fila local + reenvio**, costuma reduzir picos de indisponibilidade sem parar a linha (Fernandes; Teixeira, 2011).

GESTÃO/NEGÓCIO

Subindo para a gestão/negócio, o risco é inconsistência: um estoque que diverge, uma nota que não concilia. Métricas como reconciliações por período, volume de ajustes manuais e tempo de acerto apontam se o processo é confiável. Trilha de auditoria ("quem mudou o que, quando e por quê"), **idempotência** nas integrações e validações de domínio antes de gravar dado diminuem retrabalho (ISO, 2017; Pressman; Maxim, 2016).

NUVEM/SERVIÇOS

Na nuvem/serviços, a conversa é observabilidade e metas de serviço: SLIs/SLOs de latência e erro, taxa de sucesso em integrações, saúde de filas e *backoffs*. Painéis claros, alertas com bom sinal/ruído e *feature flags* para ativar/desativar funcionalidades dão segurança para evoluir sem big bang (Pressman; Maxim, 2016; Sommerville, 2018).

APOIO À DECISÃO

Por fim, no apoio à decisão, importa qualidade e linhagem de dados. De onde veio, como foi transformado, com que frescor e se houve drift. Medimos completude, atualidade, acurácia amostral e disparamos avisos quando um modelo sai da faixa esperada (Sommerville, 2018). Sem isso, um gráfico bonito pode orientar mal a operação. Perceba o padrão: em cada camada, existe uma pergunta mestra, um risco dominante e um conjunto mínimo de evidências. Se o painel não atualiza, olhamos ingestão e eventos (nuvem); se a ordem não confirma, investigamos fila e retry (execução); se o saldo não fecha, rastreamos integrações e conciliações (gestão); se o sensor "pula", medimos P95 e perda de pacotes (dispositivo); se a previsão está errando mais, verificamos drift e amostra (análises).

Quando você identifica, com precisão, onde está o problema no sistema, o ruído cai: a conversa sai de "culpa" e vai para onde medir, o que ajustar e qual evidência comprova. Essa postura acelera o diagnóstico, qualifica decisões e encurta o ciclo de correção.

Adotar um olhar estratificado – por camadas, prioridades e evidências – transforma a vaga "o sistema falhou" em um plano de ação claro, com métricas, responsáveis e próximos passos definidos.

INDICAÇÃO DE FILME

Ford vs Ferrari (2019)

O que um filme de corrida tem a ver com software industrial? Pense comigo: prazos rígidos, decisões técnicas sob risco, busca por desempenho sem renunciar a confiabilidade.

Refletindo sobre a história: assista com este olhar: o que define prioridade? Quais métricas importam? Qual o custo de um erro em alta velocidade? Depois, traduza para o nosso contexto: disponibilidade, tempo de resposta, segurança e rastreabilidade.



O padrão se repete: uma pergunta mestra por camada, um risco dominante e um kit mínimo de evidências. Localizar o ponto do sistema já corta metade do problema, o debate sai de "culpa" e vai para "causa + prova".

Requisitos que mudam a régua

Para fugir do achismo, avalie o sistema por cinco lentes e torne cada uma mensurável. Comece por **disponibilidade**, registrando minutos fora do ar por turno e distinguindo paradas planejadas das incidentais para achar a causa dominante. Em seguida, olhe o tempo de resposta percebido pelo P95 ("95% das requisições respondem até X s"), o qual expõe a pior experiência típica que a média esconde e orienta SLOs realistas (Pressman; Maxim, 2016; Sommerville, 2018).

A terceira lente é integridade e rastreabilidade: conecte requisito, teste e evidência em uma cadeia única; cada mudança deve ter um ID que ligue *issue*, PR, pipeline e release, permitindo responder o que mudou, por que mudou e como ficou comprovado? Depois, segurança e privacidade: menor privilégio, criptografia em trânsito e repouso, gestão de segredos (nada de chaves no código), bem como checagens automáticas de vulnerabilidades/segredos nos *gates* (Brasil, 2018; ISA, 2002). Por fim, observabilidade: logs estruturados, métricas (latência, erro, filas) e *tracing* entre serviços; alerta bom tem alto sinal, baixo ruído e aponta o componente culpado.

O contexto define o peso dessas lentes: em saúde, privilegie confidencialidade e alertas confiáveis; na manufatura, reação rápida e confirmações de ordens; no agro, operação off-line com reconciliação; na logística, latência percebida e eventos íntegros. Com esse recorte, as prioridades ficam **defensáveis**: estabilizar P95 antes de lançar *features*, manter trilha de auditoria para reduzir retrabalho e investir em gestão de segredos para evitar incidentes. Em síntese, escolhemos "o que medir" a partir do impacto: primeiro, garantimos o essencial (P95 estável, auditoria mínima e gestão de segredos), depois, evoluímos *features* com segurança. Evidência primeiro, decisão depois.

Para tornar essas lentes mais práticas, vale observar como o contexto muda conforme o tipo de software. A natureza do produto (embarcado, corporativo, móvel, crítico, nuvem, apoio à decisão) determina quais lentes pesam mais e quais evidências precisam ser coletadas, bem como acompanhadas.

Agora que você entende "onde" vivem os sistemas, vale olhar "que tipo" de software está em jogo. O tipo ajusta o peso das lentes e orienta o que medir e provar (Sommerville, 2018), como veremos a seguir.

EMBARCADO
Opera em dispositivos; exige tempo de resposta e confiabilidade (IEC, 2006; RTCA, 2011).
CORPORATIVO (GESTÃO)
Processos e dados; pede integridade e auditoria.
CRÍTICO
Erro = risco elevado; requer evidências, validação independente e redundância (IEC, 2010; ISO, 2017; RTCA, 2011).
MÓVEL INDUSTRIAL
Campo e off-line; demanda sincronização robusta e UX em condições adversas.

SERVIÇO EM NUVEM
Multiusuário; precisa de observabilidade, SLO/SLI e segurança por padrão (Pressman; Maxim, 2016).
APOIO À DECISÃO
Dados/modelos; foca qualidade de dados, explicabilidade e ciclo de vida do modelo.

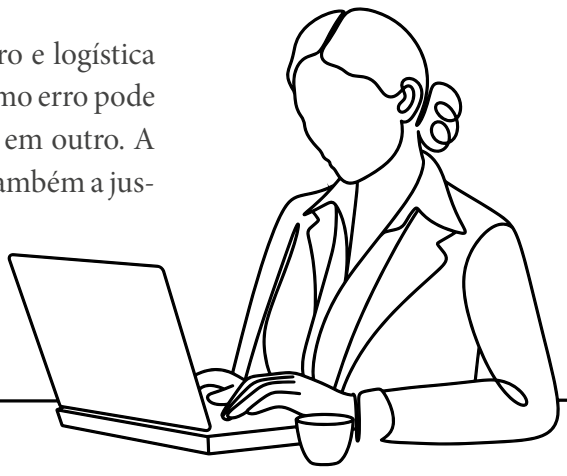
Fechamos essa parte entendendo quais lentes usar (disponibilidade, P95, rastreabilidade, segurança, observabilidade) e como o contexto altera o peso de cada uma. Agora, para aplicar de verdade, precisamos mapear onde cada software vive no ecossistema e o que é crítico medir em cada camada.

DECIDIR NO MUNDO REAL

A teoria só vira competência quando informa a priorização sob pressão. Agora, você treinará o olhar para incidentes reais, para nomear domínio e risco dominante e amarrar decisão à métrica, sempre com evidência mínima, a fim de sustentar o próximo passo.

Domínios e riscos

Os domínios como saúde, manufatura, agro e logística **mudam a régua de aceitabilidade**. O mesmo erro pode ser tolerável em um contexto e inaceitável em outro. A matriz ajuda a comparar prioridades, mas também a justificar por que uma fila, um alerta ou uma reconciliação vem primeiro.



A Figura 3 apresenta uma matriz cujas linhas representam domínios (ex.: telecom, defesa, saúde, financeiro) e as colunas, requisitos de produto/serviço (ex.: desempenho, segurança, confiabilidade, latência, privacidade, interoperabilidade, operação off-line, custo). Cada célula indica a prioridade/força do requisito no domínio (alto/médio/baixo) por cor/ícone, com padrão alternativo para P&B (hachuras/contornos). A leitura rápida destaca *trade-offs* (ex.: segurança x latência), bem como orienta decisões de arquitetura, priorização de *backlog* e ordem de testes/*builds* para cada domínio.

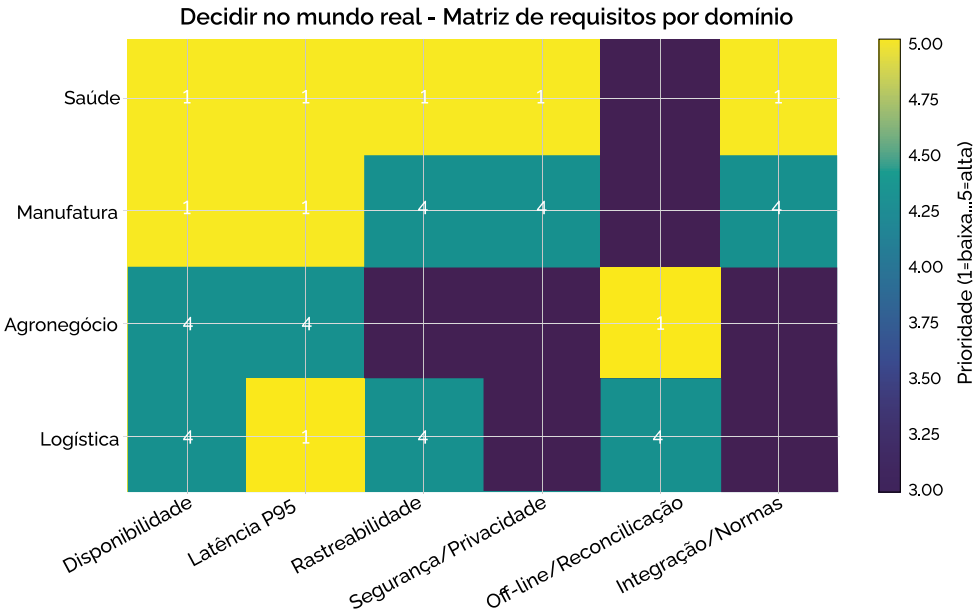


Figura 3 – Decidir no mundo real: matriz de requisitos por domínio / Fonte: adaptada de IEC (2010), ISO (2016; 2018; 2022), Pressman e Maxim (2021), RTCA (2011) e Sommerville (2018).

Descrição da imagem: um mapa de calor 4x6 intitulado "Decidir no mundo real – Matriz de requisitos por domínio". Ele compara Saúde, Manufatura, Agronegócio e Logística frente a seis lentes: Disponibilidade, Latência P95, Rastreabilidade, Segurança/Privacidade, Off-line/Reconciliação e Integração/Normas. Cada célula da matriz é colorida em tons que variam de roxo a amarelo conforme a prioridade (de três a cinco) indicada por uma barra de cores à direita, com números brancos dentro das células mostrando o valor exato de prioridade. Saúde aparece com prioridade máxima nas cinco primeiras lentes e alta conformidade; Manufatura enfatiza Disponibilidade e Latência; Agronegócio destaca Operação Off-line; Logística prioriza Latência P95 e Boa Rastreabilidade. Fim da descrição.

Pense em três chamados chegando antes do café: o painel parou de exibir um alerta de sensores; uma ordem emitida no módulo de gestão "sumiu" antes de chegar à execução; o app de campo coletou dados off-line e, ao sincronizar, criou conflitos. Qual vem primeiro?

Em vez de "quem grita mais alto", priorize pelo impacto: pessoas, operação, dinheiro, privacidade. Em saúde, alerta invisível é inaceitável; na fábrica, ordem sem confirmação pode parar a linha; no campo, sincronização conflituosa atrasa decisões e gera retrabalho. Nomear o domínio e o risco dominante reduz ruído, bem como orienta a ação certa: para alertas, visibilidade e tempo de resposta; para ordens, confirmação e idempotência; para coleta off-line, reconciliação e qualidade do dado.

Nomear o domínio e o risco dominante reduz ruído, bem como orienta a ação certa

No momento de sustentar a decisão, meça o básico, sempre do mesmo jeito. Disponibilidade por turno (% e minutos fora do ar) mostra estabilidade da operação. O P95 de resposta por etapa, por exemplo, emissão → confirmação, revela a pior experiência típica que a média esconde. Erros por integração (%, ‰) expõem pontos frágeis entre sistemas. Em cenários off-line, acompanhe conflitos a cada 100 envios e tempo médio de **reconciliação** até ficar limpo. Se houver SLO para um alerta crítico (ex.: P95 ≤ 2 s), use-o como régua de escalonamento e registre a trilha: requisito → *issue* → PR → pipeline → release → painel, para que qualquer auditoria encontre **evidências** sem planilhas paralelas.

Com esse quadro montado, as respostas tornam-se previsíveis: para o alerta invisível, revisar ingestão/eventos e degradar com canal redundante; para a ordem "sumida", conferir filas, idempotência e confirmação explícita; para os conflitos, aplicar *merge* determinístico e regras de precedência (carimbo de tempo, origem confiável). Poucas métricas, medidas iguais transformam debate subjetivo em **prioridades defensáveis**.

Se a decisão passa por integração, esse roteiro evita lacunas típicas: contrato claro, comportamento sob falha, observabilidade e "*evidence as code*". Ele é curto de propósito, para você conseguir aplicar já no próximo PR.

Com o domínio definido e as métricas mínimas na mão – disponibilidade por turno, P95 por etapa, erros por integração, conflitos/100 envios – suas escolhas deixam de ser opinativas, tornam-se defensáveis e repetíveis.

Integração e evidências

Integração não é "colou e pronto": é **previsibilidade verificável**. Contratos versionados, testes de contrato no pipeline, idempotência, DLQ e métricas por integração compõem o mínimo profissional. Evidência deixa de ser anexo manual e vira artefato da entrega.

Quando falamos em integração, o objetivo não é "fazer funcionar uma vez", mas tornar verificável e previsível cada troca entre sistemas. Isso começa pelo contrato: definir o que entra e o que sai, com exemplos reais (campos obrigatórios, códigos de erro, idempotência) e regras de compatibilidade. Em **APIs síncronas**, isso se traduz em OpenAPI com versionamento e testes de contrato; em eventos, esquemas versionados (ex.: Avro/JSON) e política clara às mudanças "quebrantes" versus compatíveis. Contrato sem teste vira papel; portanto, *consumer-driven contracts* e suítes de regressão rodam no pipeline a cada PR, cortando integrações quebradas antes de chegar à homologação.

Integração robusta também depende do comportamento certo sob falha

Integração robusta também depende do comportamento certo sob falha. Mensagens podem chegar duas vezes, fora de ordem ou atrasadas; por isso, trate idempotência e ordenação como requisitos, não como detalhe. Em *request/response*, implemente *retries* com *backoff* e *timeouts* coerentes; em filas e *streams*, separe DLQ (*dead letter queue*) para casos não tratáveis e monitore taxa de reaplicação. Decisões simples evitam grandes dores: confirmar explicitamente o que foi aplicado, registrar uma chave de deduplicação e preferir operações *append-only* em domínios que toleram reconciliação.

Para a integração ser auditável, ela precisa ser **observável**. Três sinais mínimos: logs estruturados com correlação (*trace/transaction id*), métricas por integração (latência P50/P95, taxa de erro, *throughput*) e *tracing* distribuído ligando a chamada de origem ao serviço *downstream*. Esses sinais alimentam SLIs/SLOs específicos ("confirmação em até 2s-P95", "erro ≤ 0,5%") e permitem acionar orçamento de erro quando a meta estoura: estabiliza-se primeiro, avança-se depois. Pós-*deploy*, uma verificação ativa (*smoke/health checks* com dado sintético) encerra a entrega com evidência real de que o caminho ponta a ponta está íntegro.



Evidência não é anexo manual: é *evidence as code*. O pipeline publica junto da release os relatórios de testes, o sumário de segurança (dependências, segredos) e o *changelog* que explica o impacto. No repositório, a linha de requisito → *issue* → PR → pipeline → release → SLO fica navegável; no painel, um cartão por integração mostra saúde atual e histórico recente. Quando algo falha, o *post-mortem* leve e sem culpa referencia exatamente essas peças: o contrato que não cobriu um caso, o alerta que não disparou, a correção que estabilizou a taxa de erro.

O resultado é um ciclo no qual integração, medição e aprendizado andam juntos; menos teoria, mais provas de que o sistema conversa bem e continua conversando à medida que evolui.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Entrar no mercado não exige "saber tudo", mas reconhecer o tipo de software em jogo, priorizar pelo risco e medir o que importa. Em ambientes industriais, três sinais sustentam decisões: **disponibilidade**, **P95 de tempo de resposta** e **erros por integração**, sempre com **evidências** (logs, prints, relatórios) que tornem a decisão auditável.

No dia a dia, como analista/dev júnior, mapeie o fluxo **emissão → confirmação**, meça P95 e indisponibilidade por turno, proponha um plano B simples (fila local com reenvio) e registre antes/depois. Em qualidade, traduza requisitos em critérios de aceite mensuráveis ("alertas visíveis em até 2s-P95"), automatize checagens essenciais e gere relatórios reprodutíveis. Em operações, monitore disponibilidade, P95, bem como erros por integração e faça triagem por impacto (pessoas, operação, dinheiro, privacidade). Em dados, estabeleça régua de qualidade/linhagem e alerte para *drift* que ameace a decisão.

Os recrutadores buscam sinais objetivos: identificar corretamente o problema ("era sincronização no app de campo, não falha de nuvem"), priorizar com métrica ("P95 emissão → confirmação de 3s para 12s, atacar ali primeiro"), demonstrar rastreabilidade (requisito ↔ teste ↔ resultado no mesmo quadro) e tratar segurança como padrão (segredos gerenciados, separação operação/escritório, atualização segura). Seu portfólio pode provar isso com peças simples: um **estudo curto** (1-2 páginas) de incidente simulado com métricas, decisão e *trade-offs*; um **diagrama de integração** sem siglas, indicando quem fala com quem, eventos/confirmas, ponto único de falha e plano B; uma **checklist de NFRs** por categoria (dispositivo, execução, gestão, nuvem, móvel, crítico, apoio à decisão) com 2-3 métricas por item; e um **miniroteiro de validação** com casos essenciais e evidências.

1. Organizações que operam serviços digitais costumam acompanhar sinais do comportamento do sistema a partir da observação do uso real. Esses sinais podem refletir tanto a fluidez das interações quanto a estabilidade dos processos ao longo do tempo. A definição de metas considera o contexto de negócio, as expectativas de quem utiliza o serviço e os limites técnicos conhecidos (Beyer *et al.*, 2016; ISO, 2019).

   Qual par representa melhor SLIs úteis para um painel de integração (nuvem) que recebe eventos da fábrica?

   a) Média de latência e número total de usuários cadastrados.
   b) P95 de latência de ingestão e taxa de erro por integração.
   c) Tamanho do time e número de *deploys* por mês.
   d) Métrica de CPU e quantidade de *commits* no repositório.
   e) Nível de satisfação subjetiva e volume de reuniões semanais.

2. Em domínios críticos, processos de desenvolvimento podem exigir práticas proporcionais ao risco associado à operação. A rastreabilidade de requisitos, a validação e a gestão de mudanças são elementos frequentemente considerados. Diferentes setores adotam referenciais específicos para orientar essas práticas ao longo do ciclo de vida (IEC, 2010; 2015; ISO, 2018).

   Em qual cenário a IEC 62304 é tipicamente aplicada?

   a) Software de controle de aeronaves.
   b) Software de usina de geração.
   c) Software automotivo embarcado.
   d) Software de dispositivo médico.
   e) Software de ERP financeiro.

3. As integrações entre camadas operacionais e de gestão podem definir expectativas explícitas sobre entradas e saídas. Os cenários de falha e tentativas repetidas demandam estratégias conhecidas para evitar efeitos indesejados. O registro verificável das trocas pode apoiar a consistência entre sistemas heterogêneos (Newman, 2021; Richardson; Smith, 2018).

   Qual combinação favorece integrações confiáveis entre execução/manufatura e gestão?

   a) Requisições sem confirmação e logs desabilitados.
   b) Contratos versionados + idempotência + confirmação explícita de aplicação.
   c) *Retries* infinitos sem *backoff*, sem DLQ.
   d) Acordos verbais e testes manuais esporádicos.
   e) Replicação cega de dados sem reconciliação.

REFERÊNCIAS

BEYER, B. *et al*. **Site reliability engineering**: how Google runs production systems. Sebastopol: O'Reilly, 2016.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 28 nov. 2025.

FERNANDES, A. A.; TEIXEIRA, D. de S. **Fábrica de software**: implantação e gestão de operações. São Paulo: Atlas, 2011.

IEC – INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 61508**: functional safety of electrical/electronic/programmable electronic safety-related systems. Geneva: IEC, 2010. 7 parts.

IEC – INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 62304:2006+A1:2015** – medical device software – software life cycle processes. Geneva: IEC, 2006.

ISA – INTERNATIONAL SOCIETY OF AUTOMATION. **Programa de certificação em cibersegurança ISA/IEC 62443**. Research Triangle Park: ISA, 2002.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 13485**: medical devices – quality management systems – Requirements for regulatory purposes. Geneva: ISO, 2016.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 26262**: road vehicles – functional safety. Geneva: ISO, 2018.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 12207**: systems and software engineering – software life cycle processes. Geneva: ISO, 2017.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 20000-1**: service management system requirements. Geneva: ISO, 2019.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 27001**: information security, cybersecurity and privacy protection – information security management systems – Requirements. Geneva: ISO, 2022.

NEWMAN, S. **Building microservices**. 2nd ed. Sebastopol: O'Reilly, 2021.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

RICHARDSON, C.; SMITH, F. **Microservices patterns**. Shelter Island: Manning, 2018.

RTCA – RADIO TECHNICAL COMMISSION FOR AERONAUTICS. **DO-178C**: software considerations in airborne systems and equipment certification. Washington, DC: RTCA, 2011.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

CONFIRA SUAS RESPOSTAS

1. Alternativa B.

   SLIs devem refletir a experiência: latência em percentil (captura cauda) e erro por integração medem diretamente ingestão/qualidade.
   As demais alternativas não são SLIs de experiência ou usam média, que ignora caudas.

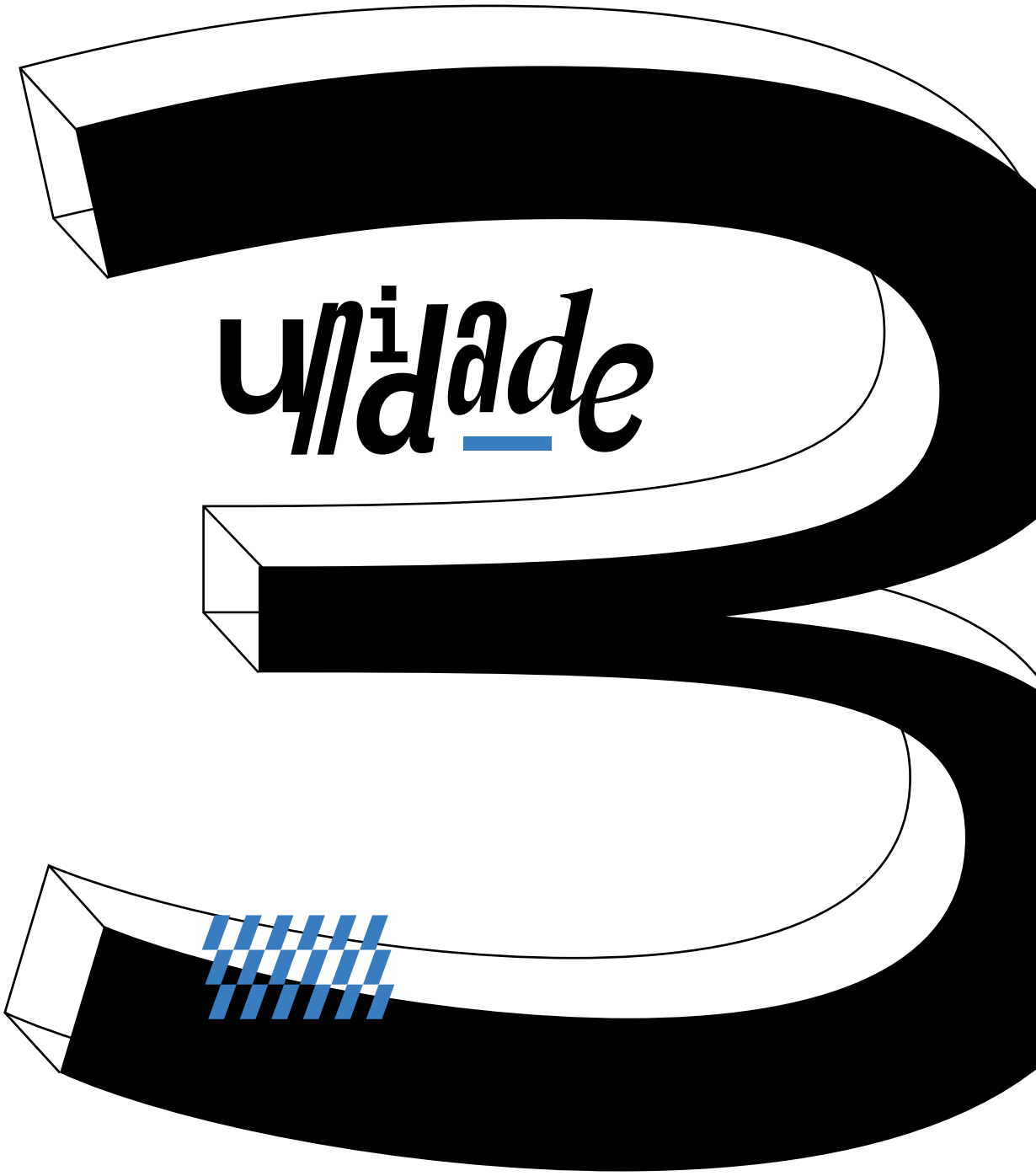
2. Alternativa D.

   A alternativa A está incorreta, pois a aeronáutica segue referenciais próprios, não a IEC 62304.
   A alternativa B está incorreta, pois a geração de energia é orientada por outras normas setoriais.
   A alternativa C está incorreta, pois o automotivo adota ISO 26262.
   A alternativa D está correta, pois a IEC 62304 é específica para software de dispositivos médicos; aeronáutico e automotivo são cobertos por outros padrões; ERP não é domínio de *safety*.
   A alternativa E está incorreta, pois ERP financeiro não se enquadra em *safety* funcional.

3. Alternativa B.

   A alternativa A está incorreta, pois a ausência de confirmação e de logs fragiliza confiabilidade e investigação.
   A alternativa B está correta, pois contratos versionados e idempotência evitam duplicidade/erro; confirmação explícita garante consistência. As demais enfraquecem a confiabilidade.
   A alternativa C está incorreta, pois *retries* sem *backoff* e sem DLQ podem amplificar falhas.
   A alternativa D está incorreta, pois acordos verbais e testes esporádicos não garantem comportamento estável.
   A alternativa E está incorreta, pois replicação sem reconciliação compromete integridade.

Unicesumar





TEMA DE APRENDIZAGEM 4

LINHA DE PRODUTO DE SOFTWARE (SPL – SOFTWARE PRODUCT LINE)

MINHAS METAS

- Compreender a SPL e porque gera escala, produtividade, padronização e previsibilidade via reutilização sistemática.
- Distinguir Engenharia de Domínio × Engenharia de Aplicação e seus papéis na SPL.
- Identificar comunalidades/variabilidades e modelá-las para derivar produtos.
- Reconhecer a arquitetura comum e os artefatos reutilizáveis como base do reuso.
- Entender fluxo de derivação, gestão de configuração e impactos da variabilidade nos testes.
- Explorar ferramentas e práticas para SPL em ambientes industriais e DevOps/CI.
- Analisar estudos de caso (telecom/defesa), avaliando benefícios, riscos e ROI.

INICIE SUA JORNADA

Você herda um portfólio com soluções muito parecidas, mas cada cliente puxou um "fio" que virou exceção: integrações duplicadas, regras de negócio reescritas, testes que não cobrem variações e prazos que estouram quando surge uma nova combinação. Por exemplo, 5G + retenção de 90 dias + modo *off-grid*. A pergunta que trava o time é simples:

VOCÊ SABE RESPONDER?

Vale continuar projeto a projeto, corrigindo as mesmas coisas em lugares diferentes, ou adotar uma Linha de Produto de Software (Software Product Line – SPL) para tratar o que é comum como ativo estratégico e o que varia como decisão explícita?

SPL não é moda: é reutilização sistemática com governança leve. Você consolida um núcleo comum – arquitetura, componentes, contratos, políticas – e explicita variabilidades em um modelo de features (Must/MAY/OR, com *requires/excludes*). Duas engrenagens dão tração: **Engenharia de Domínio** (modela a família, mantém os *core assets* e os contratos estáveis) e **Engenharia de Aplicação** (deriva produtos por decisão de features, sem copiar e colar). Resultado prático: escala e produtividade, menos retrabalho e evidências claras tanto de qualidade quanto de conformidade.

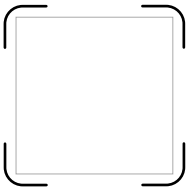
Para fechar, a ideia é conversar com você sobre **como começar pequeno**. Em 60 a 90 minutos, você já consegue rodar um ensaio rápido no seu contexto: escolher um sistema (real ou imaginado), listar o que é comum a qualquer entrega e o que varia entre clientes ou versões, desenhar de forma bem simples como esse fluxo caminha até a produção e imaginar que tipo de automação ajuda a evitar os problemas já vistos por você: *build* quebrando "do nada", diferenças de ambiente, falta de alerta.

Em vez de se preocupar com termos sofisticados ou ferramentas específicas, pense em entregáveis bem concretos: um quadro com o que se repete e o que muda, um rascunho de "linha de produção" do software e uma lista curta de verificações que não poderiam faltar antes de colocar algo no ar.

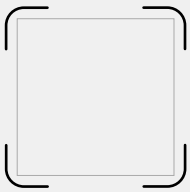
Esse exercício não é para "acertar tudo de primeira", mas para você começar a olhar seu trabalho com lentes de fábrica de software: onde padronizar, onde permitir variação, como usar automação e ambientes bem cuidados para diminuir retrabalho e aumentar a confiança nas entregas.

PLAY NO CONHECIMENTO

Veja como a **Linha de Produto de Software (SPL)** transforma projetos parecidos em uma **família de produtos**: núcleo comum forte, **variabilidade planejada**, **testes invariantes/condicionais** e **evidência no pipeline**. Entenda **Domínio** × **Aplicação** e mapeie **Comum** × **Varia** no seu contexto.



VAMOS RECORDAR?



Linha de Produto de Software (SPL) é uma estratégia de engenharia para produzir famílias de produtos a partir de um núcleo comum (arquitetura e artefatos reutilizáveis) e variabilidades explicitadas em modelos de features. Na prática, separam-se os trabalhos de **Engenharia de Domínio** (criar e manter os ativos-base e a arquitetura comum) e **Engenharia de Aplicação** (derivar produtos escolhendo combinações válidas). O ganho vem da reutilização sistemática, do tempo menor para derivar novos produtos e de uma qualidade mais previsível apoiados por configuração, testes invariantes/condicionais e evidências (logs, relatórios, SBOMs).

Nesse seminário, o professor Vander Alves discute justamente como automatizar e verificar análises em linhas de produto ao longo da evolução, conectando teoria de SPL com desafios práticos de escala, variabilidade e manutenção.

DESENVOLVA SEU POTENCIAL

DO PORTFÓLIO À DECISÃO DE ADOTAR SPL

Abrimos o tema posicionando a SPL como resposta a um problema recorrente: produtos muito parecidos sendo feitos como se fossem únicos. A intenção aqui é **conectar a sua realidade** – prazos estourados, retrabalho, integrações duplicadas – ao **conceito de família de produtos**. Primeiro, mostramos "por que" a SPL existe; depois, guiamos "como" começar: identificar o que é comum e o que varia e organizar quem cuida de cada parte.

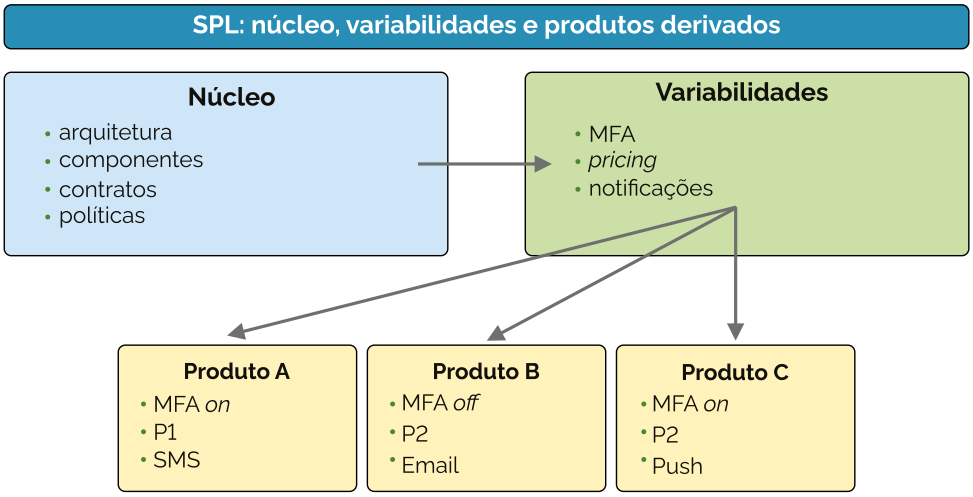


Figura 1 – SPL: núcleo, variabilidades e produtos derivados / Fonte: gerada pelo ChatGPT em 5 nov. 2025.

Descrição da imagem: diagrama de fluxo em formato de organograma com título "SPL – Núcleo Variáveis + Produtos" no topo, em caixa azul-claro. À esquerda, há uma caixa azul-claro maior dividida em duas partes verticais: a superior, rotulada "Político Núcleo", com um círculo verde do lado esquerdo, e a inferior, rotulada, "Componentes", com um círculo verde também do lado esquerdo. Dessa caixa saem setas horizontais para a direita. À direita dessa caixa, conectada por uma seta horizontal grossa, há uma caixa verde-claro rotulada "Notáveis Variáveis" com um círculo verde no centro e da qual saem três setas horizontais para três caixas amarelas dispostas lado a lado. Essas três caixas são: a primeira rotulada "Produto A" contendo "Núcleo A" em texto menor; a segunda rotulada "Produto B" contendo "Núcleo B"; a terceira rotulada "Produto C" contendo "Núcleo C". Fim da descrição.

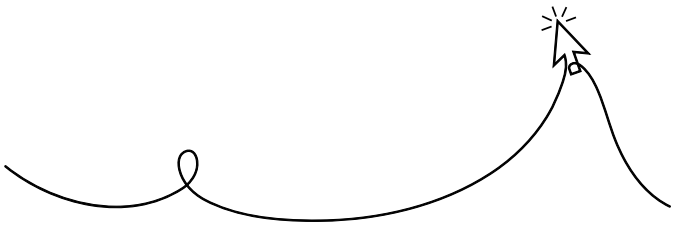
INDICAÇÃO DE FILME

O jogo da imitação

O filme conta a história de Alan Turing, matemático e pioneiro da computação moderna, cujos trabalhos desenvolveram a máquina que decifrou o código Enigma durante a Segunda Guerra Mundial. Turing mostrou que criar um modelo base reutilizável permitia gerar soluções múltiplas em menos tempo e com precisão.



Refletindo sobre a história: cada ajuste na máquina de Turing lembra a derivação de um produto em uma linha de software. Uma combinação diferente sobre a mesma base sólida.



O que é SPL e por que escala?

Antes de falarmos de modelos ou ferramentas, apresentaremos a SPL como uma estratégia de engenharia: construir uma família a partir de um núcleo comum (arquitetura, componentes, contratos, políticas) e variabilidades planejadas (features e regras). Explicamos que "escala" nasce quando a organização deixa de copiar e colar e passa a derivar produtos por decisão configurável. Você, estudante, provavelmente entende o termo no contexto do curso e vê a intenção: reduzir retrabalho e ganhar previsibilidade.

Uma **SPL** é, antes de tudo, uma forma de pensar produto em família: você investe em um núcleo comum – arquitetura, componentes, contratos e políticas – e planeja variabilidades explícitas que podem ser combinadas conforme o contexto. Em vez de reimplementar o mesmo esqueleto a cada projeto, a equipe deriva novas versões por decisão de *features* (o que liga/desliga, o que substitui o que), reduzindo retrabalho e mantendo consistência técnica.

Esse mecanismo, sustentado por reutilização sistemática, é o que transforma esforço repetido em produtividade real: menos código duplicado, menos divergência de contratos, menos correções reaparecendo em cada cliente (Sommerville, 2011).

PENSANDO JUNTOS

Olhe para seu produto atual como se fosse o primeiro membro de uma família. Quais os três elementos que **não podem mudar** entre clientes (contratos/APIs, eventos, logs) e quais os três que **precisam variar** (canal 4G/5G/rádio, retenção, criptografia)? O que hoje é "jeitinho" deveria virar **ponto de variação** explícito? Se você tivesse de provar qualidade amanhã, quais **duas evidências** mostrariam (teste invariante e condicional)? Por fim: essa decisão pertence a **Domínio** (núcleo) ou **Aplicação** (derivação)? Como isso impacta seu **ROI** nos próximos dois releases?

Quem faz o quê: domínio × aplicação (governança leve, efeitos práticos)

Aqui, definimos os papéis que dão tração ao conceito. **Engenharia de Domínio** modela a família (*feature model*, arquitetura comum, interfaces estáveis, *core assets*); **Engenharia de Aplicação** deriva produtos (seleciona combinações válidas, integra, evidencia qualidade).

Uma SPL funciona porque duas engrenagens giram coordenadas

Mostramos **como esses termos se relacionam**: Domínio cria as bases reutilizáveis; Aplicação colhe o ganho com segurança e rastreabilidade. Essa divisão explica **o fluxo lógico** do tema: primeiro padronizar o que todos usarão, depois derivar com rapidez.

Uma SPL funciona porque duas engrenagens giram coordenadas. **Engenharia de Domínio** modela a família: define o modelo de *features* (Must/MAY/OR, com regras *requires*/*excludes*), projeta a arquitetura comum, fixa interfaces estáveis e empacota *core assets* reutilizáveis (componentes, contratos versionados, templates de *build*/*deploy*, políticas de segurança e observabilidade). Tal ação dá nome às decisões, estabelece limites e evita que variações "escapem" para o coração dos contratos (ABNT, 2022; Pressman; Maxim, 2016).

Já a **Engenharia de Aplicação** usa esse catálogo para derivar produtos com segurança: seleciona combinações válidas, parametriza, integra e registra evidências do que foi entregue. Aqui, entram gestão de configuração e fluxo de derivação: cada escolha de *feature* aponta para uma versão de componente, aciona um conjunto de testes e produz artefatos versionados. O resultado é previsibilidade de prazo e qualidade, redução de disputas sobre "o que pode mudar", além de um caminho claro de aprovação para novas variações quando o negócio exigir (Sommerville, 2011).



DA ARQUITETURA À EVIDÊNCIA: QUALIDADE, DEVOPS/CI E VALOR DE CAMPO

Tendo decidido adotar SPL, agora, devemos conectar essa decisão ao software funcionando. A intenção é mostrar que não basta "ter um modelo": é preciso arquitetar para variar sem quebrar, testar o que importa e provar com evidências. Esse movimento prepara a compreensão da passagem do desenho para a operação (pipeline), fechando o ciclo com indicadores de valor.

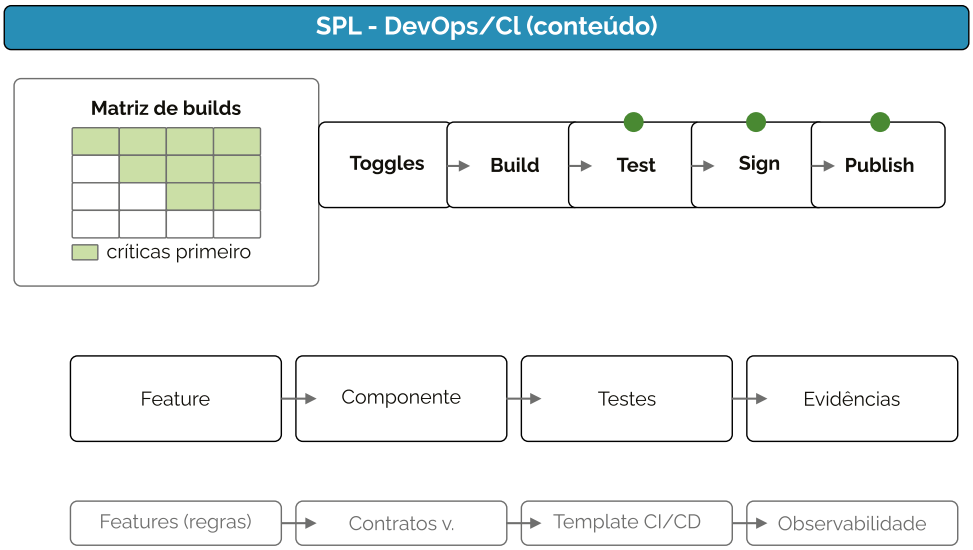


Figura 2 – DevOps/CI como lastro operacional da SPL: matriz de *builds*, pipeline, cadeia de gestão e práticas
Fonte: gerada pelo ChatGPT em 5 nov. 2025.

Descrição da imagem: diagrama de fluxo horizontal com título "SPL DevOps (Conteúdo)" em caixa azul-claro no topo. À esquerda, há uma caixa verde-claro rotulada "Matriz de Builds" contendo uma grade 2x2 menor com células vazias e o texto "Células" abaixo dela. Da borda direita dessa caixa saem setas sequenciais para caixas individuais dispostas em linha: "Toggles" (caixa cinza), "Build" (caixa laranja), "Test" (caixa vermelha), "Sign" (caixa roxa), "Publish" (caixa verde). Abaixo dessa linha, alinhadas verticalmente sob cada uma das caixas, há caixas brancas maiores: sob Toggles está "Feature" com "Features Flags" abaixo; sob Build está "Componente" com "Compilação x" abaixo; sob Test está "Testes" com "Templates GDC" abaixo; sob Sign está "Sinais" com "Observabilidade" abaixo. Fim da descrição.

Arquitetura comum, pontos de variação e impacto nos testes

Começamos definindo "arquitetura comum" como o conjunto de contratos estáveis (APIs, eventos, logs) que garantem compatibilidade entre produtos. Em seguida, explicamos "pontos de variação" como lugares seguros para mudar (módulos/*adapters*: canal, criptografia, retenção, idioma). Conectamos todo esse encadeamento aos testes: invariantes para o núcleo, condicionais para cada variação ativada. Você, estudante, entenderá a relação entre arquitetura, variabilidade e estratégia de teste e porque ela evita regressões em clientes diferentes.

Uma decisão só vira software confiável quando a arquitetura comum estabiliza contratos e a variabilidade fica isolada onde deve. Pense em camadas (apresentação, serviços, integração, dados) com APIs versionadas sem ruptura, esquemas mínimos de eventos e formatos obrigatórios de log como trilhos estáveis. Em torno deles, posicione pontos de variação em módulos ou adaptadores: canal (4G/5G/rádio), criptografia (políticas e cifras), retenção (30/90 dias), idioma. A imagem do modelo 150% ajuda: a especificação prevê todas as opções; cada produto real é um subconjunto válido escolhido por configuração, respeitando as regras do modelo (ABNT, 2022; Pressman; Maxim, 2016; Sommerville, 2011).

Essa organização define como realizar testes. O núcleo pede testes invariantes (autenticação/autorização, trilha de auditoria, validade de eventos, SLIs de latência e disponibilidade, sanidade de logs). As variações pedem testes condicionais: se 5G, valide *slicing* e metas de latência; se *off-grid*, valide fila local, persistência e reconciliação; se a cifra muda, valide interoperabilidade e rotação de chaves.

Mapear comunalidades/variabilidades na arquitetura torna nítido onde a variação acontece, qual contrato não pode quebrar e quais testes provam cada combinação, exatamente o objetivo de entender o impacto da variabilidade nos testes (ABNT, 2017; 2022; Pressman; Maxim, 2016; Sommerville, 2011).

Pipeline, ferramentas e estudos de caso, riscos e RO

Por fim, trazemos o cotidiano: DevOps/CI como lastro operacional da SPL. Definimos "matriz de *builds*" (priorizar combinações críticas), "*feature toggles*" (controlar caminhos) e "evidências" (relatórios de teste, *changelog*, artefatos assinados, SBOM). Conectamos com casos típicos (telecom/defesa) para mostrar aplicação real, apresentamos riscos (variabilidade vazando para contratos, explosão combinatória) e como medir ROI (tempo de derivação, reaproveitamento, defeitos). Assim, você, estudante, vê a sequência lógica completa: conceito – papéis – arquitetura – testes – operação – valor no mercado.

No cotidiano, DevOps/CI fornece o lastro operacional da SPL. Um pipeline com matriz de *builds* executa primeiro as combinações críticas (mais frequentes, reguladas ou arriscadas), controla caminhos com *feature toggles* e publica

evidências por derivação: relatórios de teste, *changelog*, artefatos assinados e o *Software Bill of Materials* (SBOM, uma "nota fiscal" de seu software) (ABNT, 2017; 2022; Pressman; Maxim, 2016; Sommerville, 2011).

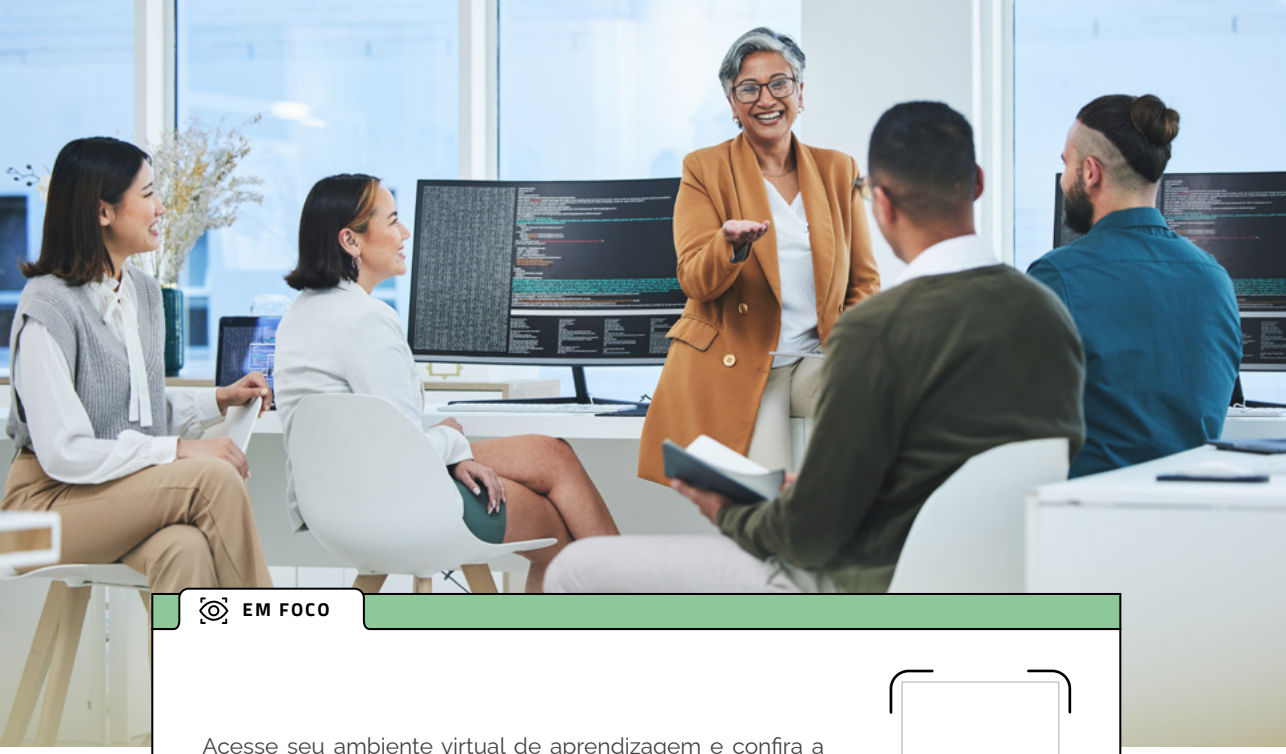
A gestão de configuração conecta tudo em cadeia: decisão de *feature* → versão do componente → casos de teste executados → evidências publicadas. As ferramentas e as práticas que ajudam: modelagem de *features* para dar visibilidade às regras, contratos versionados sem ruptura, templates de *build*/*deploy* reutilizáveis e observabilidade que capture, por produto, as rotas de execução ativas (Bass; Clements; Kazman, 2013).

Em **telecom**, o núcleo costuma reunir autenticação, fila, normalização de eventos, observabilidade e aderência à Lei Geral de Proteção de Dados Pessoais (LGPD); variam canais (voz/SMS/WhatsApp/VoIP), codecs e retenção. Os benefícios típicos são: tempo de derivação menor, alto reaproveitamento e conformidade simplificada; riscos: variabilidade vazando para contratos e explosão combinatória, mitigados por pontos de variação bem definidos e priorização inteligente na matriz de *builds* (Pressman; Maxim, 2016).

Na área de **defesa**, o núcleo inclui trilha de auditoria, criptografia, telemetria e controle de acesso; variam rádios (faixas/criptos), latência tolerada e modos degradados (operar sem rede). Os benefícios são: compatibilidade operativa sob restrições severas e homologações mais rápidas com evidências claras; risco principal: acoplamento indevido da variação ao núcleo, mitigado por adaptadores e políticas de ruptura explícitas (Bass; Clements; Kazman, 2013; Pressman; Maxim, 2016).

Para medir o **Retorno sobre o Investimento (ROI – *Return on Investment*)**, acompanhe por release três sinais simples: tempo de derivação (do núcleo ao produto), percentual de reaproveitamento (componentes e testes) e defeitos no núcleo vs na variação. Tempo em queda, reaproveitamento em alta e estabilidade de defeitos no núcleo indicam que a SPL está entregando valor. Se não, é hora de revisar interfaces estáveis, pontos de variação e a priorização da matriz do pipeline.

Assim, você fecha o ciclo dos objetivos: explorar ferramentas e práticas em DevOps/CI, analisar benefícios, riscos e ROI, mantendo a SPL como capacidade organizacional. Previsível para quem opera, auditável para quem regula e sustentável para quem desenvolve (Pressman; Maxim, 2016; Sommerville, 2011).



EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Encerrando este tema, amarraremos teoria e prática olhando para o que o mercado realmente pede quando fala em escala, previsibilidade e conformidade. A SPL que estudamos – núcleo comum e variabilidade planejada – torna-se diferencial competitivo quando se traduz em derivar produtos rapidamente, manter contratos estáveis (APIs/eventos/logs), testar o que importa (invariantes e condicionais) e publicar evidências (relatórios, SBOM, *changelog*). Em setores como telecom e defesa, isso significa atender a novas combinações (exemplo: 5G e retenção 90 dias e *off-grid*) sem *forks* e com auditoria simplificada.

No dia a dia, a separação entre **Engenharia de Domínio** (modela a família, cuida dos *core assets* e da arquitetura comum) e **Engenharia de Aplicação** (deriva o produto e comprova qualidade) reduz retrabalho, melhora *lead time* e dá linguagem comum a negócio, QA e operação.

Repare como as soluções propostas no início reaparecem, aqui, mais maduras:

- O quadro "Comum × Varia" tornou-se modelo de *features* (Must/MAY/OR, com *requires/excludes*) que guia derivação.
- O "esqueleto" tornou-se arquitetura comum com pontos de variação bem localizados, evitando variação "vazando" para contratos.
- A ideia de "testar melhor" tornou-se *pipeline* com matriz de *builds*, priorizando combinações críticas por risco/frequência e publicando evidências por derivação.
- O "medir valor" tornou-se ROI com três números fáceis de explicar: tempo de derivação, % de reaproveitamento (componentes/testes) e defeitos no núcleo vs. na variação.

Do ponto de vista de carreira, esse cenário converte-se em oportunidades claras:

- **Arquitetura/Plataforma (Domínio)**: cuidar do núcleo, contratos e evolução dos *core assets*. É onde decisões de longo prazo acontecem.
- **Times de Produto (Aplicação)**: derivar com segurança, integrar variações e sustentar evidências. É onde escala encontra prazo.
- **Qualidade/SRE/DevOps**: transformar exigências de variabilidade em automação de testes, observabilidade e rastros auditáveis (SBOM, relatórios, artefatos assinados).

Se você está começando, deixarei uma sugestão para treinar o que aprendemos. Você poderá montar um "**case de cinco minutos**" para portfólio:

- Diagrama de arquitetura comum com pontos de variação.
- Minimodelo de *features* com duas a três regras *requires/excludes*.
- Matriz A×B (exemplo: telecom/defesa) mostrando as escolhas.
- Recorte do pipeline com um relatório de testes e um SBOM.

É exatamente o tipo de prova concreta que os entrevistadores valorizam: derivação previsível, contratos que não quebram, evidência automatizada.

Experiência prática (olhar de campo): onde falha? Normalmente, em dois elementos: pontos de variação mal posicionados (quebram contratos a cada release) e testes que ignoram variabilidade (tudo passa no "feliz" e falha no cliente B).

O antídoto é simples e disciplinado: mover variação para bordas (*adapters*/estratégias), versionar contratos com política de ruptura e amarrar decisão de *feature* → caso de teste → evidência no CI.

VAMOS PRATICAR

1. Agrupar produtos em uma mesma família pode envolver representações que evidenciam o que é comum e as opções de variação. Essas representações procuram apoiar decisões consistentes ao longo do tempo, sem determinar antecipadamente tecnologias ou estruturas internas. Diferentes organizações adotam convenções para tornar explícitas restrições entre escolhas possíveis (Pohl; Böckle; Linden, 2005; Pressman; Maxim, 2016)

   Qual alternativa melhor caracteriza o papel de um modelo de *features* em SPL?

   a) Substituir diagramas de arquitetura por ilustrações de interface.
   b) Listar exclusivamente requisitos funcionais de um único produto.
   c) Descrever um conjunto de escolhas válidas/relacionamentos entre variações.
   d) Catalogar apenas bugs conhecidos para priorização.
   e) Mapear papéis de equipe e horários de reunião.

2. Ao projetar famílias de produtos, a definição de fronteiras claras ajuda a preservar comportamentos estáveis e a localizar diferenças. Soluções estruturais podem reduzir efeitos colaterais quando novas combinações são criadas. Em geral, interessam arranjos que apoiem mudanças previsíveis sem comprometer a compatibilidade (ABNT, 2022; Bass; Clements; Kazman, 2013).

   Em SPL, qual ação é mais coerente para preservar contratos e isolar variações?

   a) Centralizar toda lógica variável em um único módulo monolítico.
   b) Expor APIs estáveis e usar adaptadores/estratégias nas bordas variáveis.
   c) Compartilhar bancos de dados sem controle de esquema.
   d) Evitar versionamento de interfaces para reduzir sobrecarga.
   e) Eliminar logs para simplificar o fluxo.

3. Ao aplicar decisões de variação para formar um produto, pode ser útil manter relações entre escolhas, versões de componentes e registros de verificação. Essas relações favorecem o entendimento do que foi alterado e por que, sem impor uma tecnologia específica de implementação. O objetivo é apoiar o acompanhamento e a auditoria do ciclo de vida (ABNT, 2017).

   Qual encadeamento melhor representa uma derivação rastreável?

   a) Decisão – planilha pessoal – *deploy* manual.
   b) Decisão – componente (versão) – casos de teste executados – evidências registradas.
   c) Ideia verbal – código em produção – confirmação por e-mail.
   d) Demanda – atalho no servidor – aceite informal.
   e) Escolha de *feature* – alteração direta no banco – anúncio no chat.

REFERÊNCIAS

ABNT – ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 12207:2017** – Engenharia de software – Processos do ciclo de vida do software. Rio de Janeiro: ABNT, 2017.

ABNT – ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC/IEEE 42010:2022** – Engenharia de sistemas e software – Arquitetura de sistemas. Rio de Janeiro: ABNT, 2022.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Arquitetura de software em prática**. 3. ed. Porto Alegre: Bookman, 2013.

POHL, K.; BÖCKLE, G.; LINDEN, F. J. van der. **Software product line engineering**. Berlin: Springer, 2005.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011.

CONFIRA SUAS RESPOSTAS

1. Alternativa C.

   a) Incorreta: pois confunde representação de variações com aspectos de interface visual.
   b) Incorreta: pois ignora a família e as combinações válidas.
   c) Correta: pois modelos de features representam variantes e restrições (*requires/excludes/* grupos) de uma família.
   d) Incorreta: pois trata de gestão de defeitos, não de variabilidade.
   e) Incorreta: pois trata de alocação organizacional, não de variabilidade do produto.

2. Alternativa B.

   a) Incorreta: pois concentra acoplamento e dificulta evolução independente.
   b) Correta: pois interfaces estáveis com pontos de extensão isolam variações e preservam contratos.
   c) Incorreta: pois aumenta o risco de quebras e incompatibilidades.
   d) Incorreta: pois impede a compatibilidade controlada ao longo do tempo.
   e) Incorreta: pois remove evidências necessárias para diagnóstico e conformidade.

3. Alternativa B.

   a) Incorreta: pois não assegura registros confiáveis e auditáveis.
   b) Correta: essa alternativa relaciona decisão de variação a versionamento, testes e evidências.
   c) Incorreta: pois carece de controle e rastros verificáveis.
   d) Incorreta: pois depende de confirmação informal.
   e) Incorreta: pois altera dados sem trilhas adequadas.

MEU ESPAÇO



TEMA DE APRENDIZAGEM 5

AUTOMAÇÃO E VIRTUALIZAÇÃO EM FÁBRICAS DE SOFTWARE

MINHAS METAS

- Entender como a automação organiza o fluxo de valor, do *build* ao monitoramento.
- Aplicar Integração Contínua e Entrega Contínua (CI/CD) para reduzir variabilidade, encurtar ciclos e tornar entregas previsíveis.
- Usar observabilidade (métricas e logs) para enxergar gargalos e melhorar pipelines.
- Compreender os fundamentos da Infraestrutura como Código (IaC) para criar ambientes estáveis e reprodutíveis.
- Comparar virtualização e contêineres quanto a isolamento, portabilidade e gestão de ambientes.
- Entender o papel de plataformas internas na oferta de ambientes e serviços padronizados.
- Avaliar como padronização e governança apoiam múltiplos clientes/produtos e estratégias de release.

INICIE SUA JORNADA

Você, provavelmente, já viveu isso: ajusta algo simples hoje, amanhã, o *build* quebra "do nada". Em outro time, a mesma versão funciona; no seu, falha. O *deploy* atrasa porque cada ambiente é configurado de um jeito; o monitoramento não avisa a tempo e, quando vários produtos rodam em paralelo, cada entrega parece uma gambiarra nova. Nada disso é raro, é a rotina de equipes que ainda dependem de passos manuais, conhecimento escondido e muita tentativa e erro. É desse incômodo que nasce este tema de aprendizagem: como tornar o processo menos frágil, mais previsível e alinhado ao que o mercado realmente espera?

Quando falamos em automação, virtualização e fábrica de software, estamos falando sobre organizar o trabalho e reduzir a variabilidade que desgasta times inteiros. Cada *build* que falha sem motivo aparente custa tempo e confiança; cada diferença entre máquinas vira risco; cada falta de visibilidade sobre o que está em produção dificulta decisões rápidas. Ao olhar o desenvolvimento como um fluxo contínuo – da ideia ao *deploy* – os conceitos deixam de ser abstratos e passam a explicar problemas que você já enfrenta.

Além da automação do pipeline, é fundamental reconhecer os principais tipos de testes automatizados e entender o papel de cada um no fluxo de qualidade: testes unitários validam partes pequenas e isoladas do código, tais como funções, classes ou métodos, garantindo que cada componente se comporte corretamente de forma independente. Testes de integração verificam se módulos, serviços e camadas diferentes "conversam" bem entre si, identificando problemas em interfaces, dependências e troca de dados; já os testes funcionais avaliam o sistema do ponto de vista do usuário ou do processo de negócio, ao checar se as funcionalidades entregam o que foi especificado nos requisitos. Quando bem combinados em uma esteira de CI/CD, esses testes reduzem retrabalho, aumentam a confiança nos *deploys* e ajudam a manter a qualidade mesmo em ciclos rápidos de entrega.

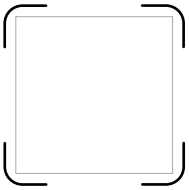
Estudante, o convite, aqui, é observar seu contexto com outros olhos: onde existem tarefas repetitivas? Quem depende de quem? Onde há "mágica" que só uma pessoa sabe fazer? E como tudo isso muda se parte do fluxo for automatizado? Às vezes, não é preciso grandes ferramentas: um script simples, um teste automatizado ou um log bem pensado já transformam como o time identifica falhas e entrega valor.

Por fim, automação e virtualização não são assuntos distantes, são parte da identidade profissional que você começa a construir agora. A pergunta é: qual tipo de engenharia de software você quer praticar? Uma que dependa da sorte, ou uma que confie em processos claros, ambientes reprodutíveis e entregas mais sustentáveis?

A jornada começa ao reconhecer esse fato: caos não é regra, e você pode ser protagonista na construção de um desenvolvimento mais organizado, colaborativo e preparado para o mercado de tecnologia.

PLAY NO CONHECIMENTO

Esse episódio mostra como **automação**, **CI/CD**, **IaC**, **observabilidade** e **virtualização/contêineres** transformam releases em rotina previsível, do *build* ao monitoramento. Dê o play, anote uma a duas pequenas ações (um teste automatizado, um *health check*, um trecho de IaC) e já aplique hoje no seu fluxo.



VAMOS RECORDAR?

Antes de avançar para automação, virtualização e plataformas internas, vale retomar alguns pontos que você viu em temas anteriores: a ideia de fábrica de software como organização de processos (da demanda à entrega), a importância de modelar requisitos e arquitetura com clareza, o papel das métricas para enxergar qualidade e desempenho, a noção de fluxo de valor que atravessa times e tecnologias. Se quiser reforçar essa base, volte a algum trecho do livro ou aos vídeos indicados sobre ciclo de vida do software, fábricas de software e gestão de qualidade: observe como ali já apareciam, mesmo que de forma menos explícita, questões de padronização de processos, reutilização de componentes, rastreabilidade e medição contínua. Esses elementos serão o "chão firme" para entender como automação e virtualização ajudam a tornar esse fluxo mais previsível, reprodutível e alinhado às exigências do mercado.

DESENVOLVA SEU POTENCIAL

AUTOMAÇÃO DO FLUXO DE VALOR NA FÁBRICA DE SOFTWARE

Automatizar o fluxo de valor na fábrica de software significa transformar o caminho da ideia até a produção em uma esteira previsível, rastreável e menos dependente de improvisos. Em vez de passos manuais espalhados (*build*, testes, empacotamento, *deploy*), cada etapa começa a ser tratada como um bloco automático, com critérios claros de entrada e saída.

Pipelines cuidam do repetitivo, ambientes são criados por código, as execuções geram métricas que mostram onde estão os gargalos. Assim, a equipe reduz variação entre projetos, evita "surpresas" em produção e consegue concentrar energia naquilo que realmente gera valor para o negócio.

APROFUNDANDO

Quando você lê sobre automação, IaC e plataformas internas, é fácil imaginar apenas ferramentas específicas (Jenkins, GitLab, Docker, Terraform etc.). O ponto central, porém, não é a ferramenta, e sim **de que forma você pensa o sistema como um todo**: fluxo contínuo de mudança, ambientes reprodutíveis e decisões baseadas em evidências. Tente sempre dar um passo atrás e perguntar: "se eu automatizar esse trecho, o que muda no fluxo?", "se eu descrever esse ambiente em código, o que ganho em diagnóstico e auditoria?", "se eu coletar essa métrica, que decisão concreta ela me ajuda a tomar?". Esse olhar crítico transforma o tema de aprendizagem de um conjunto de tecnologias da moda em um jeito mais profissional de enxergar a engenharia de software.



Do requisito ao deploy: pensando em fluxo

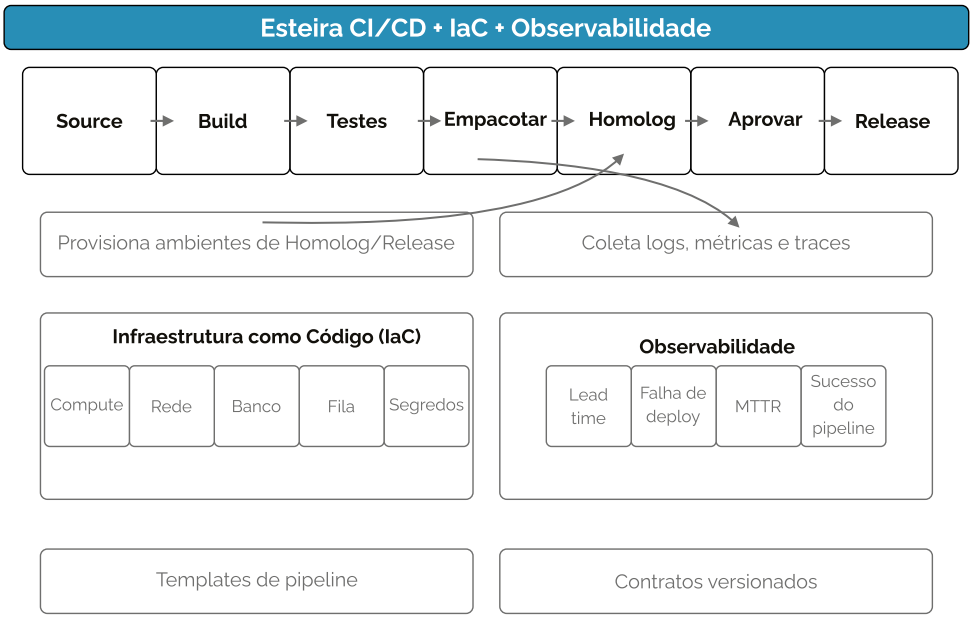


Figura 1 – Esteira CI/CD + IaC + observabilidade: pipeline, *call-outs* de provisão/telemetria, painéis de IaC e observabilidade e políticas de reuso / Fonte: gerada por ChatGPT em 1 dez. 2025.

Descrição da imagem: diagrama em quatro níveis cujo topo mostra uma pipeline com sete cartões organizados nessa sequência, da esquerda para a direita: "Source – Build – Testes – Empacotar – Homolog – Aprovar – Release". Cada termo aponta para o próximo, por meio de uma pequena seta. Abaixo desses cartões, há dois balões: "Provisiona ambientes de Homolog/Release", de onde sai uma seta longa que aponta para o cartão "Homolog", e "Coleta logs, métricas e traces", que recebe uma seta longa que sai do pipeline. No terceiro nível, dois quadros: "Infraestrutura como Código (IaC)" com chips "Compute", "Rede", "Banco", "Fila", "Segredos"; e "Observabilidade" com chips "Lead time", "Falha de deploy", "MTTR", "Sucesso do pipeline". No rodapé, há cartões "Templates de pipeline" e "Contratos versionados". Fim da descrição.

Quando você observa apenas "tarefas isoladas" (codificar, testar, subir em produção), é fácil perder-se em urgências e apagar incêndios. A fábrica de software propõe outra lente: olhar para o **fluxo de valor** completo, da chegada da demanda até o sistema em produção, gerando resultado.

Como podemos ver pela **Figura 1**, Fernandes e Teixeira (2011) descrevem fábricas nas quais esse fluxo é organizado em etapas claras – concepção (*source*), construção (*build*), testes, homologação, implantação – apoiadas por ferramentas de gestão, medição e rastreabilidade, reduzindo os improvisos e a dependência de "heróis".

Nessa perspectiva, automação não é "rodar um script esperto", mas tornar o fluxo **repetível**, **observável** e **confiável**. Cada etapa importante – *build*, testes, empacotamento, criação de ambientes, *deploy* – vira um bloco explícito do processo, com critérios de entrada e saída bem definidos. O ganho não está apenas em velocidade, mas em previsibilidade: você passa a saber quanto tempo, em média, uma mudança leva do *commit* até a produção e em quais pontos o fluxo costuma "travar".

PENSANDO JUNTOS

Pegue um sistema do seu contexto ou um cenário hipotético e escreva, em poucas linhas, o caminho de uma mudança simples: de "*feature* aprovada" até "versão em produção". Identifique onde existem passos manuais, quem são as pessoas-chave, sem as quais nada acontece, e em quais pontos quase nunca há registro do que foi feito. Esses são candidatos naturais à automação e à padronização.

Integração Contínua e Entrega Contínua (CI/CD) como esteira padrão

A Integração Contínua (CI) é o hábito de integrar código com frequência no repositório principal, disparando automaticamente *build* e testes para verificar se o sistema continua íntegro.

A Entrega Contínua (CD) estende essa prática até o *deploy*: cada mudança aprovada está, em princípio, pronta para ser colocada em produção, porque o caminho até lá está automatizado e protegido por verificações.

Humble e Farley (2013) descrevem CI/CD como um conjunto de práticas técnicas e organizacionais que transforma releases em eventos frequentes e previsíveis, e não em "operações de guerra".

Em termos práticos, um pipeline típico de CI/CD inclui etapas como:

- *Build* e análise estática do código.
- Testes automatizados (unitários, integração, contrato).
- Empacotamento e publicação de artefatos.
- *Deploy* automatizado em ambientes de teste, homologação e produção.
- Mecanismos de *rollback* e validação pós-*deploy* (*smoke tests, health checks*).

Bass, Weber e Zhu (2015) chamam atenção para o papel arquitetural desses pipelines: ao decidir que verificações são obrigatórias, quem pode promover versões e quais evidências de qualidade precisam ser registradas, a organização está tomando **decisões de arquitetura** tão relevantes quanto a escolha de *frameworks* e padrões de projeto.

EXEMPLIFICANDO

> Imagine um pipeline em que toda nova versão: (a) compila; (b) roda testes automatizados; (c) gera um pacote versionados; (d) atualiza automaticamente o ambiente de homologação; (e) só pode seguir para produção após aprovação de revisão de código e execução bem-sucedida de testes de fumaça. Observe que, mesmo simples, esse fluxo já reduz fortemente o risco de "subir código quebrado" e cria trilhas de auditoria para cada decisão tomada.

Observabilidade e métricas para melhorar o fluxo

Automatizar sem medir é apenas "fazer mais rápido sem saber se melhorou". Em uma fábrica de software, a **observabilidade** – métricas, logs e rastreamento distribuído – conecta automação a decisões informadas.

Humble e Farley (2013) ressaltam que feedback rápido é central para entrega contínua: se cada *build*, cada teste e cada *deploy* deixam registros claros, torna-se possível ajustar o processo de maneira iterativa.

Um conjunto enxuto de indicadores já ajuda a guiar a melhoria contínua, em linha com a visão de processos de Pressman e Maxim (2016): **lead time de mudança**, **taxa de falha de deploy**, **tempo médio de recuperação (MTTR)** e **taxa de sucesso dos pipelines** mostram se a automação está, de fato, reduzindo variabilidade e risco. Quando esses números são acompanhados ao longo do tempo, o

time consegue perceber se uma mudança de arquitetura, uma nova etapa de teste ou uma reconfiguração de ambiente trouxe ganho real, ou apenas mais complexidade.

Selecione um desses indicadores e pense como medi-lo no seu contexto, por meio de algumas perguntas:

VOCÊ SABE RESPONDER?

- Quais eventos precisam ser registrados (exemplo: hora do *commit*, hora do *deploy*)?
- Em quais ferramentas esses dados já aparecem (repositório de código, plataforma de CI, monitoramento)?
- Que tipo de decisão você conseguiria tomar com base nesse número?

VIRTUALIZAÇÃO, INFRAESTRUTURA COMO CÓDIGO E PLATAFORMAS INTERNAS

Virtualização, infraestrutura como código e plataformas internas se combinam para transformar "máquinas soltas" em um ecossistema padronizado de ambientes que a fábrica de software consegue criar, ajustar e destruir com poucos comandos. Em vez de configurar servidores na mão, a equipe descreve recursos em código, versiona mudanças e garante que desenvolvimento, teste e produção sigam o mesmo modelo.

Contêineres e máquinas virtuais passam a ser oferecidos como serviços internos – bancos, filas, APIs, *clusters* – consumidos pelos times de produtos de forma segura e repetível. Com isso, cai o tempo gasto em manutenção manual, reduz-se o risco de diferenças entre ambientes e aumenta a capacidade de experimentar, bem como escalar soluções de forma coordenada.

Infraestrutura como código e ambientes reprodutíveis

Se os pipelines cuidam do caminho do código até a produção, a **Infraestrutura como Código (IaC)** cuida de como os ambientes que suportam esse código são definidos, versionados e atualizados. Em vez de "configurar servidor na mão"

e torcer para lembrar o que foi feito, a equipe descreve recursos – máquinas virtuais, redes, bancos, filas, segredos – em arquivos versionados e utiliza ferramentas tanto para criar quanto para atualizar ambientes a partir desse código.

Morris (2018) argumenta que IaC leva boas práticas de desenvolvimento – controle de versão, revisão de código, testes, automação – para o domínio da infraestrutura, facilitando reprodutibilidade e reduzindo divergências entre ambientes. Em uma fábrica de software, isso significa que **ambientes de desenvolvimento, teste e homologação podem ser recriados com o mesmo padrão**, o que favorece diagnósticos mais rápidos de defeitos e experimentos mais seguros com novas configurações.

Virtualização, contêineres e "ambientes como serviço"

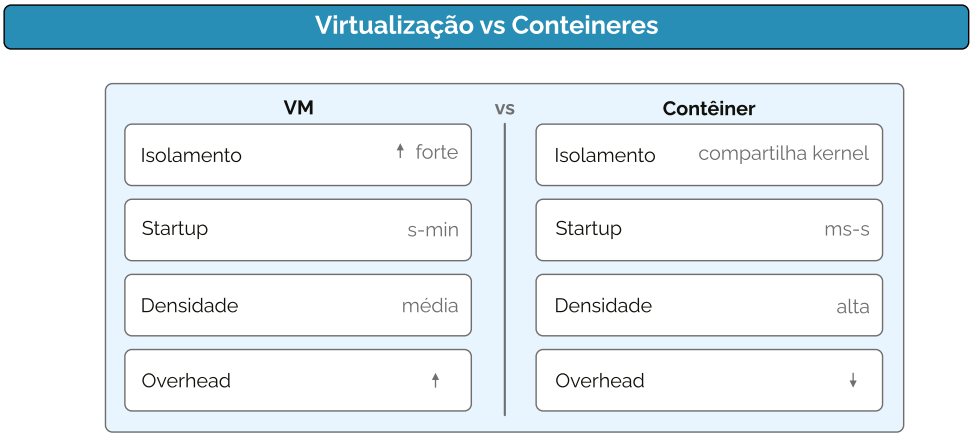


Figura 2 – Virtualização vs Contêineres: comparação de isolamento, tempo de inicialização, densidade e *overhead* / Fonte: gerada por ChatGPT em 1 dez. 2025.

Descrição da imagem: tabela comparativa intitulada "Virtualização vs Contêineres" em uma barra azul no topo. A tabela possui duas colunas principais: "Isolamento VM" à esquerda, e "Isolamento Contêiner" à direita, separadas por uma linha vertical. Na coluna esquerda, há três linhas: "Startup" com o termo "5 min" logo abaixo, "Densidade" com o termo "média" logo abaixo, e "Overhead" com o termo "alta" logo abaixo. Na coluna direita, as linhas correspondentes mostram "Startup" com o termo "ms" logo abaixo, "Densidade" com o termo "alta" logo abaixo, e "Overhead" com o termo "baixa" logo abaixo. Fim da descrição.

A virtualização permitiu que a fábrica passasse de servidores físicos dedicados a máquinas virtuais flexíveis, compartilhando recursos e facilitando a criação de ambientes sob demanda. Contêineres levam esse conceito adiante ao empacotar aplicações e suas dependências em unidades leves, isoladas e portáteis. Fernandes e Teixeira (2011) já apontavam a relevância de infraestruturas flexíveis para fábricas distribuídas, a fim de conectar times em diferentes locais a ambientes padronizados de desenvolvimento e teste.

Quando a organização combina virtualização, contêineres e IaC, surge a ideia de "ambientes como serviço": times de produto deixam de "montar ambiente" e passam a consumir catálogos internos ("banco pequeno com backup diário", "fila de mensagens com monitoramento básico", "*cluster* para testes de carga"), todos baseados em templates definidos pela área de plataforma. Para quem está no dia a dia de desenvolvimento, tal ação reduz tempo gasto com tarefas repetitivas, além de aumentar o foco em requisitos de negócio e qualidade de código.

EXEMPLIFICANDO

Compare dois cenários:

- Cada time "sobe" seu próprio banco, com versões, configurações e políticas de backup diferentes.
- Há um serviço interno que oferece bancos padronizados, criados a partir de IaC, já integrados ao monitoramento e às políticas de backup da organização.

No segundo caso, erros de configuração tendem a cair, incidentes ficam mais previsíveis e a fábrica consegue aprender com cada problema, refinando o template usado por todos.

EXEMPLIFICANDO

Podemos ainda diferenciar rapidamente nuvens privadas e públicas, já que ambas impactam diretamente como a fábrica de software entrega valor. Em uma nuvem pública, a infraestrutura é compartilhada entre vários clientes de um provedor (como AWS, Azure ou Google Cloud), com forte foco em elasticidade, pagamento sob demanda e rapidez para escalar novos ambientes, o que costuma ser vantajoso aos times que precisam experimentar, crescer rápido ou trabalhar com produtos digitais de maior variabilidade.

Já a nuvem privada dedica a infraestrutura a uma única organização, no próprio data center ou em ambiente exclusivo em provedor, oferecendo maior controle sobre segurança, conformidade e configuração, algo especialmente relevante em setores regulados ou com dados muito sensíveis.

Na prática, muitos arranjos adotam modelos híbridos, combinando nuvem pública para serviços que exigem escala e nuvem privada para sistemas e dados críticos. Dessa forma, equilibram custo, controle e desempenho conforme as demandas do negócio.

Governança, múltiplos clientes e decisões de plataforma

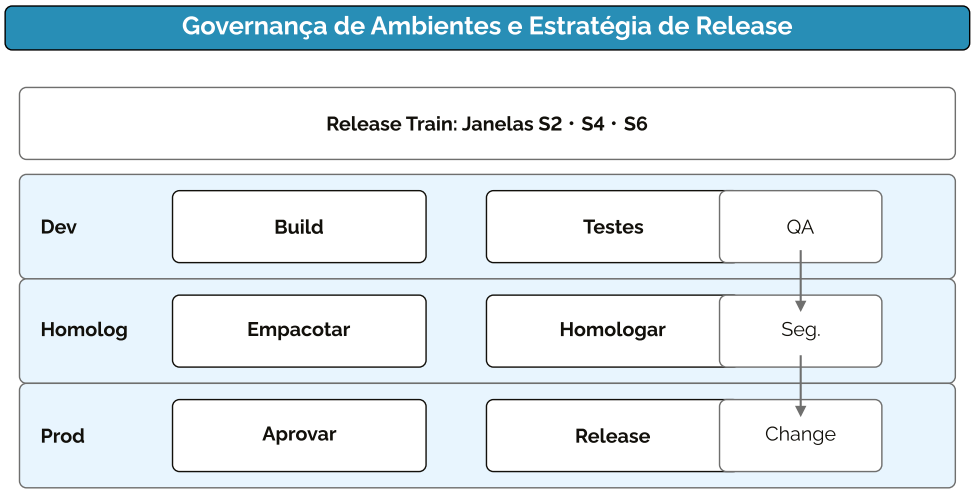


Figura 3 – Governança de ambientes e estratégia de release: trilhas Dev/Homolog/Prod com *gates* de QA, segurança e *change*, além de cadência de releases / Fonte: gerada por ChatGPT em 1 dez. 2025.

Descrição da imagem: diagrama de fluxo representando a governança de ambientes para a release 52-56, com título principal "Governança de Ambientes Estratégia Release 52-56" em uma caixa azul-claro no topo. O diagrama é organizado em uma tabela de duas linhas e quatro colunas, com linhas rotuladas como "Dev", "Homolog" e "Prod" na parte inferior esquerda de cada seção vertical, e colunas separadas por linhas verticais. Na primeira linha, sob "Dev", há caixas "Build" e "Testes", seguidas de uma caixa "QA" que se estende por duas colunas; sob "Homolog", há "Empacotar" alinhado abaixo de "Build" e "Homologar" abaixo de "Testes", com uma seta fina apontando da caixa "Homologar" para a caixa "QA". Na segunda linha, sob "Prod", há caixas "Aprovar" abaixo de "Empacotar" e "Release" abaixo de "Homologar", com uma caixa "Change" à direita de "Release" que contém o texto "Release Train: 52-56". No rodapé da imagem, há repetições do texto "Build Release Train: 52-56", "Testes Release Train: 52-56", "Homolog Release Train: 52-56" e "QA Testes", alinhados abaixo das respectivas seções. Há setas verticais ligando "QA", "Seg". e "Change", representando a passagem controlada entre ambientes até a liberação em produção. Fim da descrição. .

Em organizações que atendem múltiplos clientes ou mantêm um portfólio diversificado de produtos, a fábrica precisa equilibrar padronização, qualidade e variação. Como ilustrado na **Figura 3**, Fernandes e Teixeira (2011) discutem modelos de fábrica nos quais o escopo de fornecimento – projetos, programas, serviços recorrentes, off-shore, fábrica virtual – e a estrutura organizacional determinam que partes do processo são comuns e quais variam conforme cliente ou produto.

Na camada de automação e virtualização, essa governança materializa-se em decisões como:

- Quais templates de pipeline e módulos de IaC são obrigatórios para qualquer novo sistema.
- Como ambientes são segregados (por cliente, por domínio de negócio, por criticidade).
- Quais métricas e requisitos mínimos de qualidade todos devem cumprir (exemplo: disponibilidade-alvo, cobertura mínima de testes, padrões de log e auditoria).

Pressman e Maxim (2016) reforçam que a gestão de portfólio e de processos em engenharia de software envolve não apenas escolher tecnologias, mas estabelecer **pontos de decisão e de controle (gates)** que mantenham a fábrica sustentável a longo prazo.

Quanto mais essas decisões forem refletidas em código (pipelines, infraestrutura, políticas automatizadas), menos a organização dependerá de acordos informais e mais conseguirá alinhar escalabilidade, segurança e velocidade de entrega.



PENSANDO JUNTOS

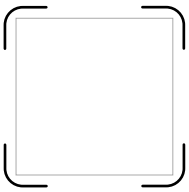
Imagine o "mínimo comum" da plataforma interna da sua fábrica. Por exemplo, todo pipeline poderia ter *build*, testes automatizados, checagens de qualidade/segurança, empacotamento e *deploy* com um *smoke test* final. Nos serviços padronizados de infraestrutura, entram itens como: banco de dados com backup configurado, fila de mensagens, cache, autenticação/autorização, monitoramento e log centralizado. Como métricas mínimas, pense em: disponibilidade, latência (média/P95), taxa de erros, *lead time* de mudança e MTTR. Use esses exemplos de ponto de partida para registrar o seu "mínimo comum" de pipeline, serviços e métricas, depois, marque dois ou três aspectos que realmente variam entre clientes (regras, volume, privacidade) para tratar como parametrizações, e não como exceções manuais.

Para encerrar essa etapa, vale olhar em retrospecto e perceber o quanto você avançou: deixou de enxergar apenas "tarefas soltas" de desenvolvimento para reconhecer um fluxo de valor automatizado, sustentado por pipelines, infraestrutura como código, virtualização e plataformas internas. Viu que não se trata de decorar siglas, mas de pensar engenharia de software como capacidade organizacional, ou seja, padronizar o que é comum, tratar variações de forma explícita, medir o que importa e usar esses sinais para melhorar continuamente.

A partir daqui o desafio será levar esse olhar para os contextos em que você atua ou atuará: questionar onde a automação faz sentido, propor mínimos comuns de pipeline, serviços e métricas, bem como participar ativamente da construção de uma fábrica de software mais previsível, sustentável e alinhada às demandas reais do mercado.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.



NOVOS DESAFIOS

Você caminhou da ideia abstrata de "fábrica de software" para enxergar fluxos, pipelines, automação, infraestrutura tal quais código, virtualização e plataformas internas como partes de um mesmo sistema sociotécnico.

O próximo passo é **transformar esse conhecimento em posicionamento profissional**: entender como essas práticas aparecem em ambientes reais de desenvolvimento e quais competências são esperadas de você em times que lidam com entrega contínua, disponibilidade, segurança e governança.

Na prática, o mercado anuncia vagas de desenvolvedor, engenheiro DevOps, SRE, analista de qualidade, engenheiro de plataforma, arquiteto de software. Em todas essas funções, surgem demandas como: ler e ajustar pipelines de CI/CD; colaborar na definição de ambientes reprodutíveis com IaC; interpretar métricas de produção (erros, latência, disponibilidade); propor automatizações que reduzam trabalho manual e risco; participar de decisões sobre templates de infraestrutura, logs e monitoramento. Quem entende o fluxo de ponta a ponta e sabe dialogar com desenvolvimento, operações e negócio tende a ter mais autonomia, mas também visibilidade.

Ao mesmo tempo, o cenário típico é de **transição**. Legados convivem com contêineres, parte da infraestrutura ainda é manual, nem tudo tem métrica, nem todo *deploy* é automatizado. É aí que surgem seus novos desafios: contribuir com pequenas melhorias consistentes – automatizar um trecho de *build* ou teste, descrever um ambiente em código, propor um indicador simples para acompanhar o fluxo, sugerir padronização de logs – em vez de esperar por uma "grande transformação" perfeita. Seu papel é ajudar a fábrica a dar passos seguros, articulando teoria e restrições reais de custo, prazo e cultura.

Como exercício de preparação para o mercado, desafie-se a analisar descrições de vagas e identificar menções a CI/CD, IaC, contêineres e observabilidade; montar um pequeno pipeline para um projeto simples; descrever em uma página o fluxo de um sistema real e marcar o que poderia ser automatizado ou padronizado em uma fábrica de software.

Assim, o conteúdo deste tema deixa de ser apenas "matéria da disciplina" e passa a compor um **vocabulário profissional prático**, que você poderá usar em entrevistas, projetos e na sua atuação cotidiana em equipes de engenharia de software.

VAMOS PRATICAR

1. Os pipelines organizam verificações de forma repetível, com critérios que determinam quando uma mudança avança e quais registros ficam disponíveis para revisão. A configuração deve equilibrar custo, risco e velocidade, priorizando a visibilidade do que importa. O objetivo é alinhar execução, evidências e decisão (Humble; Farley, 2013; Pressman; Maxim, 2016).

   Qual alternativa melhor caracteriza um pipeline de CI/CD bem configurado?

   a) Aprovações manuais substituem critérios; logs não são necessários.
   b) Testes apenas em produção, para reduzir tempo de integração.
   c) Estágios com critérios objetivos e publicações de evidências que subsidiam a promoção de versões.
   d) Execução única ao final do projeto, sem registros intermediários.
   e) *Build local* sem integração com verificações.

2. Os mecanismos de observabilidade fornecem sinais complementares sobre comportamento do sistema e processo de entrega. Leituras consistentes permitem avaliar tendências, priorizar correções, ajustar o fluxo. A utilidade desses sinais depende de como são coletados, publicados e usados em decisões (Humble; Farley, 2013; Pressman; Maxim, 2016).

   Qual combinação representa melhor o uso de observabilidade no fluxo de entrega?

   a) Logs não estruturados e métricas apenas anuais.
   b) Métricas de pipeline, logs estruturados e *trace* para isolar gargalos e regressões.
   c) Alertas somente após incidentes graves.
   d) Métricas de negócios sem relação com execução técnica.
   e) Auditoria manual sem coleta automática.

3. Ao combinar virtualização, contêineres e infraestrutura como código, organizações podem ofertar ambientes padronizados enquanto serviços internos. Essa abordagem busca reduzir divergências de configuração, aumentar a previsibilidade e acelerar a criação de ambientes. A adoção depende do contexto e dos requisitos de operação (Bass; Weber; Zhu, 2015; Humble; Farley, 2013).

   Qual alternativa melhor expressa o conceito de "ambientes como serviço" na fábrica de software?

   a) Cada time configura manualmente seu banco, sua rede e seus backups.
   b) Provisionamento ad hoc conforme melhor prática individual.
   c) Catálogo interno com templates padronizados e criação via IaC integrada a monitoramento.
   d) Acesso direto a servidores físicos, para mais autonomia.
   e) Somente imagens de máquinas virtuais sem políticas comuns.

REFERÊNCIAS

BASS, L.; WEBER, I.; ZHU, L. **DevOps**: a software architect's perspective. Boston: Addison-Wesley Professional, 2015.

FERNANDES, A. A.; TEIXEIRA, D. de S. **Fábrica de software**: implantação e gestão de operações. São Paulo: Atlas, 2011.

HUMBLE, J.; FARLEY, D. **Entrega contínua**: como entregar software de forma rápida e confiável. Porto Alegre: Bookman, 2013.

MORRIS, K. **Infraestrutura como código**: gerenciamento de servidores na era DevOps. São Paulo: Novatec, 2018.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

CONFIRA SUAS RESPOSTAS

1. Alternativa C.

   a) Incorreta: pois aprovações sem critérios e ausência de logs inviabilizam a rastreabilidade.
   b) Incorreta: pois postergar verificações aumenta a exposição às falhas.
   c) Correta: essa alternativa define critérios de passagem e evidências publicadas, sustentando decisão e repetibilidade.
   d) Incorreta: pois a execução única concentra risco.
   e) Incorreta: pois isola o *build* e elimina integração/controle.

2. Alternativa B.

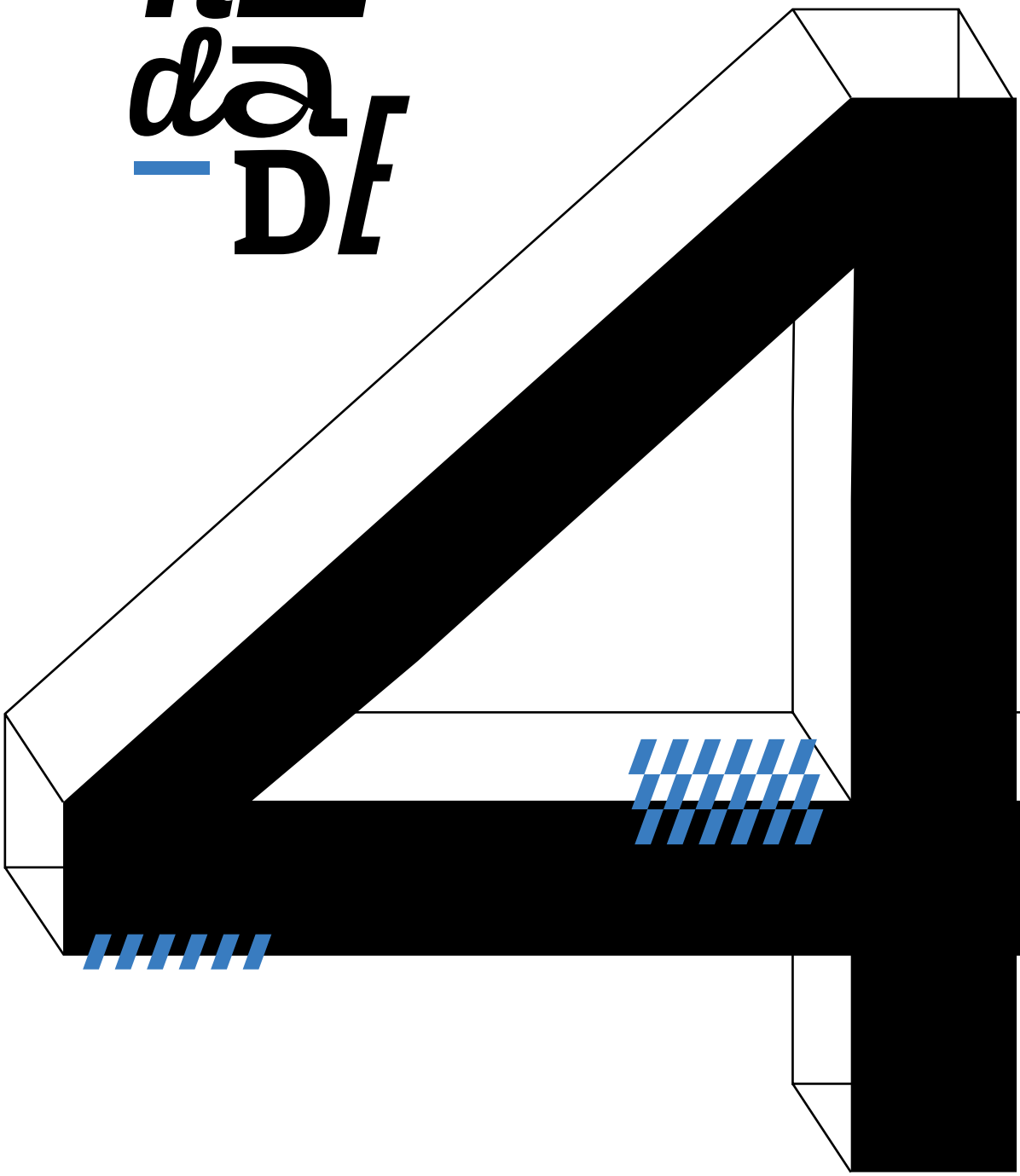
   a) Incorreta: pois logs não estruturados e baixa frequência comprometem a análise.
   b) Correta: essa alternativa integra métricas com logs estruturados e *trace*, permitindo diagnóstico, bem como ajuste contínuo.
   c) Incorreta: pois o alerta reativo reduz o tempo de resposta.
   d) Incorreta: pois desconecta medição do processo técnico.
   e) Incorreta: pois abdica de evidências automatizadas.

3. Alternativa C.

   a) Incorreta: pois multiplica variações e riscos.
   b) Incorreta: pois incentiva arranjos não reprodutíveis.
   c) Correta: essa alternativa materializa templates + IaC + monitoramento como serviço interno padronizado.
   d) Incorreta: pois ignora isolamento/padronização.
   e) Incorreta: pois carece de políticas e integração.

Unicesumar

unidade





TEMA DE APRENDIZAGEM 6

FÁBRICAS DE SOFTWARE IN-HOUSE E OUTSOURCING

MINHAS METAS

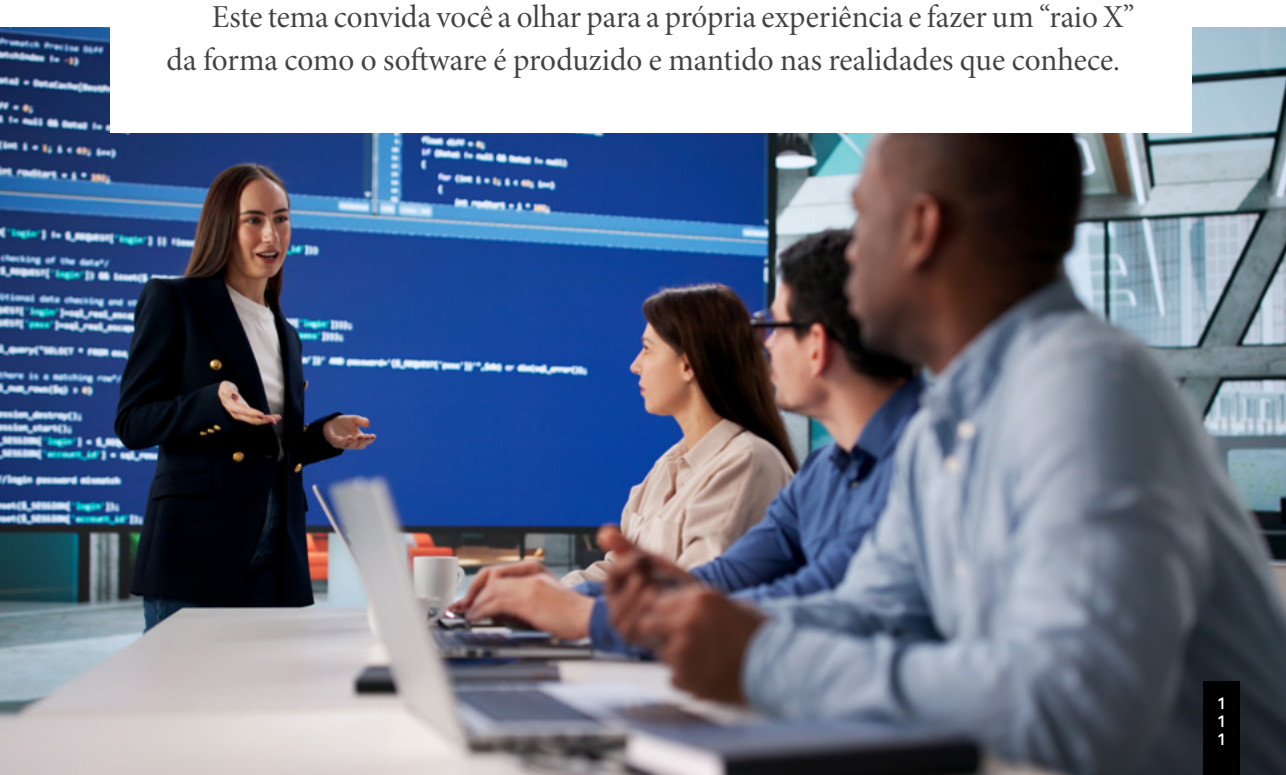
- Comparar modelos e diferenciar fábrica *in-house*, *outsourcing* e híbrida, com prós e contras em custo, controle, expertise e escala.
- Avaliar economicamente e estimar TCO/ROI e custos ocultos para justificar a estratégia.
- Definir governança e contratos e estabelecer SLAs/KPIs, papéis (Raci), SOW/OLA e critérios de aceitação.
- Planejar arranjos híbridos e desenhar *squads* mistos, *handoffs* claros e retenção de conhecimento.
- Gerir riscos e compliance e mitigar *lock-in*, rotatividade e qualidade com segurança e LGPD.
- Integrar DevOps/qualidade e padronizar CI/CD, *branches*, DoR/DoD, testes e evidências (logs, relatórios, SBOM).
- Conduzir transições e criar plano de *insourcing/outsourcing* com marcos, métricas e melhoria contínua.

INICIE SUA JORNADA

Quando você ouve o termo “fábrica de software”, o que vem à mente? Talvez a imagem de um time interno cuidando de tudo, do levantamento de requisitos ao *deploy*, ou, ao contrário, uma empresa terceirizada que “entrega o sistema pronto”, enquanto a sua organização apenas acompanha prazos e faturas. Em muitos contextos, essa decisão não é tão clara assim: a pressão por custo menor, entregas mais rápidas e qualidade consistente faz a gestão oscilar entre montar uma fábrica *in-house*, contratar um parceiro em *outsourcing* ou combinar ambos em um modelo híbrido. Cada escolha traz impactos diretos no seu dia a dia de trabalho.

Ao longo da sua trajetória, é bem provável que você já tenha vivido alguns desses cenários, ainda que não os chamasse por esses nomes. Talvez já tenha sentido na pele o que é depender de um fornecedor para qualquer ajuste simples, ou, ao contrário, ver a equipe interna sobrecarregada, tentando dar conta de tudo sem escala nem especialização suficiente. Também é comum perceber conflitos silenciosos: quem decide prioridades? Como garantir que um time terceirizado cumpra o que foi combinado em termos de qualidade, prazos, segurança e conformidade com a Lei Geral de Proteção de Dados (LGPD)? Essas situações mostram que o tema não é apenas técnico: afeta governança, carreira, relacionamento entre áreas e até com a reputação da organização.

Este tema convida você a olhar para a própria experiência e fazer um “raio X” da forma como o software é produzido e mantido nas realidades que conhece.



VOCÊ SABE RESPONDER?

Há *squads* internos, parceiros externos ou ambos atuando juntos? Existem contratos claros, SLAs e métricas de qualidade e valor ou tudo se resolve no improviso? Os pipelines de desenvolvimento, teste e *deploy* são padronizados ou cada fornecedor trabalha à sua maneira?

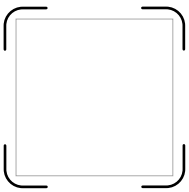
Ao se perguntar tudo isso, você começa a experimentar um olhar mais analítico sobre modelos *in-house*, *outsourcing* e híbridos: um olhar que considera custo total, riscos de dependência, rotatividade, segurança, fluxo de conhecimento e capacidade de inovar.

Mais do que decorar definições, o objetivo é que você reflita sobre como essas decisões estratégicas impactam o seu desenvolvimento profissional. Entender por que uma organização opta por internalizar ou terceirizar sua fábrica de software – e como transitar entre esses modelos com plano, métricas e melhoria contínua – amplia sua visão de futuro: você deixa de ser apenas "alguém que codifica tarefas" e passa a enxergar o papel da engenharia de software na estratégia do negócio. Ao final do percurso, a ideia é estar preparado para comparar modelos, participar de análises econômicas, apoiar a definição de boas práticas de governança, colaborar em arranjos híbridos mais saudáveis e contribuir para transições mais maduras entre *in-house* e *outsourcing*, seja qual for o contexto em que atuar.

Diante desse cenário, como o modelo adotado na sua realidade influencia sua atuação profissional hoje e quais mudanças seriam necessárias para torná-lo mais eficiente e sustentável?

PLAY NO CONHECIMENTO

Em poucos minutos, o podcast explica as diferenças entre *in-house*, *outsourcing* e modelos híbridos, e como impactam TCO/ROI, controle, expertise, escala, qualidade e conformidade. Aborda SLAs/KPIs, Raci, SOW/OLA, LGPD e padronização DevOps/CI/CD. Ouça!



VAMOS RECORDAR?

Antes de decidir se a fábrica de software será *in-house*, terceirizada ou híbrida, vale retomar algumas bases já vistas na disciplina: o ciclo de vida do desenvolvimento de software (SDLC), com suas etapas de requisitos, viabilidade, projeto, desenvolvimento, testes, implantação e manutenção, e o papel das plataformas internas, DevOps e automação de pipelines (CI – *Continuous Integration*/CD – *Continuous Delivery* ou *Continuous Deployment*, padronização de ambientes, monitoramento e observabilidade) na sustentação da operação contínua. Em cada fase, a organização pode internalizar atividades ou delegá-las a parceiros, impactando custo, prazo, qualidade e risco. Esses conceitos funcionam como pano de fundo para compreender, neste tema, as diferentes escolhas entre fábrica interna, *outsourcing* e modelos híbridos.

DESENVOLVA SEU POTENCIAL

ESCOLHAS DE MODELO: IN-HOUSE, OUTSOURCING E HÍBRIDO

Quando uma organização decide "onde" e "com quem" fará seu desenvolvimento de software, ela está, na prática, escolhendo um modelo de fábrica. Em linhas gerais, uma fábrica *in-house* mantém times internos responsáveis por todo o ciclo de vida: requisitos, desenvolvimento, testes, implantação e suporte. O ganho está no alinhamento estratégico com o negócio, no controle direto sobre padrões técnicos e na retenção de conhecimento crítico sobre processos e clientes (Fernandes; Teixeira, 2011; Sommerville, 2011). Em contrapartida, exige investimento contínuo em pessoas, processos, infraestrutura e liderança técnica (Pressman; Maxim, 2016).

Já uma fábrica em *outsourcing* transfere parte relevante da execução para parceiros externos, buscando redução de custos, flexibilidade de capacidade, acesso rápido a expertise e cobertura de horários ou regiões (Pressman; Maxim, 2016). O modelo híbrido combina um núcleo interno forte (arquitetura, produto,

governança) com fornecedores que ampliam escopo e volume, permitindo "testar" novas iniciativas sem inflar demais a estrutura interna (Fernandes; Teixeira, 2011). O desafio é equilibrar custo, controle, especialização e escala, evitando tanto o isolamento da equipe interna quanto a dependência excessiva de terceiros.

Antes de discutir contratos e governança, é útil enxergar, de forma compacta, como cada modelo de fábrica se comporta nos critérios que mais influenciam a decisão.

A Figura 1 sintetiza *in-house*, *outsourcing* e híbrido em oito dimensões: custo, controle, expertise, escala, *Total Cost of Ownership* (TCO – Custo Total de Propriedade) e *Return on Investment* (ROI – Retorno sobre o Investimento), risco, DevOps e LGPD, usando uma escala simples para leitura rápida. Essa visão ajuda a alinhar expectativas e a direcionar a análise econômica e de riscos que vem na sequência do tema.

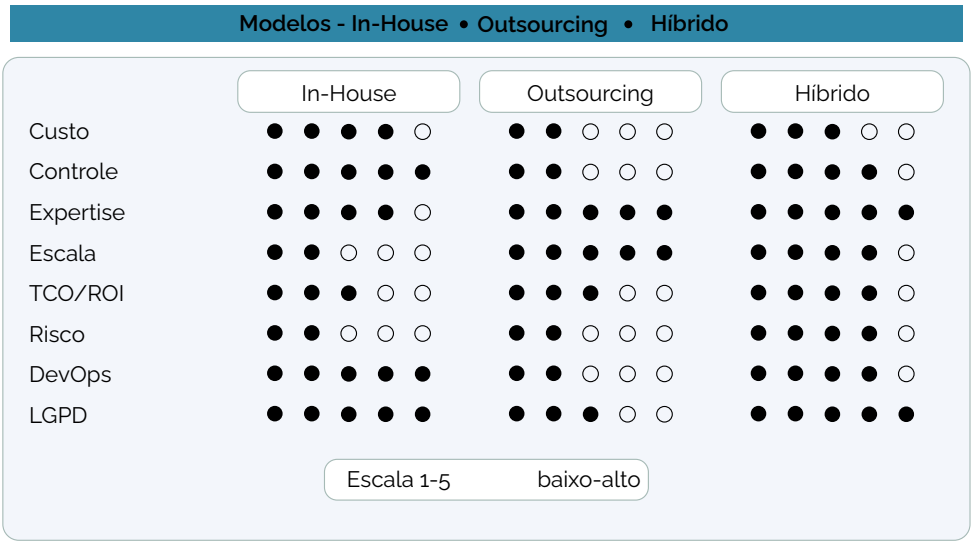


Figura 1 – Comparação de modelos de fábrica de software: in-house, outsourcing e híbrido
Fonte: gerada por ChatGPT em 10 dez. 2025.

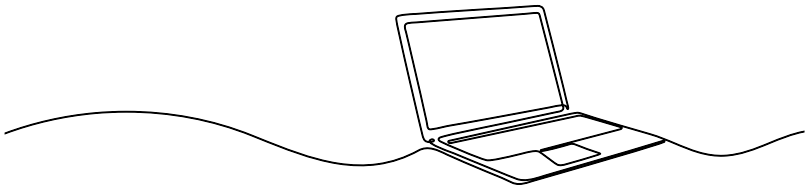
Descrição da imagem: matriz com três colunas e oito critérios: Custo, Controle, Expertise, Escala, TCO e ROI, Risco, DevOps e Qualidade, LGPD e Compliance. Em cada célula, há uma escala de um a cinco pontos indicada por círculos preenchidos, em que um ponto representa nível baixo e cinco pontos representam nível alto. Em resumo, *in-house* tende a maior custo e maior controle, com menor elasticidade; *outsourcing* privilegia escala e acesso a especialistas, com menor controle; o modelo híbrido busca equilíbrio com governança. Fim da descrição.

Olhar econômico: TCO, ROI e custos ocultos

Comparar modelos apenas pelo valor da hora do desenvolvedor é perigoso e costuma levar a decisões míopes. A literatura de engenharia de software lembra que decisões estruturais precisam considerar **Custo Total de Propriedade (TCO)** e **Retorno sobre Investimento (ROI)**, incluindo manutenção, retrabalho, falhas em produção, tempo de resposta ao negócio, esforço de coordenação e risco de interrupções (Pressman; Maxim, 2016; Sommerville, 2011).

Em um contrato terceirizado, por exemplo, podem surgir custos ocultos em alinhamentos constantes, *handoffs* mal planejados, perda de conhecimento quando o fornecedor troca a equipe, barreiras contratuais a experimentos e integrações, além de despesas com auditorias e adequação à LGPD (Brasil, 2018).

Em uma operação *in-house*, os custos concentram-se em atração e retenção de talentos, atualização tecnológica, ferramentas, capacitação contínua e disciplina de processos (Pressman; Maxim, 2016). Ao avaliar modelos, o exercício é sempre perguntar: "quanto custa, ao longo de anos, manter esse arranjo saudável?", e não apenas "quanto custa agora esse projeto?"



APROFUNDANDO

Uma forma prática de comparar opções é montar uma tabela com três colunas – *in-house*, *outsourcing*, híbrido – e linhas para itens como: custo de contratação, custo de coordenação, velocidade de resposta, retenção de conhecimento, adequação regulatória, capacidade de inovação e risco de *lock-in* (Fernandes; Teixeira, 2011). Em cada célula, você registra evidências, percepções e riscos observados ou esperados. Se possível, traga dados reais (por exemplo, número de incidentes, tempo de entrega e rotatividade de equipe). Esse quadro não "fecha a conta" sozinho, mas ajuda a tornar explícitos os *trade-offs* que ficam, muitas vezes, apenas no discurso ou na intuição de poucas pessoas.



Governança e contratos: alinhando expectativas

Independentemente do modelo escolhido, a fábrica de software só funciona bem com governança clara que, por sua vez, passa por acordos de nível de serviço (SLAs) e indicadores de desempenho (KPIs) ligando prazos, qualidade e custo ao que é realmente entregue, em vez de confiar apenas em boa vontade ou esforço individual (Fernandes; Teixeira, 2011; Pressman; Maxim, 2016).

No dia a dia, ferramentas como a matriz Raci ajudam a esclarecer quem é responsável por executar, quem aprova, quem deve ser consultado e quem apenas é informado em cada etapa do fluxo, da priorização de demandas ao suporte em produção (Sommerville, 2011).

Os documentos de escopo e trabalho (SOW – *Statement of Work*, Declaração de Trabalho) e os acordos operacionais internos (OLA – *Operational Level Agreement*, Acordo de Nível Operacional) detalham fronteiras entre times internos e parceiros: o que está dentro ou fora do contrato, quais artefatos devem ser produzidos, como serão tratadas mudanças de requisito e quais são as sanções em caso de descumprimento (Fernandes; Teixeira, 2011).

Por fim, critérios de aceitação objetivos (funcionais, não funcionais, de segurança e conformidade) evitam disputas subjetivas na hora de validar entregas. Para você, futuro engenheiro de software, entender essa linguagem de governança é essencial para participar das decisões, defender condições realistas de trabalho e fazer com que requisitos técnicos críticos apareçam nos contratos, e não apenas em conversas informais (Pressman; Maxim, 2016).

Para que o modelo escolhido funcione no dia a dia, é preciso transformar decisões estratégicas em acordos operacionais verificáveis. A Figura 2 mostra, de ponta a ponta, como SLAs/KPIs, papéis Raci e critérios de aceitação se distribuem entre *in-house*, fornecedores e PMO/Cliente, com *gates* de qualidade e segurança, que geram evidências para o aceite.

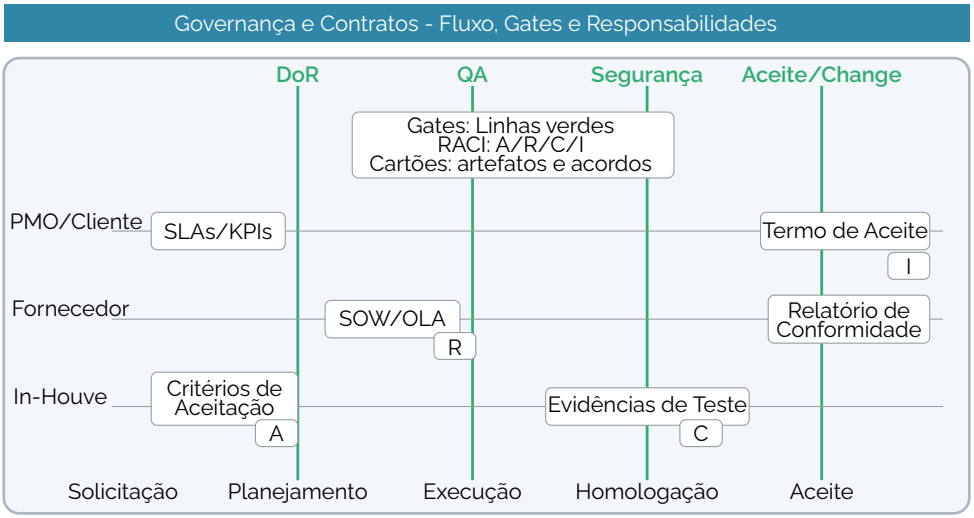


Figura 2 – Governança e contratos: SLAs/KPIs, Raci e critérios de aceitação ao longo do fluxo
Fonte: gerada por ChatGPT em 10 dez. 2025.

Descrição da imagem: três trilhas horizontais (*in-house*, fornecedor e PMO/cliente) dispostas sobre uma linha do tempo com cinco marcos: Solicitação, Planejamento, Execução, Homologação e Aceite. Linhas verticais verdes indicam *gates* de controle (*Definition of Ready*, QA, Segurança e Aceite/ *Change*). Em cada trilha, cartões representam artefatos e acordos: "SLAs/KPIs", "SOW/OLA", "Critérios de Aceitação", "Evidências de Teste", "Relatório de Conformidade" e "Termo de Aceite". Pequenos chips marcam papéis Raci (*Responsible, Accountable, Consulted, Informed*) em momentos-chave. A leitura evidencia como contratos, qualidade e responsabilidades convergem para um aceite com evidências. Fim da descrição.

ARRANJOS HÍBRIDOS, DEVOPS E TRANSIÇÕES ENTRE MODELOS

Na prática, poucas organizações vivem em um extremo puro. O mais comum é encontrar **arranjos híbridos**, nos quais existem *squads* internos, fornecedores especializados, sustentação terceirizada, parceiros de nuvem e uma variedade de contratos e fusos horários (Fernandes; Teixeira, 2011). Nesse contexto, um ponto-chave é definir **o que permanece como competência central** e o que pode ser realizado por terceiros sem comprometer o futuro da organização. Em muitos casos, faz sentido manter *in-house* arquitetura, decisões de produto, segurança crítica, gestão de dados sensíveis e plataforma interna, usando *outsourcing* para ampliar capacidade de desenvolvimento, testes ou operação em serviços específicos (Pressman; Maxim, 2016).

Se a pergunta estratégica é "o que não podemos perder?", a resposta costuma apontar para três pilares: conhecimento de negócio, domínio da arquitetura e controle sobre requisitos regulatórios (Sommerville, 2011). A Figura 3 traduz essas prioridades em prática: um quadro de programa/portfólio que organiza *sprints*, releases e dependências entre o *squad* interno e os fornecedores, destacando onde o núcleo permanece dentro de casa e onde ocorrem as entregas terceirizadas. Assim, o desenho do modelo híbrido aparece de forma operacional – com *handoffs* claros, sincronização por cadências e pontos de controle visíveis – alinhando times e contratos ao que é crítico reter.

Conhecimento de negócio, domínio da arquitetura e controle sobre requisitos regulatórios

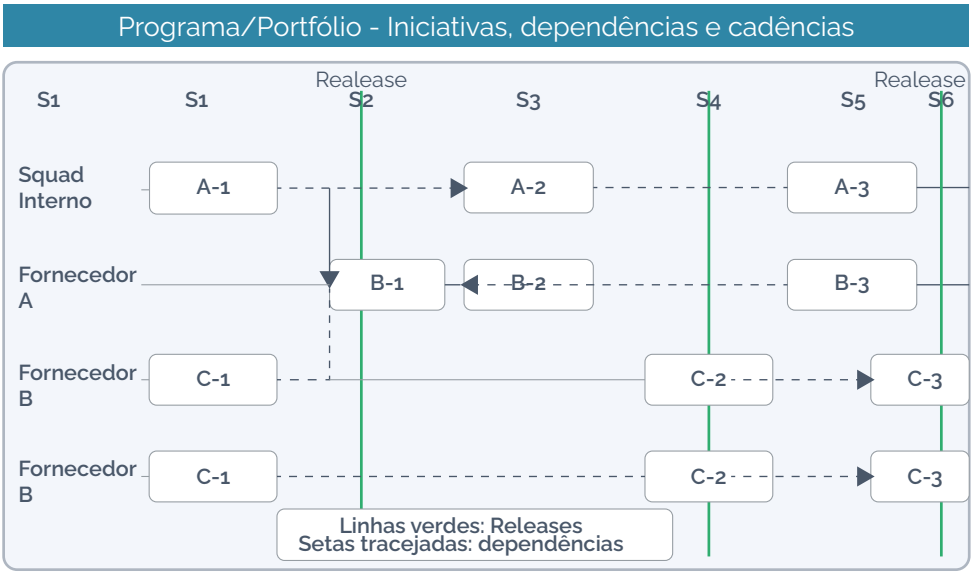


Figura 3 – Programa/Portfólio: iniciativas, dependências e cadências
Fonte: gerada por ChatGPT em 10 dez. 2025.

Descrição da imagem: três trilhas horizontais (*Squad* Interno, Fornecedor A e Fornecedor B) dispostas ao longo de seis *sprints* (S1 a S6). Em cada trilha, cartões identificados por A-1, A-2, A-3; B-1, B-2, B-3; C-1, C-2, C-3 representam entregas previstas. Linhas verticais verdes marcam janelas de release nas *sprints* S2, S4 e S6. Setas tracejadas indicam dependências dentro e entre as trilhas (exemplos: A-1 para B-1; A-2 para B-2; B-2 para C-3; C-1 para A-2). A leitura evidencia sincronização por *sprints*, coordenação entre equipes e pontos formais de liberação. Fim da descrição.

Squads mistos, handoffs e retenção de conhecimento

Arranjos híbridos saudáveis costumam trabalhar com **squads mistos**, combinando pessoas da casa e do fornecedor em torno de objetivos comuns (Bass; Weber; Zhu, 2015). Esse cenário permite alinhar visão de produto, cultura e padrões técnicos, ao mesmo tempo que se aproveita a escala e a especialização do parceiro.

Para reduzir atritos, é essencial desenhar **handoffs claros**: como uma demanda sai do backlog de produto e entra no fluxo do fornecedor, quais artefatos (histórias, critérios de aceitação, cenários de teste, riscos conhecidos, restrições de arquitetura) devem acompanhar a entrega e como os aprendizados voltam para o núcleo interno (Pressman; Maxim, 2016).

A retenção de conhecimento pede práticas como documentação viva (arquitetura, decisões, riscos), revisão de código cruzada entre times, sessões de compartilhamento de conhecimento, comunidades técnicas compartilhadas e rotação planejada de pessoas entre iniciativas (Sommerville, 2011). Assim, a fábrica evita ficar refém de indivíduos ou de uma única empresa, preservando sua capacidade de evoluir mesmo quando contratos mudam ou equipes são reconfiguradas.

Riscos, LGPD e qualidade contínua com DevOps

Quando o desenvolvimento envolve terceiros e múltiplos ambientes, cresce a responsabilidade de **gerir riscos e compliance**. A LGPD define obrigações para controladores e operadores de dados pessoais, exigindo que contratos explicitem papéis, medidas de segurança, fluxos de resposta a incidentes e regras para subcontratação (Brasil, 2018). Na prática, isso significa integrar requisitos de segurança e privacidade aos próprios **pipelines de DevOps**: análise estática, testes de segurança, políticas de acesso, rastreabilidade de mudanças, geração de SBOM (lista de componentes de software) e registros de auditoria (Bass; Weber; Zhu, 2015; Humble; Farley, 2014).

Padronizar CI/CD, convenções de *branches*, critérios de DoR/DoD, estratégias de teste e evidências mínimas (logs, relatórios, artefatos assinados) entre times internos e fornecedores reduz a variabilidade, torna incidentes mais visíveis e facilita a demonstração de conformidade em auditorias internas e externas (Pressman; Maxim, 2016).

Para você, tudo isso se traduz em escrever código e configurar pipelines já pensando em **como provar**, amanhã, que aquele software foi desenvolvido e implantado sob boas práticas de segurança e qualidade.

PENSANDO JUNTOS

Em uma entrega típica, mapeia-se quem acessa dados pessoais ao longo do fluxo, onde o código-fonte e os artefatos de *build* ficam armazenados e quais perfis têm acesso. Também se identifica em que etapas do pipeline são checadas vulnerabilidades, dependências inseguras e políticas de acesso, além dos registros que permitem reconstituir uma falha (logs, trilhas de auditoria, relatórios de teste e incidentes). Ao ligar tudo isso a SLAs, KPIs e cláusulas contratuais, o DevOps deixa de ser só automação, tornando-se ferramenta de governança e compliance em fábricas com múltiplos atores, contratos e ambientes, apoiando a conformidade por meio da LGPD e das boas práticas de engenharia de software (Bass; Weber; Zhu, 2015; Brasil, 2018; Humble; Farley, 2014).

Transições planejadas entre in-house, outsourcing e híbrida

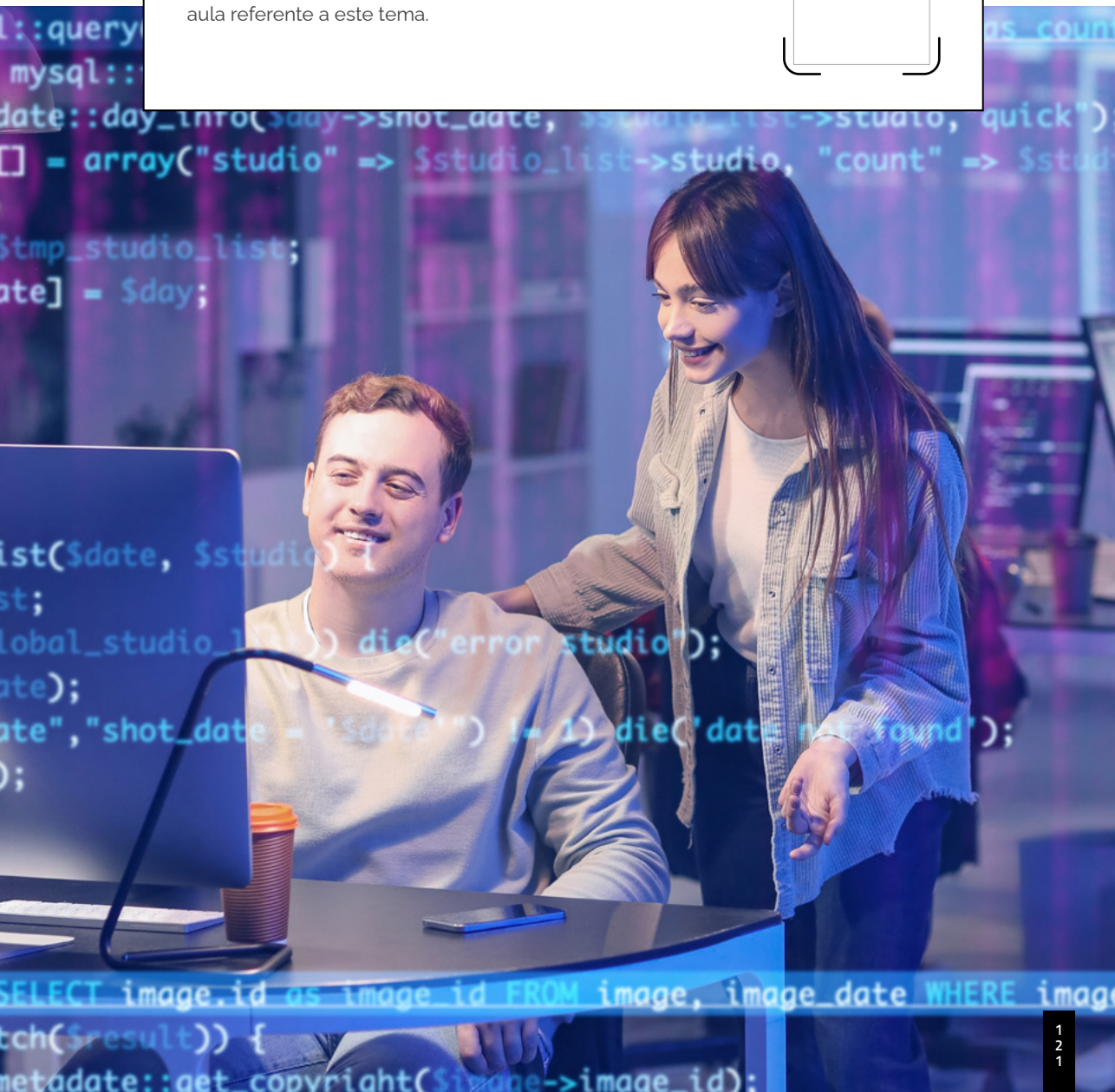
Por fim, mudar de modelo – internalizar algo que estava terceirizado, terceirizar uma parte da operação ou redesenhar um arranjo híbrido – não deveria ser uma reação impulsiva, e sim uma transição planejada (Pressman; Maxim, 2016). Na prática, envolve mapear o portfólio de sistemas e serviços, identificar dependências críticas, definir ondas de transição com critérios de pronto, por exemplo, nível mínimo de automação de testes, cobertura de monitoramento, documentação de arquitetura e de dados.

É necessário estabelecer métricas para acompanhar o impacto – tempo de entrega, custo por funcionalidade, defeitos em produção, satisfação do usuário, incidentes de segurança – além de revisar contratos e processos à luz desses resultados (Fernandes; Teixeira, 2011; Sommerville, 2011). Em vez de ver *in-house, outsourcing* e híbrido como rótulos fixos, a organização aprende a tratá-los como **estratégias em evolução**, ajustadas continuamente ao que o negócio precisa e ao que a fábrica é capaz de entregar com qualidade (Bass; Weber; Zhu, 2015).

Sua atuação profissional ganha força quando você consegue ler esses movimentos, contribuir com dados e ajudar a manter a continuidade técnica e a saúde dos sistemas em meio às mudanças organizacionais.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.



NOVOS DESAFIOS

Ao olhar para o futuro da sua carreira, é bem possível que você atue em equipes que misturam desenvolvedores internos, consultorias terceirizadas, fábricas de software e até times distribuídos em outros estados ou países. A escolha entre manter tudo *in-house*, contratar via *outsourcing* ou adotar um modelo híbrido não é apenas uma decisão "de diretoria": ela impacta diretamente o seu dia a dia, seus ritos de trabalho, a forma como você registra evidências, protege dados (LGPD) e negocia prazos e prioridades.

Pense no **ambiente profissional** onde você imagina atuar: um software *house*, o time de TI de uma indústria, uma *fintech*, uma *healthtech*, um órgão público, uma consultoria? Em cada um desses cenários, o equilíbrio entre fábrica interna e fornecedores externos será diferente. Em alguns casos, o time interno foca no *core* do negócio e terceiriza rotinas de sustentação; em outros, a empresa mantém competências críticas dentro de casa e usa parceiros para acelerar picos de demanda. Entender essas combinações ajudarão vocês a lerem melhor editais, contratos, SLAs e oportunidades de carreira.

Enquanto profissional, você será cada vez mais chamado a participar de decisões como:

> "Vale a pena migrar esse sistema para uma fábrica externa?", "Como evitar *lock-in* em um único fornecedor?", "Quais métricas e evidências precisamos para cobrar qualidade da equipe terceirizada?" e "como garantir que o conhecimento não fique preso apenas no fornecedor?".

O desafio é conectar a teoria deste tema – modelos *in-house*, o*utsourcing* e híbridos, TCO/ROI, governança, DevOps e compliance – com situações reais: processos seletivos, entrevistas técnicas, reuniões de *kickoff* com fornecedores e revisões de contrato.

Ao final do tema, o convite é que você enxergue esses modelos não só como "estrutura organizacional", mas como parte do seu projeto de vida profissional: que tipo de ambiente quer construir ou escolher para trabalhar, quais competências técnicas e de gestão precisa fortalecer e como se posicionar para ser protagonista na definição e na melhoria contínua das fábricas de software com as quais interagirá.

VAMOS PRATICAR

1. Em operações com times internos e fornecedores, a clareza sobre tipos de serviço, insumos mínimos e saídas verificáveis reduz ambiguidades e melhora a leitura do fluxo. Acordos de nível de serviço e indicadores tornam expectativas explícitas. O arranjo deve ser proporcional a riscos e objetivos (Fernandes; Teixeira, 2011; Pressman; Maxim, 2016).

   Qual alternativa melhor caracteriza a padronização de serviços em fábricas com times internos e fornecedores?

   a) Aceites informais por e-mail.
   b) Controle por esforço individual, sem métricas.
   c) Catálogo por tipo de serviço, critérios de prontidão/conclusão e SLAs/KPIs visíveis.
   d) Reuniões de status sem evidências.
   e) Adoção de ferramenta única como regra.

2. Em portfólios com soluções semelhantes, práticas de reuso distinguem elementos comuns e organizam variações. Versionamento e comunicação estruturada de mudanças reduzem rupturas. O objetivo é manter adoção sustentável sem perda de autonomia dos consumidores (Fernandes; Teixeira, 2011; Pressman; Maxim, 2016).

   Qual alternativa favorece o reúso e a compatibilidade em ecossistemas multiequipes?

   a) Duplicar componentes para acelerar.
   b) Contratos executáveis, componentes versionados e *changelog* por versão.
   c) Política "mudar quando necessário", sem registros.
   d) Acoplamento por banco compartilhado.
   e) Bloqueio de atualizações por tempo indeterminado.

3. Projetos com prazos críticos tendem a combinar marcos, entregas incrementais e verificações padronizadas para reduzir incertezas. Integrar à esteira comum facilita a repetibilidade, as evidências e a reversão segura. O design pondera custos de controle e impacto de falha (Bass; Weber; Zhu, 2015; Humble; Farley, 2014).

   Qual alternativa apresenta a prática que mais alinha previsibilidade e risco em projetos regulados?

   a) Big bang ao final.
   b) Incrementos integrados à CI/CD com critérios de passagem e evidências publicadas.
   c) Aceite informal do cliente.
   d) *Deploy* direto em produção.
   e) Aprovação por esforço estimado.

REFERÊNCIAS

BASS, L.; WEBER, I.; ZHU, L. **DevOps**: a software architect's perspective. Boston: Addison-Wesley Professional, 2015.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014. Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 23 jan. 2026.

FERNANDES, A. A.; TEIXEIRA, D. de S. **Fábrica de software**: implantação e gestão de operações. São Paulo: Atlas, 2011.

HUMBLE, J.; FARLEY, D. **Entrega contínua**: como entregar software de forma rápida e confiável. Porto Alegre: Bookman, 2014.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011.

CONFIRA SUAS RESPOSTAS

1. Alternativa C.

   a) Incorreta: pois aceite informal não substitui critérios.
   b) Incorreta: pois foco no indivíduo não governa o sistema.
   c) Correta: essa alternativa define catálogo, mais critérios objetivos e mais SLAs/KPIs.
   d) Incorreta: pois status sem evidência não garante previsibilidade.
   e) Incorreta: pois ferramenta é diferente de governança.

2. Alternativa B.

   a) Incorreta: pois duplica o débito.
   b) Correta: pois contratos, versões e *changelog* promovem compatibilidade.
   c) Incorreta: pois sem trilha, cresce a ruptura.
   d) Incorreta: pois é um acoplamento frágil.
   e) Incorreta: pois congela a evolução e cria riscos.

3. Alternativa B.

   a) Incorreta: pois concentra risco.
   b) Correta: incrementos, pipeline e critérios/evidências.
   c) Incorreta: pois aceite é subjetivo.
   d) Incorreta: pois eleva exposição.
   e) Incorreta: pois esforço é diferente de qualidade/risco.



TEMA DE APRENDIZAGEM 7

GOVERNANÇA, QUALIDADE E PADRONIZAÇÃO

MINHAS METAS

- Compreender *frameworks* de governança (ISO/IEC 12207, CMMI, MPS.BR, Itil, Cobit) e quando aplicá-los.
- Estruturar políticas, papéis (Raci), controles e fluxos decisórios para fábricas de software.
- Definir métricas de qualidade de processo e produto (eficiência, defeitos, cobertura, DRE) e metas.
- Implementar auditorias, rastreabilidade *end-to-end* e gestão de configuração como evidências.
- Gerir riscos e conformidade (LGPD, segurança, contratos) com plano de tratamento e monitoramento.
- Padronizar artefatos e práticas para escalar com previsibilidade e reduzir variação indevida.
- Preparar a organização para certificações e exigências de grandes clientes/órgãos públicos, por meio da medição do ROI.

INICIE SUA JORNADA

Em muitas organizações, a conversa sobre fábricas de software começa em termos sedutores – agilidade, inovação, times multidisciplinares – e termina em palavras bem menos glamourosas: não conformidade, reprovação em auditoria, atraso em edital e perda de contrato. Quando sistemas sustentam operações críticas, dados pessoais de milhões de pessoas ou contratos com grandes clientes e órgãos públicos, não basta "entregar funcionando": é preciso provar como foi feito, por quem, com quais controles e evidências. Nesse cenário, governança, qualidade e padronização deixam de ser detalhe burocrático e passam a ser o alicerce de qualquer fábrica de software que queira escalar sem perder o controle.

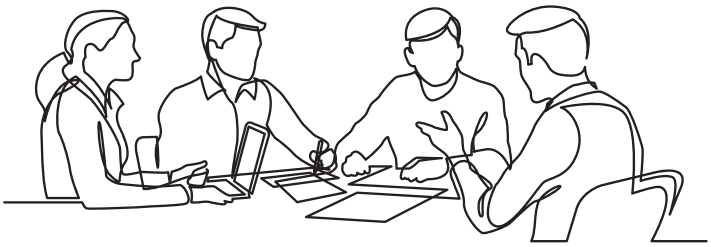
Ao longo da sua trajetória (ou daquela em que deseja atuar), é provável que você já tenha vivenciado o choque entre expectativas técnicas e exigências de negócio. De um lado, times falando em *sprints*, Git, pipelines e automação; de outro, clientes e gestores preocupados com cláusulas contratuais, prazos legais, relatórios e riscos. Auditores pedem rastreabilidade, gestão de risco e aderência à LGPD; a diretoria cobra previsibilidade, indicadores claros e redução de incidentes.

Frameworks como ISO/IEC 12207, CMMI, MPS.BR, Itil e Cobit surgem justamente para organizar essa conversa, oferecendo um vocabulário comum para definir processos, papéis, métricas e níveis de maturidade. Mais do que decorar siglas, o desafio é compreender como essas referências ajudam a transformar boas intenções em políticas, fluxos decisórios e práticas consistentes ao longo do tempo.

Este tema convida você a enxergar a fábrica de software como um sistema vivo de decisões: qual é o caminho de um requisito até o *deploy* em produção? Em que momento são definidos SLAs, KPIs e critérios de aceite? Como defeitos, incidentes e riscos são registrados, priorizados e transformados em planos de ação, em vez de virarem apenas "caça às bruxas" quando algo dá errado? Ao refletir sobre essas perguntas, pense na sua realidade: existem matrizes Raci claras, rastreabilidade ponta a ponta, padrões de teste e de documentação? Os times entendem por que certas evidências são exigidas ou apenas "preenchem o que mandaram"?

À medida que você avançar no conteúdo, procure relacionar cada conceito com o seu desenvolvimento tanto profissional quanto acadêmico. Profissionais capazes de transitar entre código, processos, contratos e indicadores tornam-se pontos de conexão valiosos: são eles que traduzem exigências de governança para o dia a dia dos times e, ao mesmo tempo, levam dos times insumos concretos para melhorar políticas, padrões e controles. Entender como a padronização bem aplicada reduz variação indevida, de que maneira métricas sustentam decisões e como evidências constroem confiança é um passo importante para deixar de enxergar normas e modelos apenas tais qual "checklist de certificação" e utilizá-los como aliados na construção de fábricas de software mais previsíveis, seguras e respeitadas no mercado.

Na sua realidade profissional, os processos e evidências da fábrica de software apoiam a tomada de decisão ou são utilizados apenas para cumprir exigências formais?



PLAY NO CONHECIMENTO

Em poucos minutos, o podcast mostra como governança, qualidade e padronização convertem esforço em entregas confiáveis: métricas-chave, Raci claro, rastreabilidade ponta a ponta, gestão de risco/LGPD e CI/CD com evidências. Dê o play e acompanhe um episódio direto e sem jargões.

VAMOS RECORDAR?

Antes de avançarmos para governança, qualidade e padronização em fábricas de software, vale retomar algumas bases que você já encontrou em temas anteriores. Lembre-se do ciclo de vida de desenvolvimento de software (SDLC), com suas etapas de levantamento de requisitos, projeto, construção, testes, implantação e manutenção e de como as abordagens ágeis e DevOps conectam essas fases em fluxos contínuos, com entregas incrementais e automação de *build*, testes e *deploy*. Relembre também a ideia de fábrica de software como um arranjo organizado de pessoas, processos e ferramentas, voltado a produzir e manter soluções em escala, seja em modelos internos, seja em modelos terceirizados ou híbridos. Esses elementos – ciclo de vida, métodos de trabalho, times e pipelines – serão, agora, revisitados sob uma nova lente: como criar regras, métricas, responsabilidades e padrões para que tudo isso funcione de forma integrada, previsível e auditável.

DESENVOLVA SEU POTENCIAL

GOVERNANÇA EM FÁBRICAS DE SOFTWARE: MAIS DO QUE "SEGUIR PROCESSO"

Ao falar em governança em fábricas de software, é comum imaginar apenas fluxos desenhados em documentos, planilhas de controle e uma longa lista de normas a cumprir. Na prática, porém, governança é a forma como as decisões são tomadas e sustentadas por evidências: quem prioriza demandas, quem aprova exceções de segurança, como conflitos entre negócio e tecnologia são resolvidos e quais critérios definem o que "pode" ou "não pode" ir para produção.

Normas e modelos como a ISO/IEC/IEEE 12207, que estabelece um *framework* de processos para todo o ciclo de vida de software, do conceito ao descarte (ISO, 2017), oferecem uma base para organizar essa conversa, principalmente quando combinadas com modelos de maturidade como o CMMI *Development V2.0*, voltado à melhoria contínua de capacidades e desempenho organizacional (CMMI Institute, 2018).

A Figura 1 sintetiza essa perspectiva integrada: ao centro, o pipeline de desenvolvimento e operação; ao redor, as camadas de governança, qualidade e padronização que dão suporte às decisões, às métricas e às evidências nas fábricas de software.

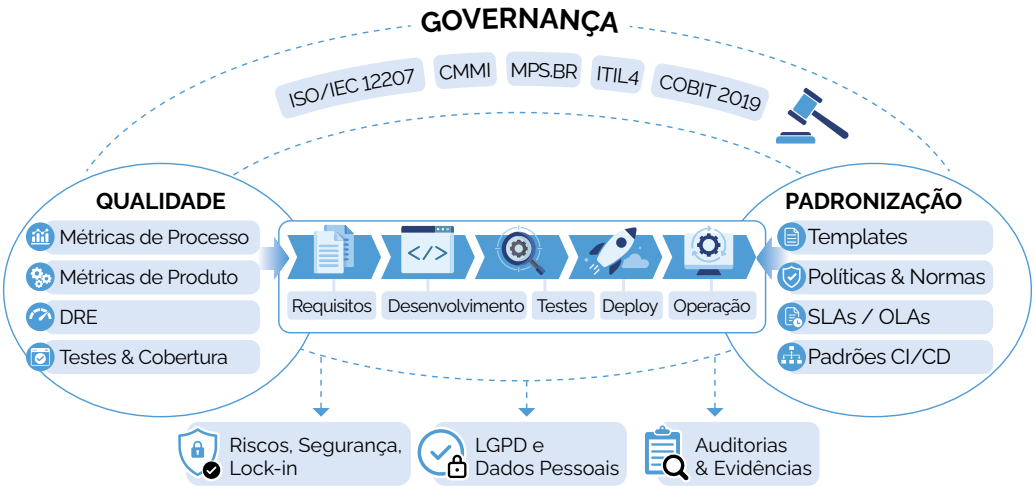


Figura 1 – Visão integrada de governança, qualidade e padronização em fábricas de software
Fonte: gerada por ChatGPT em 2 jan. 2026.

Descrição da imagem: no centro da imagem, há um pipeline horizontal de fábrica de software com cinco estágios: "Requisitos", "Desenvolvimento", "Testes", "*Deploy*" e "Operação", representados por cartões escuros com ícones (respectivamente, documento, código, lupa, foguete e monitor com engrenagem). Acima do pipeline, um arco rotulado "Governança" agrupa cartões com *frameworks* como "ISO/IEC 12207", "CMMI", "MPS.BR", "Itil 4" e "Cobit 2019", sugerindo o arcabouço de processos e decisões. À esquerda, um bloco "Qualidade" aponta para o pipeline e lista os itens "Métricas de Processo", "Métricas de Produto", "DRE" e "Testes & Cobertura". À direita, o bloco "Padronização" aponta para o pipeline com cartões "Templates", "Políticas & Normas", "SLAs/OLAs" e "Padrões CI/CD", indicando padrões transversais. Abaixo do pipeline, três caixas conectadas por setas representam "Riscos, Segurança, Lock-in", "LGPD e Dados Pessoais" e "Auditorias & Evidências", mostrando de que forma o fluxo gera provas para conformidade e gestão de risco. Fim da descrição.

APROFUNDANDO

Ao comparar *frameworks*, note o foco de cada um: ISO/IEC/IEEE 12207 descreve processos de ciclo de vida para software; CMMI organiza práticas em níveis de maturidade para previsibilidade de prazo, custo e qualidade; MPS.BR adapta isso ao Brasil, visando à produtividade e à competitividade em TIC; Itil 4 gerencia serviços e cadeia de valor; Cobit 2019 estrutura governança de TI para alinhar estratégia e riscos. Na prática, combina-se referenciais, priorizando coerência entre políticas, papéis, métricas e decisões, não apenas certificações.

Papéis, Raci e fluxos de decisão

Uma das tarefas centrais da governança é tornar explícito o que, muitas vezes, fica implícito: quem é responsável (R), quem responde em última instância (A), quem deve ser consultado (C) e quem apenas é informado (I) em cada processo relevante. Matrizes Raci ajudam a tornar visíveis essas relações em temas como aprovação de requisitos, mudanças em produção, priorização de dívidas técnicas, aceites de release, exceções de segurança ou de LGPD.

Em fábricas complexas – com *squads* internos, fornecedores, PMO e várias áreas de negócio –, essa clareza é fundamental para evitar zonas cinzentas de responsabilidade, reforçando o que modelos de governança corporativa como o Cobit já destacam: a importância de estruturas, políticas e fluxos de informação bem definidos para que a TI entregue valor de forma consistente (Isaca, 2019).

EXEMPLIFICANDO

Imagine uma falha crítica na produção que expõe dados de clientes. Sem Raci claro, surgem encaminhamentos contraditórios: operação culpa desenvolvimento, este aponta o fornecedor, que espera autorizador do cliente, enquanto segurança intervém sem mandato e jurídico é acionado tarde. Com governança definida, o fluxo está mapeado: quem declara incidente, lidera resposta técnica, autoriza *rollback/feature toggle*, comunica cliente, registra evidências para auditoria e alimenta *backlog* com aprendizados. A lógica vale para priorizações, aceites, exceções: menos improviso torna a fábrica previsível e auditável.

MÉTRICAS DE QUALIDADE DE PROCESSO E PRODUTO

Sem indicadores, a governança vira opinião: cada área defende sua percepção de que "está tudo bem" ou "está tudo ruim". Em fábricas de software, medir é essencial para entender onde o processo ajuda e onde atrapalha. Métricas de **eficiência** – *lead time*, *cycle time*, *throughput* e taxa de retrabalho – indicam se o fluxo de trabalho está fluindo ou travando em certos pontos do pipeline.

Métricas de **qualidade de produto** – número de defeitos em produção, densidade de defeitos por tamanho funcional, taxa de *hot fixes* e complexidade de módulos críticos – mostram se as entregas são estáveis e sustentáveis.

Autores clássicos de engenharia de software, como Pressman e Maxim (2016) e Sommerville (2019), reforçam que a medição é um pilar para controlar e melhorar processos, especialmente em ambientes de grande porte e criticidade.

APROFUNDANDO

Indicadores de qualidade de testes – cobertura, taxa de *builds* quebrados, tempo para corrigir falhas de pipeline – e eficiência de remoção de defeitos (DRE) ligam processo e produto. A DRE compara defeitos achados pré-produção com o total (pré + pós), avaliando revisões, testes e inspeções por etapa (Pressman; Maxim, 2016). Se usuários finais acham a maioria, o pipeline é frágil e a verificação mal distribuída. Metas realistas de DRE, cobertura, redução de incidentes, alinhadas a CMMI ou MPS. BR, guiam investimentos em automação, requisitos, revisão de código e testes exploratórios objetivamente, não por intuição (CMMI Institute, 2018; Softex, 2023).

Do indicador à decisão

Coletar métricas não basta: é preciso transformá-las em decisões. Isso envolve definir cadências de análise (reviews semanais, mensais, trimestrais), preparar painéis que combinem indicadores de diferentes *squads* e fornecedores, criar planos de ação com responsáveis e prazos. Em uma análise de governança madura, o foco não é "procurar culpados", mas identificar padrões:

VOCÊ SABE RESPONDER?

Quais demandas geram mais incidentes? Onde itens param mais? Quais times têm melhor DRE e práticas adaptáveis? Comitês de portfólio/mudanças/arquitetura priorizam melhorias, ajustes contratuais, testes críticos e políticas de qualidade (CMMI Institute, 2018; Pressman; Maxim, 2016).

AUDITORIAS, RASTREABILIDADE, RISCOS E CONFORMIDADE

Em ambientes regulados ou que atendem grandes clientes e órgãos públicos, a capacidade de **mostrar evidências** é tão importante quanto a capacidade de entregar funcionalidade. Auditorias internas e externas verificam se os processos desenhados de fato são seguidos, se os controles são eficazes e se riscos relevantes estão sendo identificados e tratados (Axelos, 2019; Isaca, 2019).

Para tanto, a rastreabilidade ponta a ponta – ligando requisitos, tarefas, *commits*, *builds*, testes, releases, incidentes e decisões de negócio – torna-se um ativo estratégico: sem rastreabilidade, é difícil reconstruir o caminho de uma alteração ou explicar por que determinada decisão foi tomada em um momento específico (Pressman; Maxim, 2016).

Gestão de riscos, LGPD e conformidade

A gestão de riscos conecta governança à conformidade. Em fábricas de software, os riscos não se restringem a "estourar prazos": envolvem *lock-in* de fornecedores, obsolescência de tecnologias, vulnerabilidades de segurança, uso inadequado de dados pessoais, descumprimento de cláusulas contratuais e exposição a sanções legais.

A gestão de riscos conecta governança à conformidade



A Figura 2 sintetiza a gestão integrada de risco, LGPD e evidências em fábricas de software. Iniciamos pela identificação de ameaças e impactos no negócio, estimamos probabilidade e definimos plano de tratamento (mitigar, aceitar, transferir); em paralelo, mapeamos dados pessoais e sensíveis no fluxo, registrando bases legais/finalidade e papéis de controlador e operador; por fim, cada decisão gera evidências auditáveis: logs e trilhas de auditoria, relatórios de testes e conformidade, bem como matriz de riscos atualizada. Com isso, risco e proteção de dados deixam de ser listas separadas e orientam tanto o trabalho diário quanto a prestação de contas.

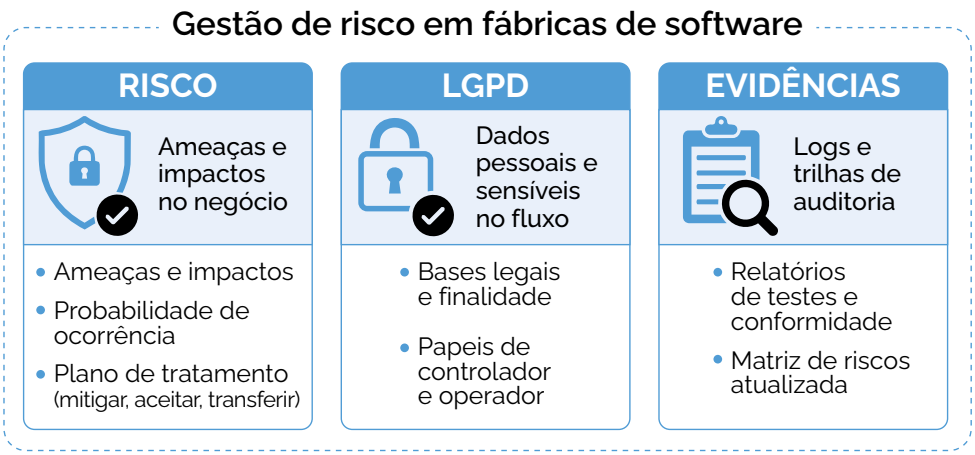


Figura 2 – Gestão de riscos, LGPD e evidências em fábricas de software
Fonte: gerada por ChatGPT em 2 jan. 2026.

Descrição da imagem: painel tripartido com três colunas: "Risco" (ameaças/impactos no negócio, probabilidade de ocorrência e plano de tratamento: mitigar, aceitar, transferir), "LGPD" (dados pessoais e sensíveis no fluxo, bases legais/finalidade e papéis de controlador e operador) e "Evidências" (logs e trilhas de auditoria, relatórios de testes e conformidade, matriz de riscos atualizada). Cada coluna mostra, respectivamente, os seguintes ícones: alerta, cadeado e lista. O arranjo destaca que risco e proteção de dados precisam gerar evidências auditáveis ao longo do ciclo de vida. Fim da descrição.

APROFUNDANDO

Perceba que gestão de riscos, LGPD e conformidade não são temas exclusivos de grandes bancos ou órgãos públicos: qualquer equipe que lide com dados de usuários e serviços em produção já está, na prática, assumindo riscos todos os dias. Quando você usa dados reais em ambiente de teste, liberar um acesso emergencial, adiar uma correção de vulnerabilidade ou aceitar uma exceção de processo, está fazendo escolhas capazes de gerar impacto jurídico, financeiro e de reputação. Aproximar-se da área jurídica, de segurança da informação e de privacidade, entender minimamente as bases legais de tratamento de dados e participar ativamente tanto do mapeamento quanto da priorização de riscos são passos importantes para amadurecer como profissional e contribuir para uma governança mais responsável.

No contexto brasileiro, a Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais – LGPD), estabelece normas para o tratamento de dados pessoais por organizações públicas e privadas, visando proteger direitos fundamentais de liberdade, privacidade e livre desenvolvimento da personalidade (Brasil, 2018). Essa lei implica, por exemplo, definir claramente quem é o controlador e quem é o operador em cada contrato, registrar bases legais, limitar acessos, adotar anonimização ou mascaramento em ambientes de desenvolvimento e manter registros de operações de tratamento.

Um registro vivo de riscos, com análise de impacto, probabilidade, plano de tratamento e responsáveis, ajuda a priorizar ações, dialogar com o jurídico, a segurança da informação e a privacidade, além de demonstrar diligência em auditorias e fiscalizações regulatórias (Brasil, 2018, Isaca, 2019).

PENSANDO JUNTOS

Ao analisar a fábrica de software em que você participa (ou pretende ingressar), pergunte-se: que evidências comprovam o que foi feito, quando e por quem? Como provar em auditoria que funcionalidades foram testadas, requisitos aprovados corretamente ou incidentes tratados conforme previsto? Há rastreabilidade entre requisitos, código, testes, releases e suporte ou depende de memória? Esses cenários revelam governança, riscos e conformidade como rotina profissional e chance de maturação de processos, inclusive em times pequenos sem certificações (Pressman; Maxim, 2016; Sommerville, 2019).



PADRONIZAÇÃO, ESCALA E CERTIFICAÇÕES

Por fim, governança, qualidade e padronização encontram-se na busca por **escala com previsibilidade**. Padronizar não significa engessar; significa decidir onde a organização precisa de consistência para reduzir variação indevida.

Templates de requisitos, modelos de plano de testes, formatos de *Statement of Work* (SOW), estruturas de SLAs e OLAs, políticas de *branch* e de pipeline CI/CD são exemplos de elementos que, quando padronizados, facilitam o trabalho de times internos e de fornecedores, simplificam o *onboarding* de novas pessoas e tornam os resultados mais comparáveis entre *squads* e projetos (Axelos, 2019; Softex, 2023).

A Figura 3 representa esse caminho da padronização de artefatos e práticas internas à capacidade de ganhar escala com previsibilidade e atender às exigências de certificações, bem como aos grandes contratos.



Figura 3 – Caminho da padronização à escala e às certificações em fábricas de software
Fonte: gerada por ChatGPT em 2 jan. 2026.

Descrição da imagem: fluxo em três blocos horizontais conectados por setas. O primeiro bloco, à esquerda, intitulado "Padronização de artefatos e práticas" e acompanhado pelo ícone de engrenagens, traz exemplos como templates de requisitos, testes e documentação, formatos padronizados de SOW, SLAs e OLAs, padrões de *branch* e pipeline CI/CD. O bloco central, intitulado "Escala com previsibilidade" e acompanhado pelo ícone de um gráfico de barras com uma seta em modo ascendente, destaca benefícios como redução de variação indevida entre times, *onboarding* mais rápido de pessoas, fornecedores e resultados comparáveis entre projetos. O terceiro bloco, à direita, intitulado "Certificações e grandes contratos", acompanhado pelo ícone de um diploma, relaciona a existência de padrões consolidados à capacidade de atender às exigências de MPS.BR, CMMI e ISO/IEC 12207, participar de editais e contratos com órgãos públicos, além de aumentar a confiança de clientes em sistemas críticos. O conjunto reforça que a padronização bem aplicada é um passo para escalar a fábrica e preparar-se às certificações formais. Fim da descrição.

Os modelos de referência, como o MPS.BR e o CMMI, ao descreverem práticas esperadas em diferentes níveis de maturidade, ajudam a orientar quais padrões devem ser consolidados primeiro para a fábrica avançar de forma sustentável (CMMI Institute, 2018; Softex, 2023).

Preparando-se para grandes clientes e órgãos públicos

Quando a fábrica passa a atender a contratos mais complexos ou órgãos públicos, exigências formais de certificações, níveis de maturidade ou aderência a normas tornam-se comuns em editais, RFPs e processos de homologação de fornecedores.

Modelos como MPS.BR, CMMI, ISO/IEC/IEEE 12207, ITIL e Cobit deixam de ser apenas referências teóricas e começam a compor a lista de pré-requisitos para participar de licitações, renovar contratos ou atuar em sistemas críticos de alta responsabilidade (Axelos, 2019; CMMI Institute, 2018; ISO, 2017; Isaca, 2019; Softex, 2023). Nesses contextos, calcular o retorno sobre o investimento em governança e padronização envolve observar a redução de incidentes graves, o aumento da confiança de clientes, a possibilidade de disputar novos projetos, a queda de custos com retrabalho e correções emergenciais, bem como a mitigação de riscos legais, inclusive aqueles ligados à LGPD (Brasil, 2018).

Ao desenvolver seu potencial neste tema, o convite é ir além da visão de que "alguém lá em cima cuida da governança" e perceber como cada papel – desenvolvedor, analista, *tester*, líder técnico, gestor – contribui para que processos, métricas, riscos e padrões façam sentido na prática. A partir desse entendimento, você estará mais preparado para atuar em fábricas de software que buscam maturidade real, capazes de equilibrar controle e agilidade, segurança e inovação em ambientes tão regulados quanto competitivos.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

À medida que você avança em conhecimento, começa a ficar claro que governança, qualidade e padronização não são apenas "temas da prova", mas competências que aparecem em praticamente qualquer ambiente profissional de TI.

Em uma startup, elas surgem na hora de transformar o caos das primeiras entregas em um processo minimamente previsível, com *backlog* priorizado, pi-

peline confiável e indicadores simples que mostrem se o produto está evoluindo na direção certa. Em empresas médias, aparecem quando múltiplos times, fornecedores e áreas de negócio precisam falar a mesma língua sobre SLAs, incidentes, mudanças, segurança e LGPD. Em grandes organizações, tornam-se requisito explícito em auditorias, certificações, licitações e contratos de alto valor.

Para você, tudo isso se traduz em novos papéis e possibilidades de carreira. Desenvolvedores, analistas de testes, engenheiros DevOps e especialistas em segurança que compreendem conceitos como Raci, métricas de qualidade, rastreabilidade, gestão de configuração e modelos de maturidade (CMMI, MPS.BR, Itil, Cobit) tornam-se profissionais raros: conseguem dialogar com pessoas dos setores de negócio, jurídico e auditoria sem abandonar a visão técnica. Em muitos times, surgem funções híbridas – *tech leads*, *service owners*, analistas de qualidade, analistas de governança de TI – que exigem exatamente essa capacidade de conectar código, processos, contratos e indicadores para tomar decisões mais responsáveis e sustentáveis.

No horizonte, as demandas de mercado tendem a intensificar essa necessidade. A expansão de serviços digitais em governo e setores regulados, o aumento da exposição a incidentes de segurança, bem como as exigências de proteção de dados fazem empresas buscarem fábricas de software capazes de provar como trabalham, e não apenas o que entregam. Esse cenário abre espaço para consultorias especializadas, equipes internas de melhoria de processo, núcleos de qualidade e de conformidade tecnológica que atuam lado a lado com os times de produto.

Saber ler um edital, entender o impacto de uma certificação, participar de uma auditoria ou propor um ajuste de processo baseado em dados deixa de ser "coisa do gestor", tornando-se um diferencial competitivo também para quem está na linha de frente do desenvolvimento.

No horizonte, as demandas de mercado tendem a intensificar essa necessidade

O convite, daqui em diante, é olhar com essa lente ampliada para cada experiência de estágio, emprego ou projeto: que decisões de governança estão sendo tomadas? Quais métricas importam para aquele contexto? Como a organização lida com riscos, privacidade, segurança e padronização? Ao fazer essas conexões entre teoria e prática, você se prepara para responder às exigências do mercado de trabalho, mas também se posiciona como alguém capaz de ajudar a construir ambientes de desenvolvimento mais maduros, seguros e confiáveis, um diferencial cada vez mais valorizado em fábricas de software de qualquer porte.

VAMOS PRATICAR

1. Tornar explícitas responsabilidades e fluxos de decisão ajuda a reduzir zonas cinzentas em priorização, mudanças e aceites. Matrizes Raci, combinadas a políticas e critérios objetivos, facilitam a leitura comum entre áreas técnicas e de negócio. O grau de formalização depende do risco e do contexto organizacional (Isaca, 2019; Pressman; Maxim, 2016).

   Qual alternativa melhor representa o uso de Raci em fábricas de software?

   a) Responsáveis alternam-se informalmente a cada incidente.
   b) Raci explícito por processo crítico, com critérios de passagem e registro de decisões.
   c) Toda decisão é coletiva, sem papéis definidos.
   d) Papéis definidos apenas para produção; desenvolvimento segue livre.
   e) Raci substitui qualquer política escrita.

2. Métricas permitem comparar cenários e orientar melhorias. Leituras de fluxo observam variação e gargalos; medidas de produto indicam estabilidade e retrabalho; índices de remoção de defeitos mostram efetividade de verificação. A utilidade depende de coletas consistentes e uso em decisões (Pressman; Maxim, 2016; Sommerville, 2019).

   Qual combinação melhor representa o acompanhamento de qualidade?

   a) Apenas contagem de horas.
   b) Número de reuniões por mês.
   c) Apenas satisfação subjetiva.
   d) Tamanho de equipe como proxy de qualidade.
   e) Defeitos em produção, DRE por etapa e cobertura de testes.

3. Em ambientes regulados, evidências precisam demonstrar o que foi feito, quando, por quem e com quais critérios. Rastreabilidade liga requisitos, tarefas, alterações, verificações e liberações, apoiando auditorias e aprendizado. A profundidade da trilha depende de risco e exigência externa (Axelos, 2019; Pressman; Maxim, 2016).

   Qual prática configura rastreabilidade efetiva em fábricas?

   a) Registro manual eventual em planilhas independentes.
   b) Ligação sistemática requisito-tarefa-*commit*-*build*/teste-*release*-incidente-decisão.
   c) Apenas *changelog* genérico no fim do projeto.
   d) Histórico de e-mails sem associação a itens.
   e) Depoimentos informais em reuniões.

REFERÊNCIAS

AXELOS. **ITIL foundation**. 4. ed. London: The Stationery Office, 2019.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014. Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 23 jan. 2026.

CMMI INSTITUTE. **CMMI Development V2.0**: improving performance, getting results. Pittsburgh: CMMI Institute, 2018.

ISACA – INFORMATION SYSTEMS AUDIT AND CONTROL ASSOCIATION. **Cobit 2019 framework**: governance and management objectives. Schaumburg: Isaca, 2019.

ISO – INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 12207**: Systems and software engineering – Software life cycle processes. Geneva: ISO, 2017.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOFTEX – ASSOCIAÇÃO PARA PROMOÇÃO DA EXCELÊNCIA DO SOFTWARE BRASILEIRO. **MPS.BR** – melhoria de processo do software brasileiro: guia geral MPS de software. Brasília, DF: Softex, 2023.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

CONFIRA SUAS RESPOSTAS

1. Alternativa B.

   a) Incorreta: pois mantém ambiguidade e instabilidade.
   b) Correta: pois Raci explícito, critérios e registros tornam decisões auditáveis.
   c) Incorreta: pois a ausência de papéis dificulta a responsabilização.
   d) Incorreta: pois cria descontinuidade entre desenvolvimento e operação.
   e) Incorreta: pois Raci complementa políticas, não substitui.

2. Alternativa E.

   a) Incorreta: pois não mede qualidade.
   b) Incorreta: pois reuniões não refletem qualidade.
   c) Incorreta: pois falta objetividade e repetibilidade.
   d) Incorreta: pois o tamanho da equipe não é métrica de qualidade.
   e) Correta: pois combina produto (defeitos), efetividade (DRE) e suporte (cobertura).

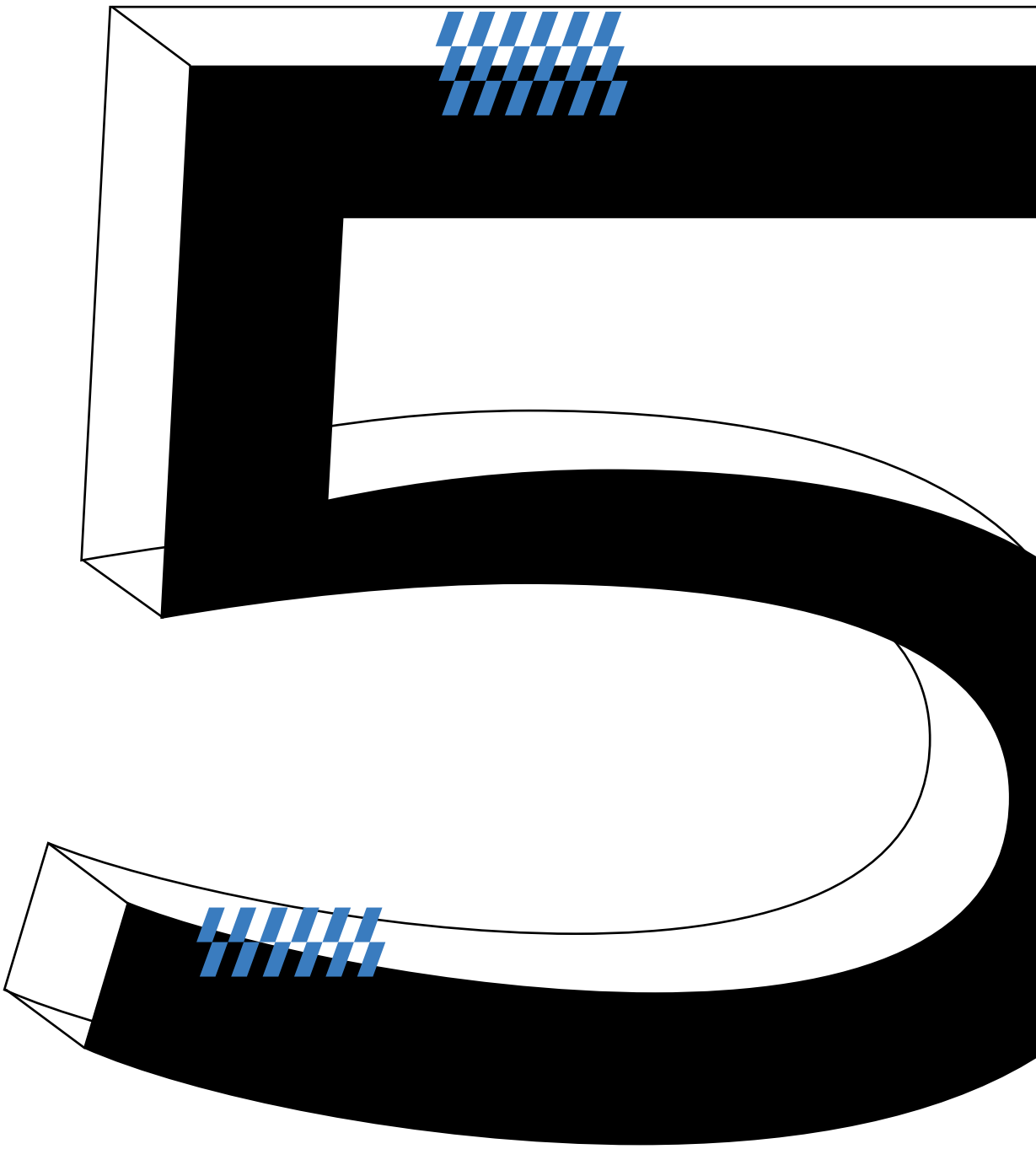
3. Alternativa B.

   a) Incorreta: pois a dispersão compromete a confiabilidade.
   b) Correta: pois estabelece uma trilha de ponta a ponta verificável.
   c) Incorreta: pois perde granularidade e temporalidade.
   d) Incorreta: pois não há vínculo entre artefatos.
   e) Incorreta: pois carece de evidência objetiva.

MEU ESPAÇO

Unicesumar

unidade





TEMA DE APRENDIZAGEM 8

TECNOLOGIAS EMERGENTES NO CONTEXTO DA FÁBRICA

MINHAS METAS

- Compreender IA, DevOps, dados, *cloud* e *low-/no-code* em fábricas.
- Diferenciar as finalidades e os elementos básicos de cada tecnologia.
- Entender os impactos teóricos em produtividade, qualidade e custos.
- Relacionar as tecnologias a falhas, manutenção preditiva e personalização.
- Identificar os requisitos de governança, segurança e LGPD.
- Mapear arquiteturas e padrões de referência para adoção em escala.
- Avaliar casos brasileiros em leitura, registrando benefícios e riscos.

INICIE SUA JORNADA

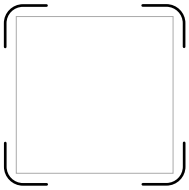
Em sua realidade profissional, tecnologias emergentes, como Inteligência Artificial (IA), nuvem, automação, *analytics* e *low-code* ou *no-code,* estão fortalecendo as tomadas de decisão na fábrica de software, com métricas e evidências confiáveis, ou estão sendo adotadas mais como "tendência", sem impacto claro no resultado? Essa pergunta é importante, porque, na prática, o desafio de uma fábrica de software não é somente entregar rápido: é manter o ritmo de entrega com qualidade, previsibilidade e segurança, mesmo quando o cenário muda, prazos encurtam, equipes se reorganizam e sistemas já em produção precisam continuar funcionando. Em muitos ambientes, novas tecnologias entram como a promessa de ganho imediato, mas apenas geram valor, de fato, quando se conectam ao fluxo real de trabalho e produzem evidências que sustentem decisões; caso contrário, podem aumentar ruídos, custos e complexidade, gerando a sensação de modernização sem melhorar o que realmente importa.

Para iniciar a sua jornada, faça um experimento rápido, em forma de registro curto. Considere o seguinte caso hipotético: uma equipe precisa entregar uma funcionalidade em duas semanas. O sistema já está em produção e existe histórico de retrabalho após releases; assim, a liderança quer reduzir atrasos sem elevar a taxa de incidentes. Escolha uma única tecnologia emergente (como IA, *cloud*, *analytics*, automação ou *low-code*/*no-code*) e, em até dez linhas, descreva onde ela entraria no fluxo (requisitos, desenvolvimento, testes, *deploy* ou operação), qual resultado você pretende melhorar (como *lead time*, retrabalho, falhas pós-*release* ou tempo de recuperação), quais evidências mostrariam que houve melhora (métricas de fluxo, resultados de testes, *logs*, indicadores de incidentes ou relatórios automatizados) e qual risco relevante essa adoção traz, junto de um controle para mitigá-lo (como validação humana obrigatória, critérios de uso, limites de escopo, políticas de dados ou governança). Esse exercício será retomado ao final do tema, para que você refina a sua resposta com base nos conceitos e conexões desenvolvidos ao longo do estudo.

Ao concluir o seu registro, reflita: as evidências que você escolheu realmente demonstram ganho real, ou apenas dão uma impressão de melhora? O que, no seu desenho, reduz os riscos e aumenta a previsibilidade, e o que pode gerar novos pontos de falha, como dependência de ferramenta, dados de baixa qualidade ou automação sem validação? Se os indicadores não melhorarem, quais sinais mostrariam que o problema está menos na tecnologia e mais no modo de adoção, na integração ao processo ou na ausência de critérios claros? Por fim, observe se a sua proposta fortalece uma cultura de decisão baseada em evidências ou se apenas adiciona mais uma ferramenta ao processo – e o que faltaria para essa escolha se tornar uma prática consistente de fábrica de software.

PLAY NO CONHECIMENTO

Este podcast explica como IA, DevOps/CI-CD, *analytics, cloud* e *low/no-code* se encaixam no fluxo da fábrica de software e o que muda em relação à qualidade, segurança e previsibilidade das entregas. Dê o play e acompanhe o panorama!



VAMOS RECORDAR?

Antes de avançar para tecnologias, tais como Inteligência Artificial, DevOps, ***Big Data Analytics***, computação em nuvem e plataformas ***low-code/no-code***, é importante retomar alguns fundamentos do desenvolvimento industrializado de software.

O ciclo de vida do software organiza as etapas que vão desde a identificação da necessidade até a manutenção do sistema em produção. Em ambientes estruturados, essas etapas são apoiadas por **processos definidos**, papéis claros, artefatos padronizados, métricas de desempenho e critérios de qualidade.

Também é essencial lembrar que automação, rastreabilidade, testes contínuos e monitoramento não são complementos opcionais, mas componentes estruturais de um sistema confiável. Esses elementos garantem previsibilidade, controle de risco e capacidade de evolução sustentável.

DESENVOLVA SEU POTENCIAL

TECNOLOGIAS EMERGENTES NA ROTINA DA FÁBRICA

Quando se fala em tecnologias emergentes na fábrica de software, muitas pessoas pensam primeiramente em "novidade da moda": IA generativa, ferramentas de DevOps, *Big Data*/*analytics*, *cloud computing* e plataformas *low-code*/*no-code*. Contudo, no dia a dia, elas aparecem em decisões bem concretas: como o código é integrado e entregue (pipelines), como ambientes são criados e descartados (nuvem), como o que acontece em produção é observado (*logs*, métricas e rastreamento), como grandes volumes de dados se tornam insumos de decisão (*analytics*) e como soluções internas são montadas com blocos prontos (*low-*/*no-code*) (Bass; Weber; Zhu, 2015; Pressman; Maxim, 2016; Sommerville, 2019).

O objetivo deste tema não é "usar tudo ao mesmo tempo", mas criar um raciocínio organizado: entender o papel de cada peça no fluxo da fábrica e ser capaz de avaliar, com calma, onde a tecnologia realmente reduz retrabalho, encurta ciclos, melhora a qualidade e/ou ajuda a cumprir os requisitos de negócio com segurança e conformidade.

IA, DevOps, analytics, cloud e low-/no-code

Na base de tudo, o **DevOps** ajuda a organizar o fluxo: pipelines de integração e entrega contínua aproximam desenvolvimento e operação, automatizando etapas de *build*, testes e *deploy* com feedback rápido a cada mudança (Bass; Weber; Zhu, 2015). Sobre essa "esteira", outras tecnologias se encaixam. A **Inteligência Artificial** pode apoiar a revisão de código, a geração de testes, a detecção de anomalias em *logs* e a triagem de chamados – sempre como apoio à decisão, e não como substituta do julgamento humano (Russell; Norvig, 2021). Já o **Big Data/analytics** e a engenharia de dados consolidam *logs*, métricas e eventos em painéis que mostram gargalos de fluxo, padrões de defeitos, comportamento de usuários e sinais de risco operacional, conectando dados técnicos a indicadores de negócio (Provost; Fawcett, 2013).

A **cloud computing**, com provisionamento sob demanda e modelos de serviço (IaaS, PaaS, SaaS), facilita a criação de ambientes efêmeros para testes, a escalabilidade em momentos de pico e a padronização de componentes de infraestrutura, desde que haja políticas de acesso e leitura atenta de custos (Mell; Grance, 2011). Plataformas **low-code/no-code**, por sua vez, aceleram aplicações internas, protótipos e automações simples, quando inseridas em um contexto de governança, versionamento e segurança que evite "ilhas de solução" difíceis de manter (Pressman; Maxim, 2016). Assim, tecnologias emergentes deixam de ser peças isoladas e passam a compor um "kit de ferramentas" a serviço do fluxo da fábrica.

A **Figura 1** sintetiza a ideia de "esteira DevOps" como base, mostrando como IA, *analytics, cloud* e *low-/no-code* se encaixam em torno do pipeline da fábrica.

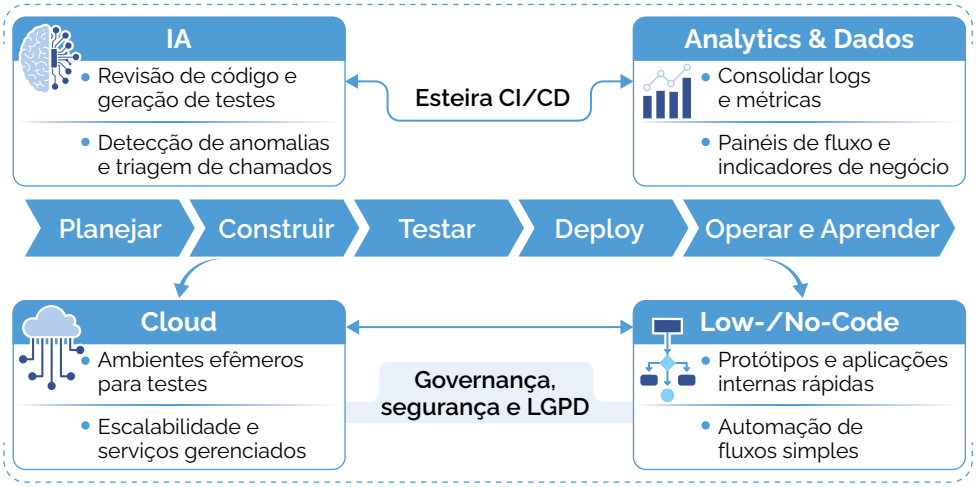


Figura 1 – Tecnologias emergentes sobre a esteira DevOps na fábrica de software
Fonte: gerada por ChatGPT em 9 jan. 2026.

Descrição da imagem: é exibida uma esteira DevOps de CI/CD organizada em formato circular, com as etapas Planejar, Construir, Testar, Deploy e Operar & Aprender posicionadas ao centro, representando o fluxo contínuo de desenvolvimento e entrega. Ao redor desse ciclo, quatro cartões destacam tecnologias emergentes que o potencializam: IA, aplicada à revisão de código, geração automatizada de testes e detecção de anomalias; *Analytics* & *Dados*, voltados à consolidação de logs e métricas e à criação de painéis de negócio; *Cloud*, relacionada ao uso de ambientes efêmeros e à escalabilidade sob demanda; e *Low-/No-Code*, destinada à criação de protótipos rápidos e à automação de fluxos simples. Na base da imagem, uma faixa horizontal indica que todo o processo é permeado por governança, segurança e conformidade com a Lei Geral de Proteção de Dados Pessoais, reforçando a necessidade de controle e proteção de dados ao longo de todo o ciclo. Fim da descrição.

Em torno desse pipeline estruturado, tecnologias, como IA, *analytics*, *cloud* e *low-/no-code* se conectam para ampliar a produtividade, a qualidade e a capacidade de decisão, mas sempre integradas aos critérios de versionamento, validação e governança já definidos. A Figura 1 reforça que a inovação gera valor quando está alinhada a um fluxo controlado, mensurável e retroalimentado por dados de operação.

ZOOM NO CONHECIMENTO

Embora frequentemente associadas, essas tecnologias têm propósitos distintos. **DevOps** é uma abordagem de trabalho que integra desenvolvimento e operação por meio de pipelines, automação e monitoramento (Bass; Weber; Zhu, 2015). **Cloud computing** é a infraestrutura elástica onde os serviços executam nos modelos IaaS, PaaS e SaaS (Mell; Grance, 2011). **Big Data/analytics** analisa grandes volumes de dados para apoiar decisões (Provost; Fawcett, 2013). **IA** automatiza tarefas cognitivas, e **low-/no-code** acelera o desenvolvimento com componentes visuais (Pressman; Maxim, 2016; Russell; Norvig, 2021). Diferenciá-las evita confusões e orienta escolhas adequadas na fábrica.



Produtividade, qualidade, custo e riscos na mesma mesa

O impacto dessas tecnologias não se mede apenas pela "velocidade do time", mas pelo equilíbrio entre **produtividade, qualidade, custos e riscos**. DevOps bem implementado tende a reduzir *lead time* e aumentar a previsibilidade de *releases*; *analytics* ajuda a identificar onde o fluxo trava e quais tipos de demanda mais geram incidentes; *cloud* permite ajustar capacidade à demanda, evitando tanto ociosidade quanto indisponibilidade; por fim, IA e *low-/no-code* podem liberar pessoas de tarefas repetitivas para focar em atividades de maior valor (Bass; Weber; Zhu, 2015; Provost; Fawcett, 2013; Pressman; Maxim, 2016).

IA NA FÁBRICA:

- Apoia a revisão de código, a geração de testes e a análise de *logs*.
- Exige validação humana, critérios de uso e registro de decisões (Russell; Norvig, 2021).

DEVOPS / CI-CD:

- Organiza o fluxo da demanda ao *deploy* com automação e feedback contínuo.
- Depende de padrões de *branch*, critérios de DoR/DoD e métricas de fluxo (Bass; Weber; Zhu, 2015).

ANALYTICS E DADOS:

- Consolida sinais do sistema (*logs*, métricas, eventos) para priorizar melhorias.
- Liga indicadores técnicos a resultados de negócio (Provost; Fawcett, 2013).

CLOUD E LOW-/NO-CODE:

- *Cloud* oferece ambientes padronizados, efêmeros e elásticos (Mell; Grance, 2011).
- *Low-/no-code* acelera soluções internas, desde que integrado à governança e à segurança (Pressman; Maxim, 2016).

Ao mesmo tempo, cada escolha traz riscos: dependência de um único provedor de nuvem, modelos de IA pouco explicáveis, soluções *low-code* espalhadas sem controle, custos variáveis sem monitoramento e exposição de dados pessoais em ambientes mal configurados (Russell; Norvig, 2021; Mell; Grance, 2011).

No contexto brasileiro, a Lei Geral de Proteção de Dados Pessoais (**LGPD**) exige atenção especial ao tratamento de dados pessoais, desde a coleta até o descarte, o que impacta diretamente as arquiteturas em *cloud*, projetos de *analytics* e o uso de IA em decisões automatizadas (Brasil, 2018). Desenvolver o seu potencial neste tema significa aprender a visualizar benefícios e riscos usando conceitos, como TCO/ROI, SLAs, métricas de fluxo, indicadores de qualidade e requisitos de privacidade, para justificar ou questionar a adoção de uma tecnologia emergente.

A **Figura 2** resume essa visão, ao mostrar a decisão em adotar tecnologias emergentes como equilíbrio entre produtividade, qualidade, custo e riscos.

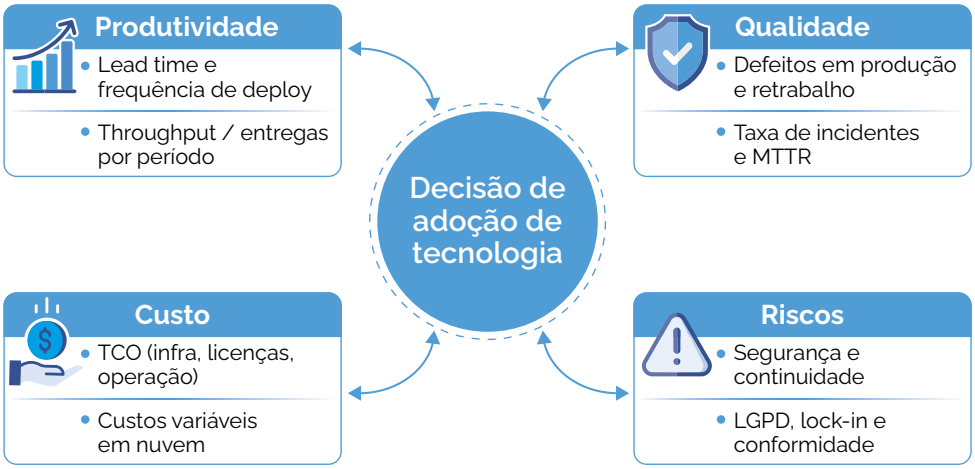


Figura 2 – Produtividade, qualidade, custo e riscos / Fonte: gerada por ChatGPT em 9 jan. 2026.

Descrição da imagem: é exibido um círculo central com a expressão "Decisão de adoção de tecnologia", do qual partem setas em direção a quatro cartões dispostos ao redor. O primeiro cartão, Produtividade, reúne indicadores como *lead time*, frequência de *deploy* e *throughput*. O segundo, Qualidade, contempla métricas como defeitos em produção, retrabalho, incidentes e MTTR (tempo médio de recuperação). O terceiro, Custo, aborda TCO (custo total de propriedade) e custos variáveis em nuvem. O quarto cartão, Riscos, inclui aspectos relacionados à segurança, continuidade do negócio, conformidade com a Lei Geral de Proteção de Dados Pessoais, *lock-in* tecnológico e exigências regulatórias, evidenciando que a decisão central considera simultaneamente desempenho, qualidade, viabilidade econômica e governança. Fim da descrição.

A Figura 2 representa a adoção de tecnologia como uma decisão estratégica central sustentada por quatro dimensões interligadas:

- **Produtividade**: medida por indicadores, como *lead time*, frequência de *deploy* e *throughput*.
- **Qualidade**: avaliada por defeitos em produção, retrabalho, incidentes e MTTR.
- **Custo**: considerando TCO e despesas variáveis, especialmente em nuvem.
- **Riscos**: envolvem segurança, continuidade, LGPD, *lock-in* e conformidade.

As setas indicam que esses fatores se influenciam mutuamente, exigindo equilíbrio entre desempenho, estabilidade, investimento e governança na tomada de decisão.

EXEMPLIFICANDO

Uma forma simples de trazer essa lógica para a sua realidade é escolher um sistema (real ou hipotético) e registrar, em uma tabela, como está hoje e como ficaria com o apoio de tecnologias emergentes. Por exemplo: na coluna "Produtividade", você compara o lead time atual com o que poderia mudar com um pipeline de CI/CD; em "Qualidade", avalia a quantidade de incidentes e como testes automatizados e analytics poderiam reduzir falhas e reaberturas; em "Custo", estima o impacto de mover parte da carga para cloud, considerando consumo e ciclo de vida; em "Riscos", analisa a exposição de dados pessoais, dependência de fornecedores e necessidade de conformidade com a LGPD (Brasil, 2018; Pressman; Maxim, 2016). Esse exercício ajuda a visualizar as tecnologias emergentes como parte de decisões estruturadas, e não apenas como "mais uma ferramenta" adicionada ao ambiente.

ARQUITETURAS, GOVERNANÇA E ADOÇÃO EM ESCALA

A adoção sustentável de tecnologias emergentes passa por arquiteturas bem pensadas e plataformas internas que organizam o uso dessas tecnologias em escala. Do ponto de vista arquitetural, práticas de DevOps e *cloud* incentivam a criação de serviços bem delimitados, com APIs claras, observabilidade embutida (*logs*, métricas e rastreamento) e contratos estáveis entre componentes, o que facilita a evolução de partes do sistema sem paralisar tudo (Bass; Weber; Zhu, 2015; Sommerville, 2019).

Do ponto de vista organizacional, faz diferença quando a fábrica deixa de ter apenas provas de conceito dispersas e passa a oferecer uma plataforma interna comum: serviços de autenticação, catálogo de dados, pipelines-padrão, modelos de IA validados, *guidelines* para *low-code*/*no-code*, *templates* de *dashboards* e bibliotecas de testes (Pressman; Maxim, 2016).

Da prova de conceito à plataforma compartilhada

Em muitas organizações, o primeiro contato com tecnologias emergentes acontece em experimentos pontuais: um time monta o próprio *stack* de IA, outro cria um pipeline diferente, um terceiro faz um projeto de *analytics* isolado. Sem coordenação, isso gera ilhas de tecnologia difíceis de manter. A transição desejada é transformar o que funcionou em **capacidades de plataforma compartilhadas**:

serviços de dados com governança definida, pipelines reutilizáveis, componentes de segurança padronizados, modelos de IA treinados e monitorados, ambientes de *cloud* com padrões mínimos de configuração e catálogos de soluções *low-/no-code* aprovadas (Mell; Grance, 2011; Brasil, 2018; Pressman; Maxim, 2016).

A governança aparece na definição de papéis e responsabilidades: quem cuida da plataforma, quem consome, como são tratados os pedidos de exceção e como novas tecnologias são avaliadas e incorporadas ao "kit básico" da fábrica. Assim, a tecnologia emergente deixa de ser uma iniciativa individual e passa a ser **capacidade organizacional**, alinhada à ideia de maturidade progressiva discutida em modelos de melhoria de processo (Pressman; Maxim, 2016; Sommerville, 2019).

A **Figura 3** ilustra essa transição, mostrando como diferentes tecnologias emergentes convergem para uma plataforma interna, apoiada por governança, segurança, políticas de uso e avaliação de TCO/ROI.



Figura 3 – Da prova de conceito à plataforma interna para tecnologias emergentes
Fonte: gerada por ChatGPT em 9 jan. 2026.

Descrição da imagem: a figura apresenta, à esquerda, quatro cartões verticais identificados como "IA", "Analytics & Dados", "Cloud" e "Low-/No-Code", cada um acompanhado de um ícone representativo, com setas direcionadas a um bloco central denominado "Plataforma Interna". Abaixo desse bloco aparece a inscrição "TCO/ROI", acompanhada de moedas, indicando a análise de custo total de propriedade e retorno sobre investimento. À direita, há uma escada iluminada com quatro degraus que, de baixo para cima, traz as etapas "Escolha criteriosa", "Medição de impacto", "Experimentos controlados" e "Evolução sustentada", conduzindo a um alvo com bandeira no topo, simbolizando o alcance de objetivos estratégicos. Na base da composição, uma faixa com o texto "Governança | Segurança | Políticas de uso" reforça que a adoção em escala dessas tecnologias depende dessas camadas estruturantes. Fim da descrição.

Essa convergência para uma plataforma comum também favorece a padronização, a escalabilidade e o aprendizado organizacional. Quando serviços, pipelines e componentes passam a ser compartilhados, reduzem-se duplicidades, inconsistências técnicas e riscos de segurança, além de facilitar auditorias e medições de desempenho. A avaliação contínua de TCO e ROI permite verificar se a tecnologia gera valor sustentável, enquanto políticas claras de uso e monitoramento asseguram conformidade e previsibilidade. Desse modo, a inovação deixa de depender de iniciativas isoladas e passa a integrar a estratégia da fábrica, fortalecendo sua capacidade de evoluir de forma coordenada e sustentável.

INDICAÇÃO DE LIVRO

Manual de DevOps

Para aprofundar a sua visão de "plataforma" e entender como grandes organizações estruturam DevOps, *cloud* e automação em escala, vale a leitura de *Manual de DevOps*, de autoria de Gene Kim e outros estudiosos. A obra mostra, com exemplos práticos, como pipelines, governança, observabilidade e plataformas internas se combinam para viabilizar entrega contínua com segurança e confiabilidade.



Além disso, essa estrutura comum facilita a incorporação controlada de novas soluções, pois estabelece critérios objetivos para testes, homologação e expansão. Com padrões definidos e responsabilidades claras, a organização reduz improvisações, aumenta a transparência das decisões e cria um ambiente propício à inovação contínua, sem comprometer a estabilidade, a segurança ou a governança.



PENSANDO JUNTOS

Ao consolidar tecnologias emergentes em plataformas internas com arquiteturas definidas, APIs estáveis, observabilidade e políticas de uso, a fábrica cria um ambiente onde experimentar é mais seguro. Novos serviços passam a reutilizar bases comuns de dados, identidade, monitoramento e automação, evitando retrabalho e riscos desnecessários. Quando integradas a métricas de fluxo, qualidade e conformidade – como proteção de dados e transparência algorítmica –, IA, *cloud*, *analytics* e *low-/no-code* deixam de ser modismo e se tornam uma estratégia orientada por evidências e melhoria contínua.

Governança, segurança e LGPD na prática

Por fim, tecnologias emergentes trazem à tona questões de governança, segurança e privacidade. Sistemas que utilizam modelos de IA para recomendar ações, segmentar usuários ou automatizar decisões precisam de critérios de explicabilidade, supervisão humana e registro das versões de modelos utilizadas (Russell; Norvig, 2021). Ambientes em nuvem e pipelines de dados exigem políticas de acesso mínimo necessário, criptografia adequada, monitoramento de anomalias e procedimentos claros de resposta a incidentes (Mell; Grance, 2011). A **LGPD** reforça que o tratamento de dados pessoais precisa ter base legal, finalidade específica, transparência e medidas de segurança proporcionais ao risco, o que vale tanto para grandes plataformas digitais quanto para pequenos sistemas internos (Brasil, 2018).

Entender esses requisitos e integrá-los ao desenho de soluções com IA, *cloud*, *analytics* e *low-/no-code* são atitudes que integram a responsabilidade profissional em fábricas de software modernas. Autores clássicos de engenharia de software lembram que qualidade, segurança e manutenibilidade não surgem por acaso: são resultado de decisões conscientes sobre processo, arquitetura, testes e operação (Pressman; Maxim, 2016; Sommerville, 2019). Tecnologias emergentes ampliam o repertório de ferramentas, mas também exigem mais maturidade para equilibrar inovação, risco e responsabilidade.



EM FOCO

Acesse o seu Ambiente Virtual de Aprendizagem e confira a aula referente a este tema.

NOVOS DESAFIOS

Ao longo deste tema, você entendeu que IA, DevOps, computação em nuvem, *analytics* e plataformas low-/no-code deixaram de ser apenas "novidades tecnológicas" para se tornar parte da infraestrutura básica das fábricas de software. Em contextos tão diferentes quanto startups, empresas consolidadas e órgãos públicos, essas tecnologias são usadas para ganhar velocidade, modernizar sistemas legados, reduzir incidentes e tomar decisões mais embasadas em dados, sempre sob exigências de governança, segurança da informação e conformidade com a LGPD. Mais do que dominar ferramentas específicas, o foco foi compreender como cada uma delas se encaixa no fluxo da fábrica, nas métricas de desempenho e nos controles necessários.

Retomando o experimento proposto no início do tema, em que você foi convidado a analisar um processo da fábrica (como testes, *deploy*, monitoramento ou atendimento a incidentes) e a imaginar como ele poderia ser transformado com o uso de tecnologias emergentes, agora, é possível fechar o ciclo. A partir dos conceitos estudados, esse processo passa a ser redesenhado com etapas mais automatizadas (via pipelines DevOps e *low-*/*no-code*), apoiadas por dados (*logs*, métricas e *dashboards* em *cloud*/*analytics*) e, quando fizer sentido, com o auxílio de IA para sugerir correções, priorizar demandas ou antecipar falhas. Assim, o experimento deixa de ser apenas um exercício imaginativo e se torna um esboço de solução: você identifica onde atuar, quais dados coletar, quais ferramentas adotar e quais riscos monitorar ao longo do ciclo de vida do software.

Profissionalmente, essa visão integrada é um diferencial importante. O mercado de trabalho busca pessoas capazes de transitar entre código, processos e indicadores de valor, dialogando com diferentes papéis da fábrica de software (desenvolvimento, operações, dados, segurança e negócio). Ao compreender como IA, DevOps, *cloud, analytics* e *low-*/*no-code* podem ser combinados de forma responsável, você amplia a sua capacidade de participar de decisões sobre modernização de sistemas, automação de rotinas, definição de SLAs e avaliação de custo/benefício. Em outras palavras, o que você aprendeu não serve somente para "usar ferramentas", mas para se posicionar como alguém que ajuda a definir como a organização inovará com segurança, previsibilidade e foco em resultados.

VAMOS PRATICAR

1. Pipelines estruturam verificações e fornecem evidências para promover mudanças com menor incerteza. Quando outras tecnologias se apoiam nessa base, decisões tendem a ser mais reproduzíveis e auditáveis. A configuração deve focar naquilo que é relevante ao objetivo, equilibrando custo, velocidade e risco.

   Qual alternativa melhor caracteriza a relação entre CI/CD e a adoção de tecnologias emergentes?

   a) Substituir critérios por aprovações informais.
   b) Centralizar decisões em ferramentas sem critérios.
   c) Organizar estágios com critérios objetivos e evidências, servindo de base para integrações com IA, *cloud* e *analytics*.
   d) Executar verificações apenas em produção.
   e) Desacoplar pipeline de qualquer métrica.

2. Consolidar sinais do sistema e do processo (*logs*, métricas e eventos) torna o comportamento mais legível para priorização e melhoria. A utilidade depende de coleta, publicação e leitura consistentes no tempo. O foco é apoiar decisões proporcionais ao impacto.

   Qual combinação melhor representa o uso de *analytics* no fluxo?

   a) Painéis com métricas de pipeline, percentis de resposta e correlação com incidentes.
   b) Apenas contagem mensal de horas.
   c) Relatos informais em reuniões.
   d) Alertas só após incidentes críticos.
   e) Métricas financeiras sem relação com operação.

3. Ambientes padronizados e efêmeros reduzem divergências e aceleram validações. A escolha entre IaaS, PaaS ou SaaS deve refletir objetivos, riscos e governança de acesso. A leitura de custos precisa considerar consumo e ciclo de vida.

   Qual alternativa melhor traduz "ambientes efêmeros padronizados" na fábrica?

   a) Configuração manual por time, caso a caso.
   b) Criação via catálogos/*templating* e destruição controlada, integrada ao pipeline.
   c) Servidores fixos, sem automação.
   d) Apenas VMs locais sem política.
   e) Acesso irrestrito a produção.

REFERÊNCIAS

BASS, L.; WEBER, I.; ZHU, L. **DevOps**: a software architect's perspective. Boston: Addison-Wesley Professional, 2015.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, [2018]. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 5 mar. 2026.

MELL, P.; GRANCE, T. **The NIST definition of cloud computing**. Gaithersburg: National Institute of Standards and Technology, 2011.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

PROVOST, F.; FAWCETT, T. **Data Science for Business**: what you need to know about data mining and data-analytic thinking. Sebastopol: O'Reilly, 2013.

RUSSELL, S.; NORVIG, P. **Inteligência artificial**. 4. ed. Rio de Janeiro: LTC, 2021.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

CONFIRA SUAS RESPOSTAS

1. Alternativa C. Pipeline com critérios/evidências sustenta integrações seguras.

   As demais alternativas estão incorretas:

   a) Sem critérios, não há previsibilidade.
   b) Ferramenta não substitui governança.
   d) Se tardia, eleva risco.
   e) Sem métricas, falta controle.

2. Alternativa A. O uso de *analytics* no fluxo integra métricas de pipeline, percentis de resposta e correlação com incidentes, oferecendo sinais operacionais acionáveis e tempestivos.

   As demais alternativas estão incorretas:

   b) A contagem mensal de horas é administrativa, sem granularidade, nem vínculo com estabilidade/qualidade.
   c) Relatos informais são subjetivos, não reprodutíveis e sem série histórica auditável.
   d) Alertas apenas pós-incidente são reativos, aumentam MTTR e impedem prevenção.
   e) Métricas financeiras isoladas não revelam estado técnico (latência/erros/gargalos), nem orientam decisões de engenharia.

3. Alternativa B. Catálogos/*templating* criam e destroem ambientes de forma padronizada, integrada ao pipeline.

   As demais alternativas estão incorretas:

   a) A configuração manual por time aumenta variação, inconsistências e risco de erro.
   c) Servidores fixos sem automação geram rigidez, *drift* e lentidão para validar mudanças.
   d) Apenas VMs locais, sem política, impedem a padronização e a governança do ciclo de vida.
   e) O acesso irrestrito à produção eleva o risco operacional e viola as boas práticas de controle.

MEU ESPAÇO



TEMA DE APRENDIZAGEM 9

TENDÊNCIAS E FUTURO DAS FÁBRICAS DE SOFTWARE

MINHAS METAS

- Compreender ética, sustentabilidade, IA, *prompts*, confiabilidade e produtos.
- Entender os impactos dessas tendências.
- Estudar os princípios de engenharia ética e sustentabilidade.
- Relacionar a IA generativa e a engenharia de *prompts* aos fluxos de fábrica.
- Explorar as práticas de confiabilidade (observabilidade, resiliência e SRE).
- Mapear fábricas orientadas a produtos e métricas de valor.
- Avaliar o papel na transformação digital e desafios inerentes.

INICIE SUA JORNADA

Imagine que você está ingressando em uma fábrica de software atual: a cobrança não é só "entregar mais rápido". Agora, cada entrega precisa responder também a perguntas novas e incômodas: esse sistema é confiável em produção? O uso de dados é responsável? Há risco de vieses? O consumo de recursos é aceitável? A IA está ajudando ou criando retrabalho e insegurança? Ao mesmo tempo, os clientes (públicos e privados) desejam soluções digitais com impacto real, enquanto as equipes convivem com alta rotatividade, disputa por talentos e mudanças constantes de ferramentas e práticas.

Nesse cenário, "tendências" deixam de ser modismos e se tornam critério de sobrevivência profissional. Ética e sustentabilidade digital entram como requisitos (e não como "tema de palestra"); IA generativa e engenharia de *prompts* aparecem como parte do fluxo de trabalho; confiabilidade se torna compromisso de operação contínua (e não apenas testes antes do *deploy*); e muitas organizações deixam de pensar em "projeto com fim" para atuar com **produtos digitais** em evolução, guiados por métricas de valor. Entender essas mudanças ajuda você a enxergar o motivo pelo qual processos, arquitetura e qualidade estão sendo redesenhados ao mesmo tempo.

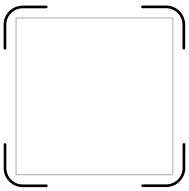
Pense em uma entrega comum (real ou hipotética) de sua área: um aplicativo de serviços, um sistema interno, uma solução em nuvem, um portal público ou um SaaS. Agora, "passe" essa entrega por um filtro rápido: quais dados pessoais entram no fluxo e quem decide como usá-los? Em qual etapa a IA generativa poderia acelerar (requisitos, testes, código ou documentação) e quais salvaguardas seriam necessárias (revisão humana, políticas, rastreabilidade e validação)? Como você perceberia rapidamente que algo deu errado (observabilidade), como reduziria o impacto (resiliência) e como controlaria o risco de mudança (práticas de SRE)? Qual métrica indicaria sucesso além de "entreguei": adoção, satisfação, tempo economizado, redução de incidentes ou melhoria de serviço?

Se o futuro da fábrica combina produtividade com responsabilidade, a diferença entre "usar tecnologia" e "ser engenheiro de software" se torna mais nítida: é preciso tomar decisões conscientes sobre arquitetura, qualidade, dados, riscos e valor entregue ao usuário.

Em sua realidade (ou na realidade profissional que você busca), a fábrica de software mede sucesso por volume de entregas ou por valor e confiabilidade em produção? Quais riscos éticos e de sustentabilidade se tornam invisíveis quando somente se olha para o prazo e o custo?

PLAY NO CONHECIMENTO

Em cinco minutos, este podcast mostra, sem jargões, as tendências que estão mudando as fábricas de software: ética e sustentabilidade digital, IA generativa e *prompts*, confiabilidade em produção (SRE/SLIs/SLOs) e orientação a produtos por métricas de valor.



VAMOS RECORDAR?

Antes de avançar para as tendências e o futuro das fábricas de software, retome três bases que sustentam o tema. A primeira é o fluxo ponta a ponta, da demanda à entrega: entenda como a solicitação se torna requisito, entra em planejamento, passa por desenvolvimento, testes e implantação, e como processos e evidências (critérios de pronto, registros de revisão, resultados de testes, *logs* de *deploy*) reduzem incerteza e aumentam previsibilidade. A segunda base é diferenciar métricas de eficiência (*lead time*, *throughput*, WIP e retrabalho) de métricas de qualidade e estabilidade (defeitos pós-*release*, incidentes, disponibilidade, MTTR): as primeiras mostram velocidade e capacidade, enquanto as segundas mostram confiabilidade daquilo que foi entregue. A terceira base é o papel de governança, risco e LGPD: definir responsabilidades, controles, tratamento de dados, auditoria e critérios de aceitação para garantir segurança, conformidade e transparência. Neste tema, essas bases serão ampliadas por IA generativa, sustentabilidade digital, confiabilidade (SRE/observabilidade) e orientação a produtos, com foco em métricas de valor.

DESENVOLVA SEU POTENCIAL

TENDÊNCIAS QUE JÁ ESTÃO NA FÁBRICA

Ao pensar no futuro das fábricas de software, é comum imaginar "tecnologias de amanhã". Entretanto, boa parte dessas tendências já está presente hoje: decisões sobre uso responsável de dados, preocupação com consumo de recursos e impacto ambiental, uso crescente de IA generativa em atividades de desenvolvimento e testes, pressão por confiabilidade contínua em produção e a mudança de "projetos que acabam" para produtos digitais em evolução (Pressman; Maxim, 2016; Sommerville, 2019). Ao contrário de tratar cada uma dessas frentes de forma isolada, propõe-se uma visão integrada: a mesma fábrica que precisa entregar rápido também é cobrada por ética, sustentabilidade, estabilidade em produção, inovação contínua e geração de valor para organizações públicas e privadas.

A Figura 1 sintetiza essas dimensões, ao articular engenharia ética e responsabilidade social, sustentabilidade digital, IA generativa e engenharia de *prompts*, confiabilidade (SRE) e fábricas orientadas a produtos digitais. A partir dela, os próximos tópicos aprofundam cada aspecto de maneira gradual, para que você consiga perceber como processos, arquitetura, qualidade e gestão de pessoas estão sendo redesenhados ao mesmo tempo.



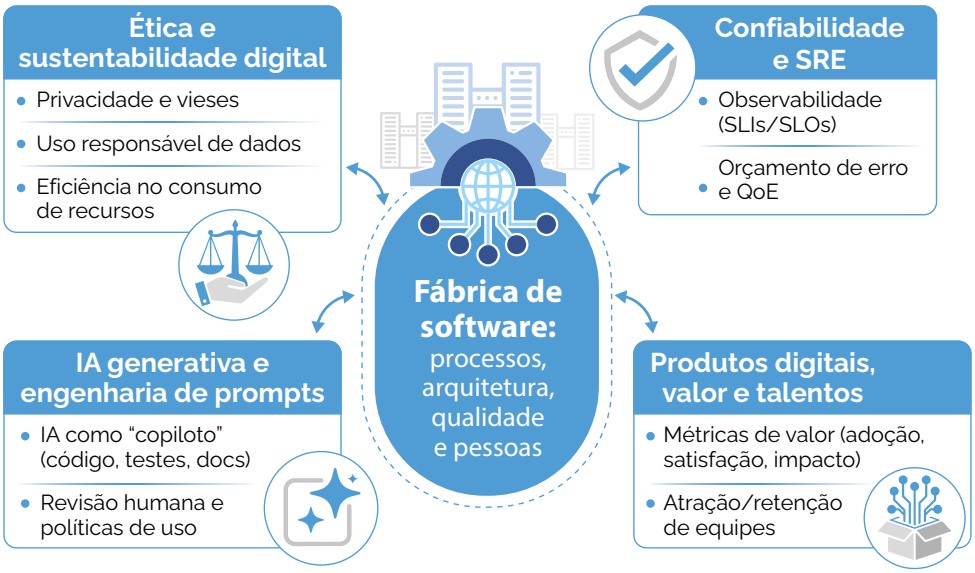


Figura 1 – Tendências que redesenham a fábrica de software do futuro
Fonte: gerada por ChatGPT em 14 jan. 2026.

Descrição da imagem: é exibido, ao centro, um círculo com o texto "Fábrica de Software". Ao redor dele, há quatro cartões retangulares conectados ao círculo por setas. O primeiro cartão traz o título "Ética e sustentabilidade digital", acompanhado das indicações "privacidade e vieses, uso responsável de dados e eficiência no consumo de recursos". O segundo cartão apresenta o título "IA generativa e engenharia de *prompts*", com os tópicos "IA como copiloto (código, testes e documentação)" e "revisão humana e políticas de uso". O terceiro cartão mostra o tema "Confiabilidade e SRE", com as indicações "observabilidade (SLIs/SLOs)" e "orçamento de erro e QoE". O quarto cartão apresenta "Produtos digitais, valor e talentos", acompanhado de "métricas de valor (adoção, satisfação e impacto)" e "atração e retenção de equipes". As setas indicam a ligação de cada um desses temas com o círculo central, representando sua relação com a ideia de "Fábrica de Software". Fim da descrição.

Engenharia ética, responsabilidade social e sustentabilidade digital

A **engenharia ética** desloca a discussão de "podemos construir?" para "devemos construir assim?". Questões, como viés algorítmico, discriminação indireta, falta de explicabilidade e uso abusivo de dados pessoais, tornaram-se centrais em sistemas que recomendam produtos, priorizam atendimentos, automatizam decisões ou apoiam políticas públicas (Russell; Norvig, 2021).

No contexto brasileiro, a Lei Geral de Proteção de Dados Pessoais (LGPD) impõe obrigações claras sobre coleta, tratamento e compartilhamento de dados pessoais, exigindo bases legais adequadas, transparência, minimização de dados e medidas de segurança proporcionais ao risco (Brasil, 2018). Na fábrica, isso se traduz em decisões concretas sobre quais dados coletar, como anonimizá-los, quem pode acessá-los e como registrar essas escolhas em políticas, contratos e pipelines.

A **sustentabilidade digital** complementa esse olhar, lembrando que software também consome recursos físicos: energia, capacidade de processamento, rede e armazenamento. Arquiteturas muito complexas, redundâncias desnecessárias e uso descontrolado de recursos em nuvem elevam custos e impacto ambiental (Pressman; Maxim, 2016). A engenharia de software passa, então, a considerar eficiência energética, uso racional de ambientes, vida útil de componentes e métrica de consumo por transação como parte da qualidade do produto, e não apenas como detalhe operacional.



A Figura 2 organiza esses elementos em três frentes complementares: privacidade/LGPD, sustentabilidade digital e governança baseada em métricas e evidências.

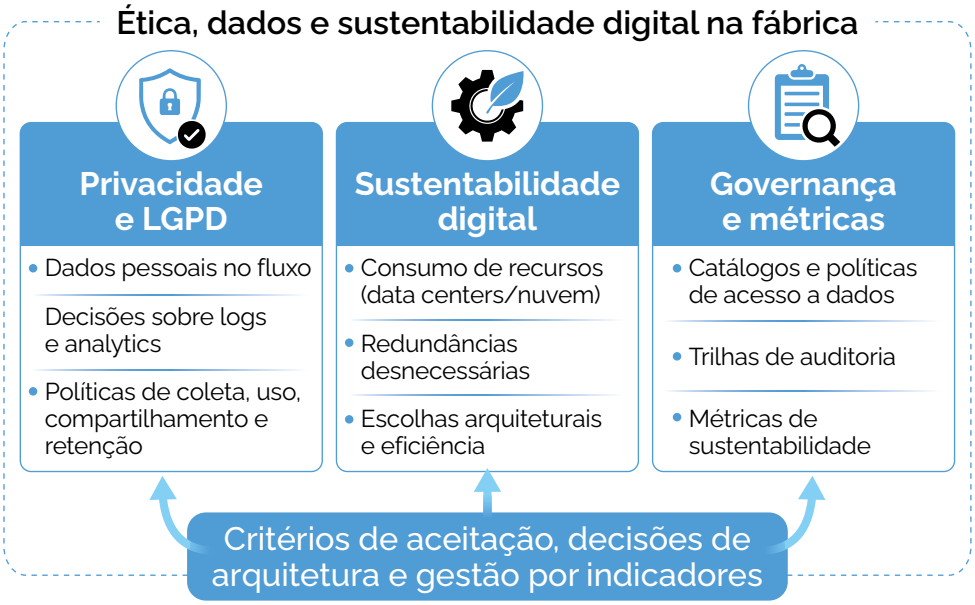


Figura 2 – Ética, dados e sustentabilidade digital na fábrica de software
Fonte: gerada por ChatGPT em 14 jan. 2026.

Descrição da imagem: é exibido um painel escuro com três cartões verticais sob o título "Ética, dados e sustentabilidade digital na fábrica". O primeiro cartão, "Privacidade e LGPD", identificado por um ícone de cadeado, traz textos relacionados à presença de dados pessoais no fluxo, às decisões sobre *logs* e *analytics* e às políticas de coleta, uso, compartilhamento e retenção dessas informações. O segundo cartão, "Sustentabilidade digital", marcado por um ícone de folha ou energia, apresenta textos sobre o consumo de recursos em data centers e na nuvem, a existência de redundâncias desnecessárias e o impacto das escolhas de arquitetura sobre o uso de recursos. O terceiro cartão, "Governança e métricas", identificado por um ícone de checklist, reúne textos referentes a catálogos e políticas de acesso a dados, trilhas de auditoria e métricas de sustentabilidade. Setas conectam os três cartões a uma faixa inferior que contém o texto "Critérios de aceitação, decisões de arquitetura e gestão por indicadores". Fim da descrição.

Nesse contexto, decisões de arquitetura, escolha de linguagens, definição de padrões de integração e estratégias de escalabilidade deixam de ser apenas questões técnicas e passam a incorporar critérios de responsabilidade socioambiental. A adoção de práticas, como monitoramento contínuo de consumo, revisão periódica de dependências, eliminação de serviços ociosos e otimização de consultas e

algoritmos, contribui para reduzir desperdícios e aumentar a eficiência sistêmica. Assim, a sustentabilidade digital não se limita ao discurso institucional, mas se materializa em indicadores acompanhados pela gestão da fábrica, integrando desempenho, custo e impacto ambiental em uma mesma lógica de qualidade.

APROFUNDANDO

A literatura de engenharia de software reforça que requisitos não funcionais – segurança, privacidade, desempenho e eficiência de recursos – devem ser tratados desde o início do ciclo de vida, e não "corrigidos" ao final (Pressman; Maxim, 2016; Sommerville, 2019). Em fábricas com múltiplos clientes e grande volume de dados, isso se conecta à governança de dados (catálogos, políticas de acesso e trilhas de auditoria) e às métricas de sustentabilidade, como consumo de recursos por transação, eficiência de ambientes e vida útil de componentes. Ao integrar ética e sustentabilidade aos critérios de aceitação, revisões de arquitetura e métricas de gestão, essas dimensões passam a orientar decisões reais de projeto e operação.

IA generativa e engenharia de prompts como ferramentas de fábrica

A popularização da **IA generativa** trouxe para a fábrica modelos capazes de gerar texto, código, casos de teste, imagens e rascunhos de documentação a partir de *prompts* em linguagem natural (Russell; Norvig, 2021). Usada com critério, essa capacidade funciona como um "acelerador" para tarefas repetitivas ou intensivas em padrão: apoio à revisão de código, criação de testes de regressão, síntese de incidentes, comparação de alternativas de design e elaboração de rascunhos de requisitos ou de manuais.

Para que isso aconteça de forma segura, surge a **engenharia de prompts**: aprender a formular pedidos claros, fornecer contexto suficiente (arquitetura, padrões e restrições), explicitar o objetivo e o formato da resposta, e registrar o que foi aceito ou descartado (Russell; Norvig, 2021; Bass; Weber; Zhu, 2015). Ao contrário de substituir o papel do engenheiro, a IA se torna um "copiloto" inserido em políticas de uso: dados sensíveis não entram em *prompts*; saídas são revisadas por humanos; decisões relevantes são documentadas; e modelos e ferramentas são avaliados à luz de critérios de ética, privacidade e conformidade (Brasil, 2018).



ZOOM NO CONHECIMENTO

Há diferenças importantes entre IA "tradicional" e IA generativa. Modelos preditivos clássicos (como classificadores ou regressões) são treinados para tomar decisões específicas com base em entradas estruturadas, como priorizar tickets ou detectar anomalias em métricas. Já os modelos generativos produzem novos artefatos (código, texto e imagens) a partir de padrões aprendidos, o que aumenta o risco de erros sutis, vieses e saídas difíceis de explicar (Russell; Norvig, 2021). Em ambos os casos, o uso responsável exige a definição clara de escopo, supervisão humana, políticas para dados de treinamento, monitoramento de desempenho e *logs* que permitam reconstruir como um resultado foi gerado.

Na engenharia de software aeroembarcado, por exemplo, ao trabalhar com módulos de controle e monitoramento embarcados, já vivenciei situações em que a análise manual de grandes trechos de código crítico demandava tempo significativo para identificar padrões repetitivos ou potenciais inconsistências. Um uso criterioso de IA generativa pode apoiar a criação inicial de casos de teste estruturados a partir de requisitos formais ou a comparação entre versões de algoritmos de controle, sempre sob revisão humana rigorosa e dentro das restrições de certificação aplicáveis. Nesse contexto, a IA não decide, nem valida sozinha; ela auxilia na preparação de artefatos que posteriormente passam por verificação, rastreabilidade e validação conforme as exigências do ambiente aeronáutico, reforçando a ideia de "copiloto", e não de substituto do engenheiro.

TENDÊNCIA 1 – ÉTICA E SUSTENTABILIDADE DIGITAL

Envolve decisões sobre privacidade, uso responsável de dados, consumo de recursos e impacto ambiental da infraestrutura, conectando LGPD, arquitetura e métricas de sustentabilidade (Brasil, 2018; Pressman; Maxim, 2016).

TENDÊNCIA 2 – IA GENERATIVA E ENGENHARIA DE PROMPTS

Coloca modelos generativos como apoio a tarefas de código, testes e documentação, exigindo revisão humana, registro de decisões e políticas claras de uso (Russell; Norvig, 2021; Bass; Weber; Zhu, 2015).

TENDÊNCIA 3 – CONFIABILIDADE E SER

Foca em observabilidade, SLIs/SLOs, orçamentos de erro e práticas de SRE para manter a experiência do usuário estável, mesmo em cenários de falhas inevitáveis (Beyer *et al*., 2016).

TENDÊNCIA 4 – PRODUTOS DIGITAIS, VALOR E TALENTOS

Reforça a passagem de projetos pontuais para times de produto orientados a métricas de valor e à retenção de talentos com visão sistêmica (Pressman; Maxim, 2016; Sommerville, 2019).

CONFIABILIDADE, PRODUTOS DIGITAIS E PAPEL PROFISSIONAL

À medida que sistemas se tornam serviços contínuos, a pergunta "funciona?" é substituída por "**funciona sempre para quem precisa?**". Isso desloca o foco de testes pontuais para uma visão de confiabilidade ao longo do tempo, em que observabilidade, resiliência e práticas de Site Reliability Engineering (SRE) ganham protagonismo (Beyer *et al*., 2016; Bass; Weber; Zhu, 2015).

Em paralelo, organizações públicas e privadas migram de estruturas centradas em projetos para **times de produto** de longa duração, responsáveis por acompanhar métricas de valor (adoção, satisfação e impacto em processos) e ajustar continuamente o software ao contexto real (Pressman; Maxim, 2016; Sommerville, 2019).

Confiabilidade além dos testes: observabilidade, resiliência e SRE

A confiabilidade não é apenas "ter poucos *bugs*"; envolve desenhar sistemas que **falham de forma controlada**, recuperam-se rápido e oferecem sinais claros quando algo está errado. A abordagem de SRE propõe conceitos, como SLIs (indicadores de nível de serviço), SLOs (metas de nível de serviço) e orçamento de erro, que conectam métricas técnicas – latência, taxa de erros, disponibilidade e saturação – a compromissos explícitos com usuários (Beyer *et al*., 2016).

Em fábricas modernas, isso se traduz em arquiteturas com mecanismos de *fallback*, degradação elegante, redundância planejada e pipelines que incluem verificações automáticas de saúde após cada *deploy* (Bass; Weber; Zhu, 2015). Para o profissional, significa aprender a ler gráficos e *logs*, participar de análises de incidentes, propor melhorias estruturais e dialogar com times de negócio sobre impactos de indisponibilidade e degradação de serviço. A Figura 3 resume essa conexão entre confiabilidade em produção, produtos digitais guiados por valor e desenvolvimento profissional.

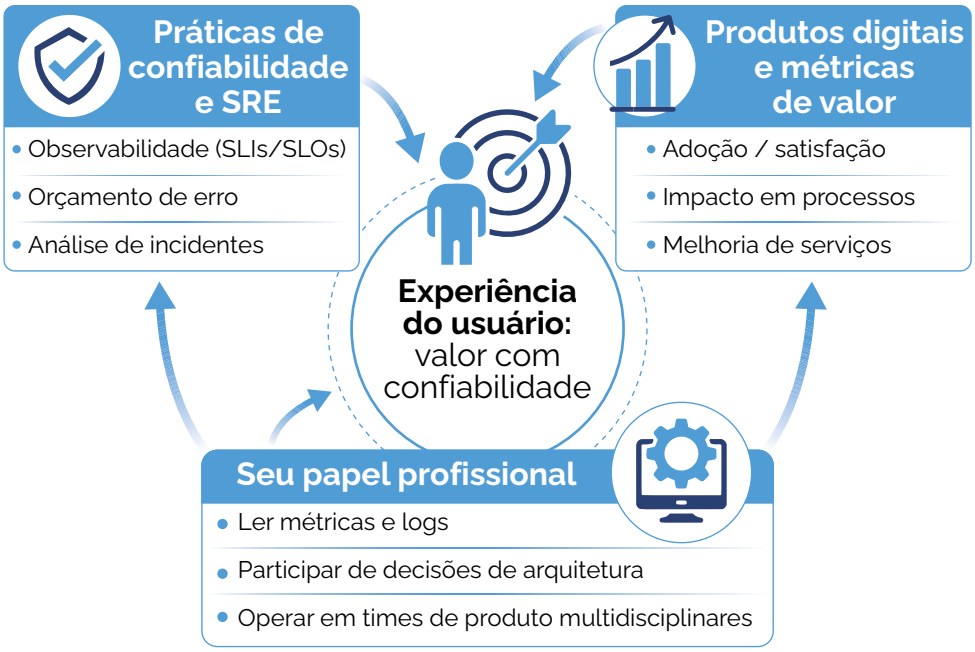


Figura 3 – Confiabilidade, produtos digitais e desenvolvimento profissional
Fonte: gerada por ChatGPT em 16 jan. 2026.

Descrição da imagem: é exposto um painel escuro com um círculo central contendo o texto "Experiência do usuário: valor com confiabilidade". Três cartões coloridos estão conectados a esse círculo por meio de setas. O cartão à esquerda traz o título "Práticas de confiabilidade e SRE" e inclui os textos "observabilidade (SLIs/SLOs)", "orçamento de erro" e "análise de incidentes". O cartão à direita apresenta o título "Produtos digitais e métricas de valor" e contém os textos "adoção", "satisfação", "impacto em processos" e "melhoria de serviços". Na parte inferior, há um cartão com o título "Seu papel profissional", que reúne os textos "ler métricas e logs", "participar de decisões de arquitetura" e "operar em times de produto multidisciplinares". Setas conectam os três cartões ao círculo central, indicando a relação entre esses elementos e a experiência do usuário baseada em valor e confiabilidade. Fim da descrição.

Na prática, isso implica incorporar rotinas simples, porém consistentes, como revisar indicadores após cada entrega, registrar causas e ações corretivas de incidentes e acompanhar tendências de desempenho ao longo do tempo. Pequenas melhorias contínuas – como ajustar limites de alerta, simplificar integrações ou otimizar consultas – reduzem riscos acumulados e fortalecem a estabilidade do produto, conectando operação técnica, decisão gerencial e geração de valor para o usuário final.

PENSANDO JUNTOS

Quando uma equipe define SLOs realistas, monitora SLIs em tempo quase real e respeita orçamentos de erro, as decisões de mudança deixam de ser intuitivas e passam a se apoiar em evidências: pode-se, por exemplo, adiar uma nova *feature* para estabilizar o sistema quando o orçamento de erro está quase estourado, ou aceitar maior risco em períodos de menor uso (Beyer *et al.,* 2016). Integrar essa lógica à fábrica – independentemente de ser interna, terceirizada ou híbrida – ajuda a alinhar produto, operação e negócio em torno de um objetivo comum: manter a experiência do usuário em um nível previsível e aceitável, mesmo frente a falhas inevitáveis.



Engenharia de confiabilidade de software (SRE) na prática

A abordagem de SRE propõe medir e gerenciar a confiabilidade com base em indicadores concretos. SLIs (indicadores de nível de serviço) descrevem o que está sendo observado – como latência, taxa de erros, disponibilidade –; SLOs (metas de nível de serviço) estabelecem patamares aceitáveis para esses indicadores; e o orçamento de erro delimita a quantidade de desvio que é tolerável em determinado período (Beyer *et al*., 2016). Ao contrário de decisões baseadas apenas em intuição ou urgência política, mudanças em produção passam a considerar se o orçamento de erro permite assumir mais riscos ou se é hora de priorizar a estabilidade.

Na arquitetura, isso se traduz em mecanismos de *fallback*, degradação elegante, redundância planejada e prática de testes de resiliência (*caos engineering*). Em pipelines, significa incluir verificações de saúde pós-*deploy*, *rollback* automatizado e registros que facilitem a análise de incidentes (Bass; Weber; Zhu, 2015). Para você, profissional, engenharia de confiabilidade implica aprender a ler métricas e *logs*, participar de *post-mortems*, sugerir melhorias estruturais e dialogar com áreas de negócio sobre o impacto real de falhas e degradações de serviço.

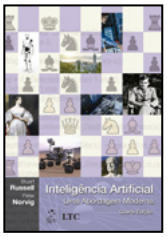
Fábricas orientadas a produtos, valor e talentos

A combinação de ética, sustentabilidade, IA generativa e confiabilidade influencia diretamente a forma como times são organizados. Cada vez mais, fala-se em fábricas orientadas a produtos digitais, nas quais *squads* multidisciplinares cuidam de um serviço ou jornada de ponta a ponta, com autonomia para decidir prioridades e responsabilidade por métricas de valor – adoção, satisfação, tempo economizado, redução de incidentes e melhoria de políticas públicas ou de resultados de negócio (Pressman; Maxim, 2016; Sommerville, 2019).

INDICAÇÃO DE LIVRO

Inteligência artificial: uma abordagem moderna

Para aprofundar os seus estudos em riscos e possibilidades da IA, especialmente em contextos que exigem confiabilidade e transparência, a obra de Russell e Norvig é uma referência essencial. Os autores apresentam fundamentos de decisão, aprendizagem e busca, oferecendo base teórica para avaliar a adequação de soluções de IA e seus limites éticos e técnicos em fábricas de software modernas.

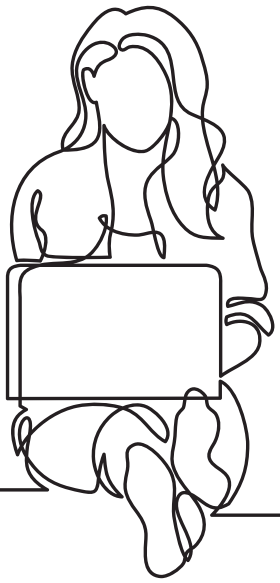


Para sustentar esse modelo, torna-se crítico atrair e reter talentos que combinem domínio técnico com visão sistêmica: pessoas capazes de dialogar sobre arquitetura, dados, riscos, experiência do usuário e impacto social ao mesmo tempo (Bass; Weber; Zhu, 2015). Nesse cenário, o seu desenvolvimento profissional deixa de ser apenas "acumular tecnologias no currículo" e passa a envolver posicionamento consciente: em que tipo de fábrica de software você quer atuar, com quais princípios de ética e sustentabilidade se compromete e o quanto deseja se aproximar de temas, como IA, observabilidade, produtos digitais e transformação de serviços públicos ou privados. As tendências apontam que o futuro da área não será apenas mais rápido ou automatizado, mas também mais cobrado em termos de responsabilidade, transparência e capacidade de aprender com dados e a sociedade em que o software opera.

Transformação digital e futuro da engenharia de software

A fábrica de software assume um papel estratégico na transformação digital de organizações públicas e privadas ao integrar processos, arquitetura, métricas e governança em uma lógica orientada a valor contínuo. A transição de projetos para produtos digitais exige foco em experiência do usuário, confiabilidade em produção, sustentabilidade e uso ético de dados e IA. Nesse cenário, a atração e a retenção de talentos se tornam fatores críticos, demandando cultura de aprendizado, autonomia técnica e propósito organizacional. O futuro da engenharia de software aponta para fábricas orientadas a dados, responsabilidade socioambiental e geração sustentável de impacto.

Essa evolução também redefine o perfil profissional esperado: engenheiros deixam de atuar somente como implementadores de funcionalidades e passam a assumir papel analítico e estratégico, capazes de interpretar métricas, avaliar riscos tecnológicos, dialogar com áreas de negócio e tomar decisões arquiteturais conscientes. Assim, a fábrica do futuro não será apenas um centro de produção de código, mas um ecossistema digital que conecta inovação, governança, ética e desempenho operacional em benefício da sociedade e das organizações.



EM FOCO

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de lhe indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema.

NOVOS DESAFIOS

Ao chegar ao fim deste tema, é perceptível que falar em fábricas de software não é mais discutir somente processos, ferramentas ou modelos de contratação. Ética, sustentabilidade digital, IA generativa, engenharia de *prompts*, confiabilidade (SRE/observabilidade) e orientação a produtos passam a definir o modo como as organizações serão avaliadas por clientes, reguladores e pela própria sociedade. Isso significa que o seu ambiente profissional será cada vez mais cobrado por transparência no uso de dados, responsabilidade em decisões automatizadas, cuidado com o impacto ambiental da infraestrutura e capacidade de manter serviços estáveis, mesmo em cenários de alta complexidade.

Imagine um portal público de agendamento de serviços de saúde em que dados, como nome, CPF, telefone e histórico de atendimentos, são tratados conforme políticas alinhadas à LGPD, com controle de acesso e registros de auditoria. A IA generativa pode apoiar na criação de casos de teste e na elaboração de documentação, desde que haja revisão humana obrigatória, anonimização de dados em *prompts*, registro das decisões e validação antes do *deploy*. A observabilidade é garantida por SLIs, como taxa de erro, latência e disponibilidade; se algo falhar, mecanismos de *fallback* e *rollback* reduzem o impacto, enquanto *deploy* gradual e orçamento de erro controlam riscos de mudança. O sucesso é avaliado não somente pela entrega, mas pelo aumento de agendamentos concluídos, redução de filas, maior satisfação do usuário e menos incidentes na produção.

No mercado de trabalho, essas tendências se traduzem em oportunidades e desafios concretos. Empresas buscarão profissionais capazes de dialogar sobre arquitetura e código, mas também sobre viés algorítmico, LGPD, SLAs de confiabilidade, métricas de valor e jornada do usuário. Times orientados a produto esperam que engenheiros de software, especialistas em dados, SREs e analistas de negócio tomem decisões técnicas alinhadas a princípios éticos e a compromissos de sustentabilidade. Seu diferencial estará em conectar o que estudou aqui – conceitos, práticas, riscos e boas práticas – com situações reais de entrevistas, projetos, estágios e atuação profissional, posicionando-se como alguém que não apenas domina tecnologias emergentes, mas contribui para que as fábricas de software do futuro sejam mais responsáveis, sustentáveis e centradas em valor.

1. Decisões sobre dados afetam arquitetura, processos e indicadores. Em ambientes regulados, é necessário explicitar papéis, bases legais e controles, além de monitorar impactos operacionais e ambientais. A utilidade das medidas depende de coleta e uso consistente no tempo.

   Qual alternativa melhor traduz as práticas éticas e regulatórias no uso de dados?

   a) Aceites informais sobre dados pessoais.
   b) Definir controlador/operador, base legal, políticas de acesso e registros de auditoria.
   c) Usar dados reais em todos os testes sem restrições.
   d) Delegar decisões de privacidade apenas ao fornecedor.
   e) Substituir medidas por declarações de boa-fé.

2. Ferramentas generativas podem apoiar tarefas de texto, código e testes. Para operar com segurança, é útil definir escopo, revisão humana e registro do que foi aceito. O objetivo é potencializar ganhos mantendo qualidade, privacidade e auditabilidade.

   Qual prática é mais adequada ao uso de IA generativa na fábrica?

   a) Aceitar toda saída gerada sem revisão.
   b) Revisão humana, trilhas de decisão e critérios de aceitação antes de incorporar saídas.
   c) Misturar dados pessoais em *prompts* sem restrições.
   d) Ocultar o uso de IA em auditorias.
   e) Proibir *logs* para acelerar.

3. A confiabilidade implica projetar para operar sob falhas controladas, com metas explícitas e visibilidade contínua do serviço. Indicadores técnicos e decisões de produto precisam convergir. O arranjo deve facilitar diagnóstico, recuperação e priorização baseada em impacto.

   Qual prática melhor reflete confiabilidade orientada por SRE?

   a) Apenas testes pré-*deploy*.
   b) Operar sem rastreabilidade.
   c) Reagir apenas a reclamações.
   d) SLIs/SLOs, orçamentos de erro e verificações automáticas pós-*deploy* com *rollback*.
   e) Proibir experimentos de resiliência.

REFERÊNCIAS

BASS, L.; WEBER, I.; ZHU, L. **DevOps**: a software architect's perspective. Boston: Addison-Wesley Professional, 2015.

BEYER, B. *et al*. **Site reliability engineering**: how Google runs production systems. Sebastopol: O'Reilly Media, 2016.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, [2018]. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 11 mar. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

RUSSELL, S.; NORVIG, P. **Inteligência artificial**. 4. ed. Rio de Janeiro: LTC, 2021.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

CONFIRA SUAS RESPOSTAS

1. Alternativa B. A alternativa B é a correta, pois explicita os papéis, as bases legais e as evidências.

   As demais alternativas estão incorretas:

   a) Não atende a requisitos formais.
   c) Ignora a minimização/mascaramento.
   d) A responsabilidade permanece compartilhada/clarificada.
   e) Falta verificabilidade.

2. Alternativa B. A alternativa B é a correta, pois garante revisão, rastreio e critérios objetivos.

   As demais alternativas estão incorretas:

   a) Elimina salvaguardas.
   c) Há risco à privacidade.
   d) Fere a transparência.
   e) Sem *logs,* não há auditoria/melhoria.

3. Alternativa D. A alternativa D é a correta, pois integra metas, observabilidade e resposta a falhas. A alternativa A está incorreta, porque a confiabilidade não é só pré-*deploy*. Já as alternativas B, C e E estão incorretas por reatividade, falta de rastros e ausência de práticas de resiliência.

MEU ESPAÇO

MEU ESPAÇO